claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_2a64bc88-2a8\agent-a470a64c55d7a42a5.jsonl`
- SHA-256: `8eba2a0758ba29695d955b826acc425b59b6df5f657a0d173ce8347b8f90f14b`
- Source modified: `2026-06-10T05:41:39+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_2a64bc88-2a8`
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`

Transcript

1. user (2026-06-10T05:33:14.936Z)

```
CONTEXT ? two sibling repos on a Windows machine:
- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? local AI badminton(->racket-sports) highlight indexer + rally detector. FastAPI + SQLite + ffmpeg + GPU CV via WSL; Gemini = paid cloud AI. docs/ tree is rich. Current branch docs/next-steps-multisport (master = main branch).
- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? golden-label acquisition/curation/ingestion CLI (system-of-record candidate for training corpus).
Project state (June 2026): scaffolding complete; owned-model roadmap agreed (docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md) but NO platform/owned-model implementation started; next committed platform slice = async job model. Licensing guardrail: MIT/Apache-2.0/BSD only for code+data+weights; paid LLMs are inference-only, never training-data sources.
ENVIRONMENT RULES: this dev environment has NO GPU/torch/cv2 ? do NOT attempt to run the video pipeline or import torch/cv2 modules. Test suites are offline/fully-mocked and SAFE to run. You are READ-ONLY: never edit/create/delete repo files; running tests and git read commands is allowed.
QUALITY BAR: cite file paths (and line numbers where possible) for every finding. Report what IS in the code/docs, not what you assume. Be skeptical and concrete. Your final structured output is consumed programmatically by an orchestrator ? return raw data, not prose for a human.
TASK: STRATEGY-DOC CONSISTENCY sweep of the indexer. Read FULLY, in this order: CLAUDE.md (repo root), docs/README.md, docs/NEXT_STEPS.md (new, on current branch), docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md, docs/ROADMAP.md, docs/DEFERRED_HARDENING_PLAN.md, docs/BACKLOG.md, docs/COMMERCIALIZATION.md, docs/PLATFORM_ARCHITECTURE.md, docs/PMF_MARKET_RESEARCH.md, docs/archives/decisions/DECISIONS.md, docs/archives/checkpoints/2026-06-strategy-foundation/README.md, docs/archives/research/OWNED_MODEL_TRAINING_STUDY.md (at least its correction banner + conclusions).
FIND, as findings: (a) CONTRADICTIONS between docs ? e.g. licensing policy wording (CLAUDE.md says 'open Apache-2.0 only' for training teachers/bases; later ratified policy is 'MIT/Apache-2.0/BSD' ? where do docs disagree?); what is 'the next slice' (async job model vs Phase-0 M0.x vs repo rename ? do docs agree on sequencing and on who decides?); v1 scope definitions; Gemma-4 stance (AMBER/bench-only vs candidate); WASB checkpoint provenance stance across docs; conversational-QA punt-vs-build framing. (b) CLAIMS CONTRADICTED BY REPO REALITY ? verify with Grep/Glob: does training.label_candidates still carry Gemini-derived labels the plan says must be purged (check backend + DB-related code/docs for what actually exists ? note output/ DB itself is gitignored so check code paths + docs claims)? does config.json default_segmenter match what docs claim? do doc-referenced files/flags exist (e.g. output/human_lovo_manifest.json is gitignored ? do docs point at artifacts a fresh clone won't have)? does the repo-rename action item appear anywhere actionable? (c) ROADMAP GAPS / missed gotchas ? sequencing conflicts (async-job slice vs Phase-0 both 'next'), undefined ship-gate F1@tIoU target, cost-to-Gen-2 unquantified, per-sport detector blocker vs the new finding that WASB transfers to table tennis (docs/NEXT_STEPS.md) ? is that finding consistently reflected or does it contradict older 'binding blocker' framing?
Each finding must QUOTE both sides (doc A says X at path:line-ish; doc B / code says Y).
```

Severity: high = a reader following docs would act wrongly; medium = confusing drift; low = cosmetic.

2. user (2026-06-10T05:33:20.644Z)

```
# CLAUDE.md ? project memory / agent handoff

Local, AI-powered **badminton (? racket-sports) highlight indexer + rally detector**:
FastAPI + SQLite + ffmpeg + GPU CV/AI, with paid **Gemini** as the cloud AI. Sibling
repos: `sports-data-collector`, a VLC-based `rally-annotator`.

## Read these first
- **`docs/README.md`** ? doc index. **`docs/CODE_MAP.md`** ? code navigation + data
  contracts (the cold-start map; read before touching code).
- **Strategy (post-scaffolding):** `docs/COMMERCIALIZATION.md`, `docs/PMF_MARKET_RESEARCH.md`,
  `docs/PLATFORM_ARCHITECTURE.md`, `docs/DEFERRED_HARDENING_PLAN.md`. Also `docs/ROADMAP.md`
  (offline-segmenter narrative) and `docs/BACKLOG.md`.
- **History:** `docs/archives/` ? ADRs (`decisions/DECISIONS.md`), research, and
  **`checkpoints/`** (latest: `2026-06-strategy-foundation/`). Archive policy: only move
  **terminal-state** (DONE/REJECTED) work; live docs stay at `docs/` top level.

## Current state (June 2026)
- **Scaffolding complete**; the **post-scaffolding strategy foundation** is laid and merged
  (#52/#53/#55/#57/#58). **No platform/owned-model implementation has started.**
- **Next slice = async job model (`DEFERRED_HARDENING_PLAN.md` #16), single-tenant.**
  **Do not start implementation without explicit owner approval.**

## Architecture in one breath
- **Pluggable registries** (ABC + `Registry` singleton): segmenters / providers / validators
  / sports / annotations ? see `backend/pipeline/segmenters/base.py`. *Future:*
  `StorageBackend` + `ComputeBackend` registries (PLATFORM §3).
- `video_id` = MD5 of the file ? `backend/utils/hashing.py::compute_video_id` (one helper).
- Typed config = `backend/config/` (Pydantic). DB schema evolves via the `PRAGMA
  user_version` migration ladder in `backend/infrastructure/database.py`.

## Committed guardrails (non-negotiable)
- **Paid LLMs (Gemini/OpenAI/Anthropic) = inference / serving / fallback ONLY ? never a
  training data source** (terms/copyright). Training teachers/bases = **open Apache-2.0 only**.
- **Owned-model base:** Apache-2.0, video-capable (Qwen-VL / Gemma 4); **reject Gemma 3**
  (viral license). **Exclude minors** from the training pool; per-user training data needs a
  separate explicit opt-in.
- **v1 = rally + highlight + history + correction loop**; **gait/biometrics strictly future**
  (GDPR Art. 9).
- **Generic data schema**; `coach ? athlete` is an **optional overlay**, not baked in.

## Workflow conventions
- **Branch off `master`; ONE PR per work item; NO PR stacking.** Don't open a PR unless asked.
- Tests: `python run_all_tests.py` (offline, fully mocked). Python 3.8+; `ffmpeg` required.
- Never push to the default branch directly; descriptive commit messages.
- ?? This dev container lacks GPU / torch / cv2 / numpy ? the **full pipeline can't run here**.
  Verify pure-logic paths and hand machine-side checks (real video, GPU/WSL suite) to the owner.

```

3. user (2026-06-10T05:33:21.463Z)

```
# docs/

Live documentation for the badminton-highlight-indexer.

## For new contributors / ramp-up (read these first)
- [CODE_MAP.md](CODE_MAP.md) ? Complete navigation map of the codebase and data contracts.
- [COMMERCIALIZATION.md](COMMERCIALIZATION.md) ? Living tracker for the commercial dimension (licensing, codec/output economics, hosting, vendoring, packaging) + PMF risks and the quality/data-phase priorities. Supersedes the archived `COMMERCIAL_READINESS_REVIEW.md`.
- [PMF_MARKET_RESEARCH.md](PMF_MARKET_RESEARCH.md) ? Cited market study: sizing, customer segments, competitor pricing, and the recommended beachhead (feeds COMMERCIALIZATION §6).
- [PLATFORM_ARCHITECTURE.md](PLATFORM_ARCHITECTURE.md) ? Strategy for the multi-user "assistant coach" platform: decoupled layers + pluggable storage/compute (BYO), multi-tenant data model, orchestration, and a phased roadmap.
- [DEFERRED_HARDENING_PLAN.md](DEFERRED_HARDENING_PLAN.md) ? Design (pending approval) for the larger deferred items: packaging, train/eval guardrails, and the async job model (#16, the first infra slice toward the platform).
- [BACKLOG.md](BACKLOG.md) ? Deferred items with rationale and revisit triggers.
- [ROADMAP.md](ROADMAP.md) ? High-level four-phase product direction.
- [VIDEO_RECORDING_GUIDELINES.md](VIDEO_RECORDING_GUIDELINES.md) ? Capture best practices (referenced by reports).
- [HOW_RALLY_DETECTION_WORKS.md](HOW_RALLY_DETECTION_WORKS.md) ? Mental model of the two-layer detection system.
- [EVALUATION.md](EVALUATION.md) ? How we measure and improve quality.

## Practical / ongoing
- [TRACKNET_WSL_SETUP.md](TRACKNET_WSL_SETUP.md)
- Golden set docs (`GOLDEN_SET_*`)
- Evaluation harness and calibration drivers (in `backend/eval/`)

## Historical documents
Completed plans, old research, ADRs, and checkpoints live in [`archives/`](archives/).

See `archives/README.md` for the umbrella policy and `archives/past_projects/README.md` for classification of small tech debt / starter projects.

```

4. user (2026-06-10T05:33:26.220Z)

```
# Next steps (live) ? owned-model + multisport

**Updated:** 2026-06-09 (end of the quality-iteration + multisport-recon session). **Authoritative roadmap:**
`docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md`. **Latest blow-by-blow:** memory `current-state.md` / `session-handoff.md`.
This file = the prioritized "what to do next," refreshed each session.

## The recommendation (one paragraph)
Build the **quality-first extraction harness** before anything else: a learned **TAL head** (ActionFormer/ASFormer, MIT)
over trajectory features is the robust path ? it **decouples from the multi-court "background games" confound** that the
unsupervised heuristic (and audio, and spatial court-gating) can't escape (all three tested + failed; it's a *temporal*
rally-vs-non-rally problem). **Framework = ms-swift, base = Qwen3-VL** (Gemma-4 AMBER/bench) come **later** for the
conversational/grounding upgrade ? start with the head. **On-device is deferred** (export-clean ops now so it stays
reachable). Everything stays **MIT/Apache-2.0/BSD** (code + data + weights). Bottleneck = **data + per-sport trajectory
detectors**, not model architecture.
```

```
## Latest findings (2026-06-09)
- **6-match badminton LOVO:** learned prototype **0.583** vs heuristic **0.480** (learned leads since n=3, AUC 0.83?0.92,
  decoupled from multi-court). The "contrast" metric (in-rally÷dead-time visibility) predicts heuristic F1 monotonically.
- **?? Multisport (Roora pickleball + TT):** labels **ingest cleanly** (`curate convert-cvat` ? 6 rallies/sport, sane).
  **WASB-transfer SURPRISE (directional, n=6 each ? TINY):** the badminton shuttle detector segments **TABLE TENNIS at
  F1 0.909** out-of-the-box (!) but **pickleball only 0.308**. ? **The per-sport-detector blocker is SPORT-DEPENDENT:** TT may
  be servable with WASB short-term; pickleball needs a proper (MIT/Apache) ball detector. Confirm/deny with more videos.
- **Owner field notes (2026-06-09):** camera height/distance **varied on purpose** (great generalization data); **multi-court
  adjacent-court SOUND dominates the signal** ? empirically reconfirms audio-foreground isolation won't work + the
  robustness-first learned-model path. Multi-court is the realistic (academy) environment, so this *is* the target distribution.

## Prioritized next steps
1. **[owner greenlight] Phase-0 (M0.x), $0/local:**
   - **M0.4 ? Gen-0 TAL harness** (first-class): productionize `rally_seq_proto.py` ? one-command train?eval?**ship-gate** over
     ActionFormer/ASFormer, reusing `metrics.match_intervals`. **Define the ship-gate target** (explicit F1@tIoU; floor = beat
     heuristic LOVO 0.480) ? "very high P/R" must be operationalized, not just "beat the floor."
   - **M0.1 ? corpus spine + the per-video event-index contract** (the conversational seam: extract ? event store ? text-to-SQL).
   - **M0.3 ? compliance:** purge Gemini-derived training labels (`training.label_candidates`); WASB **C3** own-weights/clean-swap (gates ship).
2. **Multisport thread:**
   - Ingest the Roora pickleball/TT rally CSVs into the corpus (collector `import-local`).
   - **Get more pickleball + TT videos** ? re-run the WASB-transfer eval to confirm/deny TT?0.91, pickleball?0.31 (n=6 is too few).
   - Once each sport has ?3 matches, run a **merged prototype LOVO across badminton + TT (+ pickleball)** ? the "how would a
     merged model do" experiment. Identify a clean MIT/Apache ball detector for sports where WASB doesn't transfer (pickleball).
3. **Housekeeping:** **rename repo off "badminton"** (multisport); confirm collector = corpus/event-store system-of-record.

## Pointers / artifacts
- Roadmap: `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md` · Study: `docs/archives/research/OWNED_MODEL_TRAINING_STUDY.md`
  (2026-06-09 banner) · Quality: `docs/archives/quality-iterations/2026-06-multivideo-lovo/`.
- Eval corpus: `output/human_lovo_manifest.json` (badminton n=6). Roora CSVs: `%TEMP%\roora\{pickleball,tabletennis}_rallies.csv`;
  WASB-transfer trajectories: `%TEMP%\{pickleball,tabletennis}_traj.csv`. Code: `backend/eval/{eval_partial_labels,rally_seq_proto,calibrate_local}.py`.
- ? No owned-model implementation until owner greenlights a milestone. Committed next PLATFORM slice = async job model (single-tenant).

```

5. user (2026-06-10T05:33:26.547Z)

```
# Owned-model implementation plan (roadmap) ? DRAFT for owner review

**Status:** PLAN / roadmap. No owned-model *implementation* starts until the owner greenlights a specific
milestone. Paid Gemini stays the always-ready inference/fallback throughout (never a
```

training source).
Grounded in: `docs/archives/research/OWNED_MODEL_TRAINING_STUDY.md`, the n=6 LOVO + learned prototype
(`docs/archives/quality-iterations/2026-06-multivideo-lovo/`), and a verified strategy workflow (2026-06-09).

Design principle (owner directive): *envision now, build later*
Every deferred piece (conversational-QA, on-device, multisport, the VLM) must be **architecturally envisioned
now** so we never hit a mammoth refactor. We **design the seams** (data contracts, export boundaries, the
event store) up front and **implement behind them** when each milestone is greenlit. Punting a *feature* is fine;
punting its *seam* is not.

North Star (reframed per PR #61 review)
- **Immediate deliverable:** a **training framework + harness** that produces a model with **very high precision/
  recall for extracting highlights + rallies from a clip**. *Quality is the first-order objective.*
- **Eventual product:** a **single, sport-agnostic, conversational** video model ? upload a clip, then ask
  contextual questions ("longest rally", "make a clip of all service faults", "rallies over 20 shots").
- **On-device is LATER** (a model-compression goal so power users save upload cost/hassle) ? definitely on the
  roadmap, but **not** the immediate target. Don't over-index the strategy on it now.
- **The moat = the consented dataset + the eval harness, not the weights.**

Hard guardrails (non-negotiable ? "never corrupted")
- **Commercial-clean licensing for EVERY dependency ? code, data, AND model weights: MIT / Apache-2.0 / BSD only.**
  (Ratifies the prior "Apache-2.0 only" to include MIT/BSD.) Reject viral/NC/custom-restrictive: Gemma 3 (viral),
  Gemma 3n (Gemma Terms follow derivatives ? even cleaner reject), Llama 4 (700M-MAU cap), Commons-Clause repos.
- **Never train on paid-LLM (Gemini/OpenAI/Anthropic) outputs.** Also forbidden: public grounding-CoT datasets that
  were Gemini-generated (e.g. TVG-R1's 56K) ? we must originate our own CoT supervision (a real, binding cost).
- **Exclude minors; per-user training data needs a separate explicit opt-in; split data by match.**
- ? **Base-model pretraining provenance is unauditable for *every* open VLM (incl. Qwen3-VL).** Our "no paid-LLM
  training" rule is enforceable at *our fine-tuning data* layer, not the base's pretraining ? an **explicit owner
  risk-acceptance** is required to use any open VLM. (Governed by the base's license, not our pipeline.)

The model strategy (verified 2026-06-09)
Framework ? base (the Gemma-4-vs-ms-swift clarification): a *framework* is the forge (runs SFT/GRPO, emits
weights); a *base* is the metal you forge. You pick one framework and feed it any clean base.
- **Framework: ms-swift (Apache-2.0)** ? the one forge that does native **video + audio + timestamp-grounding +
  GRPO/RFT with a custom video reward** in one stack. Keep **unsloth** (Apache-2.0 core; *avoid its AGPL Studio UI*)
  as the 8 GB-local SFT/compression tool for the deferred on-device goal. ? Point-in-time snapshot ? re-verify at impl.
- **Base (when we reach the VLM): Qwen3-VL (Apache-2.0, uniform across sizes)** ? long-video context, purpose-built
  timestamp emission (second-level localization), multi-turn video chat. Re-confirm the exact size's LICENSE
  pre-release. **Gemma-4 stays AMBER and bench-only** ? its Apache grant is likely but

its Prohibited-Use posture is
unconfirmed (counsel needed), *and* its video/timestamp capability is disputed; not the primary extractor.
InternVL3 (MIT/Apache) = clean fallback if a Qwen blocker appears. ? base swap is cheap *only within
timestamp-capable Apache bases* (timestamp data-format + grounding-CoT are base-specific).
- **START WITH THE TAL HEAD, NOT THE VLM** (the decisive call): fine-tuned VLMs **miss strict temporal boundaries**
(best RL-post-trained video VLM ? R@0.7 50% on open-domain benchmarks; "coarse regions, not accurate boundaries").
Boundary accuracy IS the product. The head (`rally_seq_proto.py`) is already de-risked on our corpus (held-out
AUC 0.83?0.92), trains in minutes at ~4.5 GB on the 2070 Super ($0), is ~1M params (trivially compressible later),
and **doubles as a pseudo-label teacher + the honest baseline the VLM must beat.** So: **head now ? VLM later**,
strictly sequenced. When the VLM comes, prefer **two-stage** (VLM coarse-proposes, head refines boundaries) over
wholesale replacement. *(Open: the VLM-ceiling number is open-domain; our amateur footage may differ ? re-measure.)*

Conversational video-QA ? build the bookkeeping NOW, punt the model
Punt the conversational *feature* until extraction quality is solid (current LOVO ~0.583 ? "very high"; a chat UI
over a weak extractor exposes the weakness). **Reject end-to-end-VLM-answers-each-turn** (expensive; bad at exact
counting; the strong teachers are license-forbidden). **But build the *seam* now** (the brief requires it):
- **Architecture = "extract once, query many":** EXTRACTION (TAL head ? later VLM) emits structured timestamped
spans ? a durable **per-video EVENT STORE** ? a **structured QUERY layer** (text-to-SQL + ffmpeg clip-cut). The
cited queries ("longest rally", "all service faults", "rallies > 20 shots") are **structured-numeric ? SQL, not a
VLM job** (SQL counts exactly where VLMs hallucinate).
- **Designed-now seam:** make the **per-video event-index schema a first-class data contract** alongside the M0.1
corpus spine: per rally `{video_id(MD5), rally_ordinal, sport, start, end, shot_count, ending_reason/fault_tags,
derived_clip_ref, provenance, confidence}`. **Reserve the event/fault-tag taxonomy (e.g. `service_fault`) up front**
even if unpopulated. ? This is a real DB delta (a `PRAGMA user_version` migration): `play_segments` today has
`video_id/start_time/end_time/duration/server/receiver/metadata` only; `highlights` isn't a table yet ? so it's an
extension with new columns + a highlights table, not a no-op. **Punt** the NL parser/agent until quality clears the bar.
- Paid Gemini may answer genuinely open-ended edge questions **at inference** over our structured index (allowed ?
it reasons over our data, it is not trained on its outputs).

Multisport (single sport-agnostic model)
- **Model/framework: UNCHANGED.** One shared TAL head + one Qwen3-VL base, fine-tuned across sports (sport = an input
feature / per-sport calibration). The collector + indexer are already sport-generic. ? **ACTION: rename the repo off
"badminton" ASAP** (owner emphatic) ? consistent with this.
- **What multisport changes = DATA + DETECTORS (scale up, the accepted "good problem"):** each new sport needs its own
labeled matches (split by match) + a **clean trajectory detector** to feed the head's features. **The binding blocker
is per-sport detectors:** WASB (badminton) has the unresolved **C3** checkpoint-provenance issue, and **clean MIT/
Apache/BSD shuttle/ball detectors for tennis/TT/pickleball/padel are not yet

identified.** Sequence by public-data richness: **badminton ? tennis/TT ? pickleball/padel.**
- ? **Single-model-multisport-temporal-generalization is an OPEN HYPOTHESIS**, not proven ? our corpus is badminton-only, and the evidence once cited for it (RacketVision) was *refuted on verification* (it's ball-tracking, not temporal). The concept is sound on first principles (rally signal = a racquet-sport invariant) but must be validated once a 2nd-sport corpus exists. Don't cite RacketVision as evidence.

Language ? Python everywhere; on-device runtime is a deferred, layer-local export shell
- **Training/eval = Python-locked, and that's fine** (PyTorch/ms-swift/ActionFormer); offline/batch, no latency hot path.
- **Platform = no Python hot path to escape** ? the heavy bytes/GPU already run out-of-process (ffmpeg, WASB-in-WSL, cloud GPU per-job); the control plane is I/O-bound. **Reject a Go/Rust hot-path split** ? single-digit-ms wins dwarfed by ~40-min GPU jobs, and it would fracture the one codebase holding the QA-state + eval harness + provider registry.
- **On-device (deferred) = the only place a 2nd language appears, and it doesn't dictate our dev language:** train in Python ? **export across the boundary** (ONNX/Core ML/ExecuTorch/GGUF) ? a thin device shell (Swift/Kotlin/C++/Rust) runs the artifact with zero Python. **The one concrete constraint now: train with export-clean ops** so the compression target stays reachable without re-architecting. (On-device base stays on the Qwen3-VL-4B Apache lineage ? *not* Gemma-3-270M.)
- Escalation order if a real Python CPU hot path ever appears: free-threading ? numpy/vectorize ? Cython/PyO3 ? only then a separate service. **None exists today.**

Phase 0 ? Foundation (start now, per-milestone greenlight; $0, local, no cloud/DPA)

M0.1 ? The labeled-corpus spine **+ the event-index contract** (the moat) · *greenlight item*
Canonical JSONL row per rally (corpus) **and** the queryable per-video event-index schema (the conversational seam) ? same row, wired to a store. Fields: `{video_id(MD5), rally_ordinal, sport, start, end, ending_reason/fault_tags, shot_count, label_type, source(human|pseudo|vendor), consent/training_opt_in, split(BY MATCH), trajectory_ref, labeled_window, derived_clip_ref, confidence}`. 3 transcoders (heuristic/LogReg · TAL ActivityNet-JSON · VLM clip?QA), one scorer spine (`metrics.match_intervals`). Authored in `sports-data-collector` (system-of-record; extends PR #13). **Reserve fault/event taxonomy now.** Acceptance: round-trip test; split-by-match enforced; **no Gemini-sourced rows in any training view**.

M0.2 ? Grow the corpus · **DONE (n=6), running** · *owner action + my scoring loop*
6 distinct matches labeled (3 singles incl. Mahadevpura, 1 doubles, 2 multi-court); per-match + LOVO recorded; the "contrast" data-quality metric validated. Owner adding 1 more varied multi-court-club match ? margin. **Bottleneck = data + calibration, not method.** Keep growing per new sport.

M0.3 ? Compliance cleanup (**hard blocker ? do before any training run**) · *greenlight item*
- **(i) PURGE the Gemini-derived TRAINING labels** in `training.label_candidates` (over Gemini CSVs) ? a **live no-paid-LLM-training violation** that bakes in the moment a training run precedes it. Retarget to human + open-teacher (own-model/WASB) labels; Gemini = eval/reference/fallback only.

- **(ii) WASB C3 ? first-class action item (owner):** the academic-data checkpoint provenance is a **commercial-ship
  blocker** that a clean head does NOT launder. Resolve via **own-trained weights** or a clean detector swap; **multiplies
  per sport.** Carry as a tracked blocker that **gates SHIP** (not Gen-0 research).

M0.4 ? Gen-0 TAL harness (**FIRST-CLASS ACTION ITEM** per owner) · *greenlight item*
Productionize `rally_seq_proto.py` ? a **one-command train?eval?ship-gate** harness over **ActionFormer (MIT, 1:1
rally=instance)** and/or **ASFormer (MIT, TAS)** (use `sj-li/MS-TCN2` if MS-TCN++, *not* the Commons-Clause repo),
reusing `metrics.match_intervals`. **Define the ship-gate target up front** (open item): a FLOOR of "beat the honest n=6
heuristic LOVO **0.480**" is necessary but not sufficient ? set an explicit **F1@tIoU** target for "very high P/R" (e.g.
F1@tIoU=0.5 for highlights, tighter for precise rally cuts) before declaring victory. Trains on the GPU/WSL box.

Phase 1 ? Gen-0 ship decision (gated on n?5 + M0.4)
Run the A/B; if the learned Gen-0 clears the ship-gate with per-video calibration, it becomes the rally/highlight
extractor (local, MIT). If not, the gap (more data/features) is the next signal ? no overfitting to declare victory.

Phase 2 ? Gen-1 / Gen-2 (gated on platform scaffolding P0?P4 + healthy corpus + explicit go)
- **Gen-1:** concatenate fully-local cue channels onto the head. ? **Audio is NOT a promising cue here** ? our E1 probe
  (PR #35) found it weak (~2.2× discrimination, AUC ~0.6) and the foreground-audio idea also failed this session; keep it
  parked, not "in reserve." Pose/court is the more credible #2 cue (license/cost-vet first).
- **Gen-2 (VLM):** ms-swift fine-tune of **Qwen3-VL** (SFT cold-start ? GRPO/RFT with a temporal-IoU-vs-golden reward,
  our own CoT supervision). **Two-stage** (VLM proposes, head refines). Cloud train on Modal (no-access) / RunPod-Secure.
- **Hard pre-serving gates (unmeasured, load-bearing):** the **$/call crossover** (self-hosted VLM vs Gemini) **+ a real
  VRAM/token + cost benchmark** on rented A100/L40S (ms-swift multimodal-GRPO examples use 6?8 GPUs ? **budget is the
  largest open risk and is currently unquantified**). Don't stand up an owned serving stack until these clear.

Decisions / status
#	item	status
1	Greenlight Phase-0 milestone(s) ? rec: **M0.1 corpus+event-index**, then **M0.4 harness**	awaiting owner
2	WASB C3 ? first-class action item (own-weights / clean swap)	**owner: yes** ?
3	Ratify licensing **MIT / Apache-2.0 / BSD** (code+data+weights)	**owner: yes** ?
4	Collector = corpus + event-store system-of-record	**owner: confirm?**
5	Rename repo off "badminton"	**owner action** ?
6	Conversational-QA: build seam now, punt model	**rec accepted?**

Open risks / unknowns (none block Phase-0 research)
Budget/cost to reach Gen-2 **unquantified** (largest risk) · ship-gate "very high P/R" target undefined · per-sport clean
trajectory detectors not yet identified · base-model pretraining provenance unauditable (risk-acceptance) · real VLM
VRAM/token cost · single-model-multisport-temporal generalization unproven · DPA + no-

train (minors-excluded) for cloud
training · on-device 4B latency on 8 GB unmeasured.

---
*Design-only and reversible. Nothing starts until you greenlight a milestone. Sequencing discipline holds: the committed
next platform slice remains the async job model (single-tenant); only the M0.1?M0.4 foundation is safe to start.*

6. user (2026-06-10T05:33:30.493Z)

Roadmap ? from annotated data to a self-improving, offline rally segmenter

This document is the **narrative thread**: how the indexer evolves from an
AI-API-dependent prototype into a self-contained tool that learns rally detection
from its own commercially-licensed, annotated video corpus. It explains *how we
got here*, *why each step matters*, and *how it shapes the final tool*. Decisions
are recorded as ADRs in DECISIONS.md; the scoring mechanics live
in EVALUATION.md.

The thesis

The indexer detects badminton rallies in long recordings. Its strongest mode
(`yolo_hybrid`) is a two-stage pipeline: cheap **permissive person-detection
player-motion** filtering (Stage 1, local ? torchvision Faster R-CNN + ByteTrack
tracking) followed by **Gemini** semantic refinement (Stage 2, paid API). That
works, but it costs money, needs the network, and never improves on its own.

> *Naming note:* the `yolo_hybrid` registry key is retained for back-compat, but
> AGPL YOLOv8 was removed and replaced with a BSD/MIT-licensed torchvision +
> ByteTrack stack ? see **ADR-005**. "YOLO" in this doc means the local
> player-motion gating stage, not the Ultralytics library.

A sibling project, **sports-data-collector**, curates sports videos with
per-rally `[start, end)` annotations and a commercial-license flag. The thesis of
this roadmap: **use that annotated, commercially-usable data to (1) measure the
pipeline objectively and (2) train an offline replacement for the paid Gemini
stage** ? turning the tool into one that improves as more data is curated, runs
air-gapped, and generalizes across sports.

Why it matters: it removes the per-run API cost and network dependency, gives us
a defensible quality metric instead of eyeballing, and creates a flywheel ?
more curated matches ? a better offline segmenter ? better highlights.

The four phases

Phase 1 ? Bridge the repos (DONE)
A sport-agnostic `curate export` in the collector emits the indexer's `start,end`
ground-truth CSVs plus a `manifest.json` pairing each CSV with its video, gated on
the `is_commercial` flag. ? *Why:* one clean producer/consumer contract that works
for all five supported sports (badminton/tennis/cricket/pickleball/table-tennis) alike.
See **ADR-002**.

Phase 2 ? Acquire full-resolution video (DONE)
The corpus was 256×144 (the original crawl used `worstvideo`). The `fetch` command
re-acquires every video at **best available resolution (1080p here)**, **in place**
(preserving annotations), via **authenticated Firefox Premium cookies** and yt-dlp's
`default` client. ? *Why:* YOLO/shuttle detection needs real pixels; and the
journey surfaced non-obvious traps (SABR streaming ? 144p; Chrome DPAPI; rate
limiting) now encoded in the tooling. **22/22 fetchable matches are now 1080p.**
See **ADR-003**.

Phase 3 ? Evaluate + tune (IN PROGRESS)
`run_phase3_eval.py` walks the manifest, processes each video, and scores detected

rallies against the ground truth ? per video and corpus-aggregate (precision/recall/ F1, mean IoU, boundary error), for both the `candidates` (raw detector) and `final` (refined) stages. ? *Why:* the candidates-vs-final split tells us *where* error lives ? a **detection** problem (candidates miss rallies) vs. a **segmentation** problem (candidates catch them, refinement is loose). That diagnosis decides where effort goes, and establishes the baseline any trained model must beat.

Phase 4 ? Learn an offline, API-free segmenter (NEXT)
Replace the paid Gemini refinement with a **lightweight classifier trained on the rally annotations**, consuming features the pipeline already computes and stores (`candidate_segments.stats`). No external API, no new labeling. ? *Why:* this is the payoff ? a free, offline, self-improving rally mode. The **substrate was chosen from data**: the ADR-004 update (2026-06-02) measured the YOLO/player-motion candidate stage at **recall ~0.05 @ IoU 0.5** on `shuttleset_8` (median best coverage of a rally only ~20%) ? too low a ceiling to build on. So the offline segmenter's substrate is **WASB shuttle-motion** (ADR-001), not player-motion. See **ADR-004**.

Complementary local cues (beyond the segmenter swap)

Once the substrate is WASB shuttle-motion, the next lever is **fusing additional fully-local cues** to raise rally recall/precision without reintroducing a paid API. The first one explored is **audio "thwack" hit-detection** (**ADR-007**, accepted as a direction 2026-06-07): a pure-numpy spectral-flux onset detector in `backend/pipeline/audio/`. The **E1 GO/NO-GO probe** has run on real GoPro audio ? verdict **AMBER / leaning-negative**: recall signal is real (hit-clusters recover WASB-missed rallies) but the classical-onset precision ceiling (~2.2×) is too weak to fuse cleanly, and a band-pass doesn't rescue it. The only path past that is a *learned timbre classifier* ? which, like Phase 4, is **gated on golden labels**. See `AUDIO_E1_FINDINGS.md`. Pose/court serve-gating is parked as the #2 cue.

The recurring blocker across Phases 3?4 and the audio work is the same: **more golden labels** is the single highest-leverage move (see the cross-cut note below).

How this shapes the final tool

- **Offline & free by default.** The production rally mode becomes a local model; Gemini is kept only as an upper-bound reference, not a dependency.
- **Self-improving.** Every newly curated, annotated match is more training signal; re-training is a local, deterministic step.
- **Measurable.** Every change is scored against the golden set, so "better" is a number, not a vibe.
- **Sport-generic.** The label ? feature ? classifier machinery and the export bridge already span sports; adding a sport is data, not new architecture.
- **Commercially clean.** Only `is_commercial` data flows through the bridge, so the trained model and any derived product rest on a licensable foundation.

Status at a glance

Phase	What	Status
1	Export bridge (`curate export` ? manifest + CSVs)	? Done
2	1080p acquisition (`fetch`, in-place, cookies)	? Done (22/22)
3	Eval + tuning loop (`run_phase3_eval.py`)	? In progress
4	Offline learned segmenter (WASB shuttle-motion substrate)	? Next (substrate decided: WASB, ADR-004)
+	Audio hit-detection cue (`backend/pipeline/audio/`, ADR-007)	? E1 probe run ? AMBER, gated on golden labels

> **Cross-cutting:** golden-set ground truth (which powers Phases 3?4 and on-demand regression testing)
> is produced via a vendor process + pluggable annotation provider ? see
> GOLDEN_SET_VENDORING.md and **ADR-006**. As of 2026-06-02 the annotation
> guideline is authored (collector `docs/annotation/`) and **Roora** (Bengaluru) is selected for an

> initial **3-sport pilot** (badminton, pickleball, table tennis). Input-capture quality requirements
> that bound achievable precision/recall are tracked in VIDEO_RECORDING_GUIDELINES.md.
>
> **Observability:** every processed video now emits an always-on per-video diagnostics
> report (spec-driven warnings/errors with FAQ refs, soft-dependency on `obsreport`), so
> pipeline regressions and footguns surface without re-running eval ? see
> CODE_MAP.md §2 (reporting/observability).

7. user (2026-06-10T05:33:31.179Z)

Deferred Hardening Plan ? Train/Eval Guardrails, Async Jobs, Packaging

Status: ? Design for review ? **no code yet**. Owner-requested follow-up to the
"config & identity consistency" tech-debt pass. Each item below becomes its own PR
only after this doc is approved.

Why these three: they were cataloged as out-of-scope during the consistency
pass because each is larger/design-heavy or needs the owner's machine. They are the
remaining high-value hygiene items not already done (config/structure/Result/logging/
reports) or in flight (detector abstraction, PR #49).

Recommended sequence: #7 Packaging (lowest risk, unblocks clean installs/CI) ?
#13 Train/eval guardrails (correctness of the data flywheel) ? #16 Async jobs
(largest, product-facing).

---

1. Packaging modernization (debt #7)

Problem
- No `pyproject.toml` on `master`. (PR #49's description lists packaging as "done",
  but it isn't present ? reconcile with the owner.)
- The root `setup.py` is a **diagnostics/dependency-checker**, not a PEP 517 build
  script ? its name collides with setuptools semantics and will confuse `pip`/`build`.
- Deps live only in `requirements.txt`; `obsreport` is a commented soft dep
  (BACKLOG) and `supervision<0.30.0` is pinned (debt #14).

Approach
- Add `pyproject.toml` (PEP 621): project metadata + runtime deps mirroring
  `requirements.txt`, with optional-dependency groups:
  - `[project.optional-dependencies] dev` ? `pytest`, coverage tooling.
  - `reporting` ? `obsreport` pinned to its git tag (`v0.1.3`, per BACKLOG).
  - `gpu` ? torch/torchvision (note: the cu124 `--extra-index-url` is **not**
    expressible in standard PEP 621 deps; document the install line or use a
    `[tool.uv]`/pip-conf approach rather than faking it).
- Console entry points: `indexer = backend.main:main` (so `indexer --server` works),
  optionally eval drivers.
- **Rename the diagnostic `setup.py`** (e.g. `scripts/doctor.py`) so it no longer
  shadows the build system; update README/`run_all_tests.py`/CI references.
- Choose a build backend (hatchling recommended) and confirm the `backend/` import
  root stays valid (keep current layout; no src/ move required).

Risks
- `setup.py` rename touches docs and any muscle-memory invocations ? grep first.
- Torch CUDA wheels can't be pinned cleanly in PEP 621; must document the
  `--extra-index-url` install path (the current `setup.py` logic can move into docs).
- CI must switch to `pip install -e .[dev]`.

Verification
- `pip install -e .[dev]` in a clean venv ? `run_all_tests.py` green.
- `indexer --help` resolves via the entry point.

---

2. Train/eval separation guardrails (debt #13)

Problem
Separation is **by convention only**. `backend/eval/calibrate_local.py` and
`calibrate_wasb.py` do honest leave-one-video-out (LOVO), but nothing *enforces*
that a video used to train the learned segmenter
(`backend/eval/training.py::build_training_set` ? `distill_local`) is absent from the
eval/regression set. Leakage would silently inflate the very metrics the product
story rests on (? in BACKLOG).

Approach
- **Split unit = match, never clip** (per `GOLDEN_SET_PHASE1_VIDEOS.md`: split by
  match, hold ~? of each sport as eval).
- Add a `split: "train" | "eval"` field to the collector's `curate export`
  manifest (cross-repo contract, ADR-002 lineage) ? or a separate eval manifest.
- New `backend/eval/splits.py`:
  - `load_split(manifest) -> (train_ids, eval_ids)` keyed by **match id** (not just
    file MD5 ? re-encoded duplicates share a match but differ in
    `compute_video_id`, so group at match level).
  - `assert_disjoint(train_ids, eval_ids)` ? raise on any overlap.
- Wire the guard into the training entrypoint (`distill_local`/`training`) and into
  `run_regression.py` so the baseline is computed strictly on the eval split.

Risks
- Requires a collector-side manifest change (coordinate the cross-repo contract).
- Small corpus ? tiny held-out set ? high metric variance; the guardrail is correct
  but the numbers will be noisy until the golden set grows.
- MD5-only dedup is insufficient (re-encodes evade it) ? must track match identity.

Verification
- Unit test: overlapping train/eval ids ? `assert_disjoint` raises.
- Integration: a deliberately-leaked manifest fails the training entrypoint.

---

3. Async job model (debt #16)

Problem
`POST /api/process` runs validate ? segment ? report **inline**, blocking the
request for the full duration of a long video. Unworkable as a hosted/product API.

Approach (minimal first slice ? single self-hosted box)
- `POST /api/process` returns **202 + `job_id`**; add `GET /api/jobs/{job_id}`
  returning `{status, progress, result, reports}`.
- Run the pipeline on a background worker ? start with a `ThreadPoolExecutor`
  (segmenters already use threads internally for AI handoff, so this fits); leave a
  real queue (RQ/Celery) for later if multi-process is needed.
- Persist job state in the existing SQLite DB via the migration ladder
  (`backend/infrastructure/database.py::_init_db`, bump `PRAGMA user_version` 1 ? 2,
  add a `jobs` table). The observability report becomes the job's artifact.
- Update the static UI to poll `GET /api/jobs/{id}` instead of blocking.

Risks
- **SQLite thread-safety:** verify `check_same_thread`/connection-per-thread;
  `_get_connection` already re-issues `PRAGMA foreign_keys=ON` per connection ?
  confirm it's safe under the executor.
- Cancellation/timeout and crash recovery (a job in RUNNING after a restart).
- In-process store is not horizontally scalable ? acceptable for the current
  single-box self-hosted target; call it out so it isn't mistaken for production-HA.

Verification
- Submit a job ? poll ? DONE with the same segments the sync path produced.
- Server restart mid-job leaves a recoverable/clearly-FAILED job, not a hang.

```
118
119     ---
120
121     ## Out of this doc
122     - **Detector/WSL abstraction (#6):** in flight (PR #49).
123     - **Unify hybrid segmenters (#7 in LOW_REGRET):** owner-deferred.
124     - **ByteTrack `supervision<0.30.0` migration + real-footage validation (#14/#15):**
125       need a GPU/WSL box and real video ? owner-machine work, tracked separately.
126
```

8. user (2026-06-10T05:33:37.512Z)

```
1       # Backlog ? deferred items to revisit
2
3       Single tracker for work we consciously deferred, so nothing gets lost between sessions. Each item records **what**, **why it's deferred**, and **the trigger to revisit**. Every entry traces to a real decision/PR/doc (no speculative work).
4
5       > Maintained as a version-controlled doc for now. Say the word and I'll mirror any/all of these to **GitHub issues** instead (or in addition).
6       > Status legend: ? blocks something · ? do when triggered · ? nice-to-have/cleanup.
7
8       _Last updated: 2026-06-07._
9
10      ---
11
12      ## Monetization / licensing
13      *(source: `archives/MONETIZATION_AUDIT.md` §7, ADR-005)*
14
15      - ? **Ultralytics Enterprise pricing** ? get a quote **only if** the torchvision/ByteTrack detector (PR #8) proves insufficient on real footage. Currently *not needed* (swap done). Quote-only pricing. **Revisit:** if detector quality forces reconsidering YOLOv8.
16      - ? **Codec patent thresholds (AVC/HEVC/AAC)** ? confirm exact Via-LA (AVC) / Access Advance (HEVC) royalty rates & free thresholds **with legal counsel** before relying on the "small SaaS is under threshold" reading. Mitigation already chosen: emit royalty-free AV1/Opus output. **Revisit:** before commercial launch.
17      - ? **Per-checkpoint weight/dataset licenses** ? if we borrow pretrained weights (YOLOX, ActionFormer, WASB, TrackNetV3), the *code* license ? the distributed *weights/dataset* terms. Either train our own or confirm each checkpoint. **Revisit:** when adopting any borrowed weights.
18      - ? **Real GPU $/video & latency** ? benchmark the three LLM-step options (offline ActionFormer vs self-hosted Qwen2.5-VL-7B vs Gemini paid-tier) on the target VM before pricing the product. **Revisit:** when choosing the production LLM step.
19
20      ## Detector swap follow-ups
21      *(source: PR #8, ADR-005)*
22
23      - ? **`supervision` ByteTrack deprecation** ? `sv.ByteTrack` is deprecated as of supervision 0.28.0 and **removed in 0.30.0**; `requirements.txt` is pinned `<0.30.0` as a stopgap. Migrate to `ByteTrackTracker` from the separate `trackers` package before lifting the cap. **Revisit:** before bumping supervision past 0.30.
24      - ? **Real-video validation of the swap** ? the torchvision + ByteTrack Stage-1 is unit-tested with mocks but **not yet validated on actual footage** for detection/tracking quality. (ADR-004 separately found player pixel-motion is a weak rally proxy on broadcast video, so expect tuning.) **Revisit:** when a real test clip + the pipeline are run together.
25      - ? **`yolo_hybrid` naming** ? the registry key, `yolo_*` config keys, and `yolo-hybrid-*` DB detector tags keep the "yolo" name for back-compat despite no YOLO remaining. Optional rename. **Revisit:** if/when a clean break is worth the migration.
26
27      ## Golden-set subproject
28      *(source: `docs/GOLDEN_SET_IMPLEMENTATION.md`, `docs/GOLDEN_SET_VENDORING.md`)*
29
30      - ? **Annotation guideline** ? **DONE (2026-06-02):** authored as the Roora brief + illustrated guide in the collector repo `docs/annotation/` ? rally CSV schema; ignore dead time
```

/ include every rally (incl. faults & lets) / shot = one contact; `ending_reason` (forced/unforced-error taxonomy, shared across net-separated sports); singles & doubles for badminton & pickleball.

- ? **Internal gold key** ? hand-annotate 5?10 licensed clips to near-perfect as the vendor answer key. **Unblocked.** **Revisit:** anytime.
- ? **GS-3 ? `HumanVendorProvider`** ? concrete adapter for the chosen annotation vendor (submit/poll/fetch + DPA/secure-link controls). **Vendor selected for the initial pilot: Roora** (Bengaluru; iMerit / Taskmonk / TELUS remain the fallback shortlist). **Revisit:** once the Roora pilot confirms quality, then build the adapter.
- ? **GS-4 oracle model** ? `AutomatedProvider` interface is stubbed (this work) with a deterministic fake. Wire a *concrete* fast-tier auto-labeler later. **Oracle rule:** it MUST be a different model lineage than production, or the regression test is blind to shared failures. **Revisit:** when a production LLM step is chosen (pick the oracle's lineage to differ).
- ? **GS-5 ? regression triggers** ? wire `regress()` into CI (pre-merge gate), on-demand CLI (the "surprise" case), and a scheduled run; include a deliberate re-baseline path. **Revisit:** after GS-4.
- ? **Vendor pilot (Roora)** ? run the paid pilot with **Roora** as a **single-vendor pilot first** (not a multi-vendor bake-off): assess quality-per-rupee against the §4 bar; escalate to the iMerit/Taskmonk fallback only if it misses. **Revisit:** the pilot doubles as calibration once the gold key exists.
- ? **Phase-1 video collection** ? gather the round-1 golden corpus per `docs/GOLDEN_SET_PHASE1_VIDEOS.md`: **static self-recorded** footage (mirror deployment, not broadcast), 3 sports incl. singles & doubles, across the diversity matrix; ~12 clips/sport with **? held out by match**. **Unblocked.**
- ? **Train/eval separation guardrails** ? train & eval videos live in **separate catalogs** today, kept apart by convention only. Add **hard guardrails** (permissions / explicit approval / re-confirmation at ingest & export) so a video can **never** land in both train and eval. **Revisit:** dedicated design session with the user.

Recording best practices
(source: `docs/VIDEO_RECORDING_GUIDELINES.md`)

- ? **Validate recording hypotheses** ? empirically test which capture conditions (resolution, fps, camera angle/height, lighting) actually move precision/recall, then promote the winners into user-facing "how to record" guidance. Only the static-camera rule is currently evidence-backed (ADR-004). **Revisit:** once a controlled corpus + the eval loop can measure per-condition deltas.

Code & test organization
(source: owner review comments on PR #35, 2026-06-07)

- ? **`tests/` reorganization to reduce fragmentation** ? owner wants to isolate test components on a grouping metric (by subsystem/component) so `tests/` is less fragmented, mirroring the new `backend/pipeline/audio/` + `backend/eval/audio/` layout (e.g. audio tests grouped under `tests/audio/`, eval tests under `tests/eval/`). The exact grouping **metric is the owner's design choice** (consolidate small files vs. add per-area dirs ? the "lesser fragmentation" goal slightly tensions with adding dirs), so this is left as a **dedicated refactor PR**, not guessed here. **Revisit:** when the owner specifies the grouping scheme. (The `backend/eval/audio/` half of the request ? comment 1 ? was done in PR #35.)

Commercial Readiness & Low-Regret Moves ? Approved Decisions Checkpoint (2026-06-07)

(source: Commercial Readiness Review + structure/sequencing discussion with Grok as second-in-command)

- ? **Docs-only checkpoint PR** ? This update + the refreshed `docs/LOW_REGRET_MOVES_PLAN.md` formally record the approved path before any code changes. The initial review docs (PR #37) introduced the findings; this entry locks the execution decisions.
- ? **Approved directory structure** ? Adopt a layered approach under `backend/` for professionalism and future growth (no flat dumping of new modules, no explosion of 15+ single-

file dirs):
61 - `backend/config/` (package ? start with this)
62 - `backend/api/` (FastAPI app, routes, static UI)
63 - `backend/pipeline/` (validators + segmenters + stitcher + audio + detectors)
64 - `backend/infrastructure/` (database + external adapters)
65 - Keep `backend/eval/` prominent as a first-class concern.
66 - Logical grouping for providers, sports, annotations, utils, and a small `core/` only when needed.
67 - ? **Sequencing (Config first, then atomic refactor)** ? Implement Deliverable 1 (Config) **first**, introducing the `backend/config/` package immediately. Perform the full directory structure refactor as a **separate high-pri atomic PR** afterwards. This minimizes conflicts, keeps PRs focused, and gives Claude a clean window with no concurrent moves.
68 - ? **Deliverable 1 (Config)** ? `backend/config/` package (`models.py` + `loader.py`) instead of a single `config.py`. Typed `AppConfig`, central loader with built-in defaults + file override, `extra="allow"` for migration. Update `setup.py` + `main.py`. Add tests. Land this before the layout move.
69 - ? **Subsequent items** ? See the refreshed `LOW_REGRET_MOVES_PLAN.md` for the full ordered list (Error contracts, logging, packaging, detector abstraction, etc.). The structure refactor is now explicitly tracked as its own high-pri step right after Config.
70
71 **Status legend reminder:** ? = approved / in progress tracking, ? = do when triggered.
72
73 ## Observability reports ? shipped; intersections with the low-regret items (2026-06-07)
74 *(source: per-video observability feature ? indexer PR #41/#42/#43, collector PR #11/#12; shared lib `obsreport`)*
75
76 - ? **Low-regret items 1 (Config pkg, PR #39) + 2 (directory restructure, PR #40) are DONE.** Remaining = items 3?9 in `LOW_REGRET_MOVES_PLAN.md`.
77 - ? **Per-video observability reports shipped in BOTH tools** (a TECHNICAL report + a CUSTOMER-friendly summary), via the **soft-dependency** lib `obsreport` (github.com/avidullu/sports-obsreport, MIT, v0.1.3). Each run writes `<video>.report.json / .report.md / .summary.md` next to the video.
78 - ? **Item 8 (Input-Quality UX) is LARGELY SUBSUMED** by these reports (validation stage + footgun flags + verdict already surface quality; a clean customer summary already exists). **Do NOT build a parallel mechanism** ? instead EXTEND the report: add an audio-readiness stage/flag to `backend/reporting/` (harvest) + the obsreport spec. **Revisit:** when item 8 is picked up.
79 - ? **Item 7 (unify hybrid segmenters) should land before the report's real AI-provider timing.** The report records AI timing as `not_captured`; once `wasb_hybrid`/`tracknet_hybrid` are unified there is ONE place to thread a recorder callback through `provider.analyze_video` ? the report. **Revisit:** with item 7 (Claude to add the timing hook there).
80 - ? **Item 3 (Error/Result contract) synergizes with the report.** Today the report INFERS failures (e.g. `silent_provider_noop` = 0 segments + no API key). A structured Result/error from providers/segmenters would let the report read the real reason as a first-class flag. **Revisit:** with item 3.
81 - ? **`obsreport` is a SOFT/commented dep** in `requirements.txt` (works locally via editable install; pipeline no-ops if absent). **Pin the `git+ssh@v0.1.3` tag** in both repos once SSH/CI access to the private lib repo exists.
82 - ? **`run_all_tests.py`** added at repo root ? run the full suite (clear PASS/FAIL, pytest exit code) before sending PRs.
83 - ? **Config-migration follow-ups (DONE ? config & identity consistency pass)** ? the two typed-config bugs from
84 #39/#40 are fixed: (1) **`/api/compile` now honors `--output-dir`** and (2) **`output_dir` is a real field on
85 `OutputConfig`** (default `"output"`); the CLI overrides it on the typed model in `main()`, retiring the
86 write-only `_runtime_overrides` hack. Same pass also unified `video_id` MD5 derivation into
87 `backend/utils/hashing.py::compute_video_id` and migrated all four `run_*.py` scripts off raw `json.load` onto
88 `backend.config.load_config`. See `CODE_MAP.md` §2.0/§6 (footguns retired).
89

9. user (2026-06-10T05:33:38.020Z)

Commercialization Tracker

Status: ? LIVING ? track this in parallel with product/quality work.
Owner: avidullu · **Last refreshed:** 2026-06-07

Purpose. Single source of truth for the **commercial** dimension of the project: licensing/legal posture, codec & output economics, hosting unit-economics, the golden-set vendoring spend, packaging/distribution, and the product-market-fit risks that gate monetization. Updated as decisions land ? see the [Changelog](#changelog) at the bottom.

Merged from / supersedes:
- `archives/MONETIZATION_AUDIT.md` (2026-05-30 deep-research licensing & economics audit) ? kept as the **research-of-record** (per-claim verification transcript, primary sources). This doc carries its *current* status forward.
- `archives/COMMERCIAL_READINESS_REVIEW.md` (code-health + PMF assessment) ? archived; its commercial findings + the quality-phase priorities live here now.

Premise (unchanged, verified): the product is a **closed-source hosted API** ? users upload videos to a cloud NVIDIA GPU VM and get highlight reels back. We never distribute code/binaries, so GPL/LGPL *distribution* copyleft is not triggered ? but **AGPL §13 (network clause)**, **metered-API terms**, and **video-codec patent royalties** all still apply.

---

1. Commercialization dashboard

Legend: ? done · ? in progress / watch · ? blocker / needs action · ?? needs legal counsel · ? future
Priority: **P0** (do first) · **P1** · **P2** (lowest). Rows are grouped **Pending ? In-flight ? Completed**, each sorted by priority. (C# IDs are stable identifiers ? they don't imply order.)

#	Workstream	Pri	Status	Where it stands	Next action / owner decision
	? PENDING (needs action / not started)				
C5	**Output codec royalties**	**P0**	?	Highlights are still encoded **H.264 + AAC** (`libx264` in `backend/pipeline/stitcher/compiler.py`) ? encode-side AVC patent exposure remains.	Switch output to **AV1 + Opus** (royalty-free) on an **Ada-class GPU (L4/L40)** for HW encode. See §4. Decision: provision Ada-class GPU?
C9	**Packaging / distribution**	**P0**	?	No `pyproject.toml`; the diagnostic `setup.py` shadows the build system.	Design in `DEFERRED_HARDENING_PLAN.md` (#7) ? **awaiting your approval** before code.
C12	**SaaS foundations (async/auth/tenancy)**	**P1**	?	Sync `/api/process` blocks on long videos; no auth/tenancy.	Multi-user/platform strategy in `PLATFORM_ARCHITECTURE.md` (pluggable storage/compute, tenancy, orchestration); async slice = `DEFERRED_HARDENING_PLAN.md` (#16).
	? IN-FLIGHT (in progress / watch)				
C4	**Runtime AI strategy (A/B/C + owned model)**	**P0**	?	Live = **(C) Gemini 2.5 Flash, paid tier**. Long-term = **own a self-hosted generative video-QA model** (fine-tuned open VLM) as primary, **paid Gemini always-ready as fallback** via the provider registry.	Make "provider registry + paid fallback" a core principle; build the owned model **after** the scaffolding/flywheel are solid. See `PLATFORM_ARCHITECTURE.md` §5A.
C13	**Beachhead validation**	**P0**	?	Market research (`PMF_MARKET_RESEARCH.md`) recommends **India badminton academies** as the #1 beachhead. Unvalidated.	Cheapest tests: 10?15 academy/coach discovery interviews; rally-detection P/R benchmark on 100+ real amateur clips; fake-door pricing (B2C ~$99/yr, B2B academy ~$600/yr). See §6.
C8	**Golden-set vendoring spend**	**P1**	?	ADR-006: **Roora (Bengaluru)** selected for an initial paid pilot; licensed/owned video only (no user uploads to vendors).	Confirm quality-per-dollar from the pilot; then GS-3 `HumanVendorProvider`. See `GOLDEN_SET_VENDORING.md`.
C3	**Shuttle-tracking weights provenance**	**P1**	? ??	WASB-SBDT **code is MIT (clear)**; distributed **checkpoints are academic-data-trained** ? not covered by the MIT code

grant. MonoTrack/YOLO-NAS banned (noncommercial). | Train-own weights (Phase-4 roadmap) **or** confirm per-checkpoint terms ?? before commercial deploy. |
C14	**Owned-model base licensing + training-data policy**	**P1**	? ??	Owned video-QA model (PLATFORM §5A) must build on **Apache-2.0-clean, video-capable** bases ? **Qwen2.5-VL-7B/32B** (Apache; 3B/72B are *not*), **Qwen3-VL-4B/8B**, or **Gemma 4** ? to keep weights private, gate, and monetize. **Reject Gemma 3** (viral derivative clause ? permanent Google PUP), Llama Vision (naming + 700M-MAU), `gpt-oss` (text-only). **Hard rule: never train on paid-LLM (Gemini/OpenAI/Anthropic) outputs** (competing-model clause). The moat is the **consented dataset + eval harness**, not the model.	Lock to Apache-2.0; **open-model teachers only**; separate explicit **training opt-in**; **exclude minors** from the training pool; per-clip consent ledger. See PLATFORM §5A/§7.
C6	**Inbound HEVC decode**	**P2**	? ??	Decoding users' GoPro H.264/**HEVC** uploads is unavoidable; HEVC pools are fragmented + highest-uncertainty.	Low/uncertain residual; confirm decode-side exposure with counsel before launch.
C7	**Hosting unit-economics**	**P2**	?	Per-video cost is *cheap* for Gemini (~$0.05 input for a 10-min clip); risk is volume × latency, not unit price. Self-host/offline costs unmeasured.	**Benchmark $/video & latency on the target VM** for A/B/C before pricing. See §5.
C10	**Value layer (product surface)**	**P2**	?	Minimal exports shipped (EDL/JSON/CSV + `basic_stats`, PR #51).	Deepen: per-rally tags/notes that feed golden, per-player/match stats, clip gallery. See §6.
C11	**Input-quality enforcement**	**P2**	?	Per-video observability reports surface a customer verdict + footgun flags incl. audio-present (PR #50).	Turn more `VIDEO_RECORDING_GUIDELINES` dimensions into validators/report rules.
C2	**`supervision` ByteTrack pin**	**P2**	?	Pinned `<0.30.0` (deprecation ahead).	Migrate before lifting the pin (BACKLOG). Not a legal risk, a maintenance one.
	? COMPLETED				
C1	**AGPL / copyleft removal**	?	?	ADR-005 shipped: AGPL `ultralytics` YOLOv8 replaced with **torchvision Faster R-CNN (BSD) + `supervision` ByteTrack (MIT)**. No AGPL code/weights in the stack.	Done. Keep new deps permissive.

---

2. Licensing & dependency posture (refreshed risk table)

Verdicts updated against the current stack (`requirements.txt` + code). Original 2026-05-30 verdicts and the 25-claim verification transcript are in `archives/MONETIZATION_AUDIT.md`.

Dependency	License	SaaS obligation?	Current verdict	Notes
~~`ultralytics` + `yolov8n.pt`~~	AGPL-3.0	Yes (§13)	? **REMOVED**	Replaced per ADR-005 ? no longer in the stack.
torchvision detectors (Faster R-CNN/RetinaNet)	BSD-3	No	? LIVE	The chosen replacement; person = COCO class (see `yolo_hybrid.py` `_PERSON_CLASS_ID`).
`supervision` (ByteTrack)	MIT	No	? LIVE	Pinned `<0.30.0` ? migrate before bump (C2).
WASB-SBDT (shuttle)	MIT (code)	No	? code clear	Weights provenance open (C3).
MonoTrack / YOLO-NAS	noncommercial	?	? BANNED	Do not use.
Gemini API (paid tier)	service terms	Paid: no training/review	? LIVE WATCH	Must bill the paid tier to keep user video private (C4).
H.264 / HEVC / AAC codecs	patent pools	royalties on encode/decode	? ??	Separate from ffmpeg's license. Encode-side open (C5); decode-side residual (C6).
ffmpeg/ffprobe, google-genai, torch, opencv, numpy, fastapi/uvicorn/pydantic/jinja2/dotenv	LGPL/Apache/BSD/MIT	No	? CLEAR	Invoked as subprocess / imported; all permissive.
`obsreport` (reports)	permissive (private)	No	? soft dep	Pin the `git+ssh@vX` tag once the repo is published (BACKLOG).

Bottom line: the two hard blockers from the audit (Ultralytics; banned weights) are resolved/avoided. Remaining commercial-legal items are **codec royalties (C5/C6)** and **weights provenance (C3)**, plus the metered-API privacy posture (C4).

---

3. Runtime AI strategy ? A/B/C

The semantic rally-boundary step is the one external/metered dependency. Three paths (from the audit), with current standing:

- **(A) Eliminate ? offline learned segmenter.** ActionFormer (MIT) / our own learned classifier on `candidate_segments.stats`. **Best steady-state margin** (no per-call cost, no external dep) and matches the Phase-4 roadmap (substrate = WASB shuttle-motion, decided in ADR-004). **Most engineering.** ? *target*
- **(B) Self-host VLM.** Qwen2.5-VL-7B/32B (Apache-2.0) or InternVL2.5-8B (MIT). No per-call fees; pays in GPU VRAM (~16?24 GB for 7B). ?? Qwen 3B is noncommercial.
- **(C) Keep Gemini 2.5 Flash (paid).** Fastest, cheap per video, video-native; ongoing metered cost + external dependency. **? current live path.**

Hosted alternatives for (C) comparison: Anthropic Claude (no training on inputs/outputs; customer owns outputs; **no native video** ? weaker here); OpenAI (no training by default; ZDR available); Gemini remains strongest video-native.

---

4. Codec / output economics (the live gap ? C5)

The trap: ffmpeg being free (LGPL) does **not** grant the *patent* rights to the codecs it implements. Royalties attach to the act of encode/decode.

- **Encode side (we control it):** highlights currently emit **H.264/AAC**. AVC (Via LA) is first 100k units/yr royalty-free with a permanent "free to end-users over the internet" provision ? a small SaaS likely falls under thresholds, but ?? confirm.
- **Mitigation:** emit **AV1 (SVT-AV1, BSD + AOMedia royalty-free patent grant) + Opus (royalty-free)**. NVENC AV1 HW-encode needs **Ada-class GPUs (L4/L40)**; Ampere/Turing must use **SVT-AV1 (CPU)**.
- **Decode side (unavoidable, C6):** inbound GoPro **HEVC** decode ? fragmented pools, highest uncertainty; low residual.

Recommendation: provision an **Ada-class GPU (L4/L40)** and standardize highlight output on **AV1 + Opus** ? removes the encode-side royalty question almost entirely. *Implementation lives in `backend/pipeline/stitcher/compiler.py` (codec args).*

---

5. Hosting unit-economics (C7)

Reference workload: ~30-min 1080p match on a cloud NVIDIA GPU VM.

- **(C) Gemini 2.5 Flash (paid):** input \$0.30/1M tokens, output \$2.50/1M; video ~263 tok/s ? 10-min clip ? 158k input tokens ? **~\$0.05 input**. Cheap per unit; the economic risk is **volume × latency**. Flash-Lite is cheaper; Pro ~10×.
- **(B) Self-host Qwen2.5-VL-7B:** \$0 API; cost = GPU-hours / throughput; needs ?24 GB VRAM (L4/A10). \$/video vs Gemini is **open ? measure**.
- **(A) Offline ActionFormer/learned:** cheapest at inference; cost = one-time training + light GPU. **Best steady-state margin.**

> The A/B/C \$/video comparison depends on GPU instance + batch throughput ?
> **benchmark on the target VM before pricing the product.**

---

6. Product-market fit & the quality/data phase

Beachhead (from market research, 2026-06-07 ? see [`PMF_MARKET_RESEARCH.md`](PMF_MARKET_RESEARCH.md))
The static-camera constraint is a **B2B filter**: the buyer must already record from a fixed angle, which points away from the median casual player and toward coaches/clubs. Ranked beachheads:

1. **Badminton coaching academies & serious players, India** *(strongest)* ? 20M+ players, formalizing academy ecosystem (Khelo India), genuine fixed-angle recording, coaches who already monetize, badminton whitespace vs incumbents. Sell to the **academy**, not the individual; needs INR-tuned pricing.
2. **US/global pickleball players & clubs** ? fastest-growing sport (19.8M US, +45.8% YoY) but **SwingVision already owns single-camera tennis/pickleball AI** (USD 179.99/yr) ? head-to-head, not greenfield.
3. **Badminton academies/clubs in East/SE Asia + Denmark** ? highest density; China needs local hosting/partners; Denmark/EU small but high-WTP, easy to pilot.

Pricing anchors *(estimate)*: software-only BYO-camera **B2C USD 60?180/yr**, **B2B academy/club USD 300?1,500/yr** per location (undercuts hardware-bundled Veo/Pixellot/ Spiideo; comparable to SwingVision software-only).

PMF risks (from the readiness review)
- **Capture quality is the gate.** Accuracy is capture-conditional; broadcast/handheld footage is a failure mode. The wedge (static, tripod, full-court) is defensible but narrows the addressable market ? enforce/guide it (C11, `VIDEO_RECORDING_GUIDELINES`).
- **Value beyond raw segments is table stakes.** Time-saved editing alone is weak; per-rally tags, stats, and a correction loop are what make it sticky (C10).
- **Golden labels are the recurring blocker** across the offline segmenter, audio fusion, and measurable quality ? the highest-leverage investment (C8).

Refined priority order (quality/data phase ? "focus on quality once new videos land")
Carried from the readiness review; commercialization items above are tracked in parallel.

1. **Golden-set expansion + data-flywheel UX** (highest with new labeled videos): ingest the new batch, expand calibration/CV/regression baselines, make label contribution + safe retraining easy and non-contaminating (eval/training separation).
2. **Hybrid unification + observability hooks** (deferred item 7, now higher): extract shared logic from `tracknet_hybrid`/`wasb_hybrid` before heavy AI timing/metrics land.
3. **Detector/windowing quality iteration** with new data: re-tune velocity/merge/ min-duration + `rally_gate`; validate WASB on real video; trial audio fusion (ADR-007).
4. **Deeper value layer** (beyond minimal exports, C10): per-rally tagging/notes ? `candidate_segments`, per-player/match stats, clip gallery, human-correction loop.
5. **Input-quality enforcement & guidance polish** (C11): more guideline dimensions ? validators/report rules; "your video is good enough" UX.
6. **Multi-sport maturation + recording best practices.**
7. **SaaS / multi-user / async foundations** (C12) ? lower until monetization is firm.
8. **Polish & distribution** (C9): installer story, first-run path, codec/output, economics.

Recommendation: with the mid-week labeled-video batch, spend ~80% on #1 + #3 (flywheel + quality iteration on real data), then unification (#2) and deeper value (#4).

---

7. Open questions needing owner / counsel sign-off ??

1. **Ultralytics Enterprise price** ? only relevant if a permissive detector ever proves insufficient (fallback). Quote-only.
2. **Per-checkpoint / dataset licenses** for WASB / TrackNetV3 / any VLM ? train-own or confirm each (C3).
3. **Exact Via LA (AVC) / Access Advance (HEVC) rates & free thresholds** ? confirm live figures with counsel before relying on the "small SaaS under threshold" reading (C5/C6).

4. **Real per-video GPU cost & latency** of self-hosted VLM vs Gemini paid at expected
   throughput (C7).
5. **Provision Ada-class GPU (L4/L40)** for HW AV1 encode? (C5 ? gates the codec switch.)

Changelog

- **2026-06-07** ? Added the market-research-of-record (`PMF_MARKET_RESEARCH.md`):
  ranked beachhead (India badminton academies #1), pricing benchmarks, and a C13
  "beachhead validation" workstream. Wired the summary into §6.
- **2026-06-07** ? Created this living tracker. Merged the 2026-05-30 monetization
  audit + the commercial-readiness review. **Refreshed since the audit:** C1 AGPL
  removal now ? shipped (was the #1 blocker); added C5 as a ? (output is still H.264,
  the audit's AV1 recommendation is not yet implemented); C9 packaging tied to
  `DEFERRED_HARDENING_PLAN.md`; reflected PR #50/#51 (input-quality reports, value-layer
  exports). Archived `COMMERCIAL_READINESS_REVIEW.md`.

See also
- `PMF_MARKET_RESEARCH.md` ? market-study-of-record (sizing, segments, competitors, beachhead; cited).
- `archives/MONETIZATION_AUDIT.md` ? licensing/economics research-of-record (sources + verification transcript).
- `archives/COMMERCIAL_READINESS_REVIEW.md` ? superseded assessment (history).
- `archives/decisions/DECISIONS.md` ? ADR-005 (permissive stack), ADR-006 (vendor process).
- `ROADMAP.md` ? Phase-4 offline segmenter (the C4 "(A) eliminate" path).
- `DEFERRED_HARDENING_PLAN.md` ? packaging (C9), async (C12) designs (pending approval).
- `GOLDEN_SET_VENDORING.md` ? vendor process + Roora pilot (C8).
- `BACKLOG.md` ? `supervision` migration (C2), `obsreport` pin, codec follow-ups.

10. user (2026-06-10T05:33:42.883Z)

Platform Architecture ? Multi-User "Assistant Coach"

Status: ? Strategy / research ? for review. No code yet.
Date: 2026-06-07 · **Owner:** avidullu

Purpose. Map the path from today's **single-user, local** tool to a **multi-tenant
"assistant coach" platform**: users upload videos, get highlights **stored and
browsable by history**, and choose **where their data lives** (Google Drive / cloud
bucket / local) and **where the compute runs** (our GPU / their VM). The organizing
principle is to **decouple the layers** ? identity, ingestion, storage, orchestration,
execution/compute, the detection+highlight pipeline, and data-serving ? behind small
pluggable interfaces, reusing the registry pattern the codebase already uses for
segmenters/providers/validators/sports/annotations.

Product framing (the north star *and* the v1 line). This is an **assistant coach**,
not just a clipper ? but the strategy is **phased and unambiguous about scope**:
- **v1 (concrete, near-term):** **rally detection ? highlight-reel compilation ? per-user
  history ? a coach/user correction loop that feeds the golden set** ("assistant-coach
  lite"). Essentially today's pipeline made multi-user ? this is the real v1 deliverable.
- **Mid-term:** **rich rally/shot semantic metadata** (shot type, rally outcome) and the
  owned video-QA model (§5A; §6 P4?P5).
- **North-star vision (strictly future ? *not* v1):** **gait / biomechanics, per-athlete
  identifying templates, and deep coaching feedback under a supervising coach** (§6 P6).
  Inspiring, but explicitly deferred ? and the part that raises the **biometric** privacy
  bar (§7).

The buyer is the **academy/coach** (beachhead in [`PMF_MARKET_RESEARCH.md`](PMF_MARKET_RESEARCH.md)); **power users self-serve**. Two consequences ripple through the design: (1) the data model keeps an **optional coach ? athlete overlay over a generic core** (§4.1) ? not baked in; (2) gait/biometrics is **special-category data**, gated and deferred (§7).

> **Competitive note (validated by research):** none of Veo / Hudl / SwingVision /
> Spiideo offer true BYO-storage or BYO-compute ? they're vendor-cloud SaaS
> (SwingVision uniquely runs *edge* compute on the user's iPhone). So **"your video,
> your storage, your compute"** is a genuine differentiator for privacy-sensitive
> academies ? and SwingVision proves user-side compute is commercially viable. All four
> share the same **teams ? athletes ? clips** data model, which we adopt (§4).

> Scope: this is a **direction-setting doc**. It does not start implementation; the
> first infra slice (an async job model) is the on-hold #16 in
> [`DEFERRED_HARDENING_PLAN.md`](DEFERRED_HARDENING_PLAN.md). Commercial constraints come
> from [`COMMERCIALIZATION.md`](COMMERCIALIZATION.md) (C4 paid-LLM no-train; C12 SaaS
> foundations) and `archives/MONETIZATION_AUDIT.md`.

1. Current state & gaps (what we're evolving from)

Concern	Today	Gap for multi-user
Identity / tenancy	none	no users, orgs, auth, or isolation
Storage	local `filepath` string; output to `output_dir`	no abstraction; can't BYO Drive/bucket; raw video must be on the box
Data model	`videos` keyed by file **MD5**, `play_segments`, `events`, `candidate_segments` (SQLite)	no owner/athlete link; **highlights aren't a first-class persisted entity** (compile writes a file and returns a path); no per-user history
Execution	pipeline runs **in-process, synchronously** in `/api/process`	blocks; can't dispatch to a user's VM/GPU; no job status
Pipeline	**already pluggable** (segmenter/provider/validator/sport/annotation registries)	reuse as-is ? this layer is the model for the rest
API	`/api/process`, `/api/query`, `/api/compile`, `/api/config`; no auth	needs auth, per-tenant scoping, async job + history endpoints

Grounding: `backend/infrastructure/database.py` (schema + `PRAGMA user_version` migration ladder, v1), `backend/main.py` (endpoints, sync flow), the registry pattern in `backend/pipeline/segmenters/base.py::SegmenterRegistry` and `backend/providers/base.py`.

2. Target layered architecture

Decouple into layers with a clean **control-plane vs data-plane** split ? the orchestrator coordinates *metadata and jobs*; the heavy *bytes and GPU work* can live elsewhere (even in the user's own cloud).

```
                ????????????????????????????????????????????????
   CONTROL      ?  Presentation / API  (auth, per-tenant)        ?
   PLANE        ?  Identity & Tenancy  (org?coach?athlete?user)  ?
                ?  Orchestration / Jobs (submit?status?result)   ?
                ??????????????????????????????????????????????????
                                ? dispatch       ? metadata
                ?????????????????????????  ???????????????????????
   DATA         ?  Execution / Compute  ?  ?  Data / Serving     ?
   PLANE        ?  (local | BYO-VM |    ?  ?  (per-tenant DB:    ?
                ?   hosted GPU)         ?  ?   videos, segments, ?
                ?   runs the Pipeline:  ?  ?   highlights, jobs, ?
                ?   detect ? highlight  ?  ?   history)          ?
                ?????????????????????????  ???????????????????????
                            ? read input / write artifacts
                ?????????????????????????????????????????????????
```

```
                ?  Storage  (local upload | Google Drive | S3 |  ?
                ?  GCS | Azure)  ? raw video + highlight outputs ?
                ??????????????????????????????????????????????????
```

The two **new pluggable seams** are **Storage** and **Compute/Execution**; everything else either exists (pipeline) or is net-new infra (identity, orchestration, serving). A **model/training layer** ? the owned generative video-QA model (§5A) ? sits atop these, fed by Storage (training corpus), Data (labels), and Compute (training + inference), and is delivered through the existing **AI-provider registry with a paid fallback**.

3. Pluggable abstractions (extend the existing registry pattern)

The codebase already proves the model: an `ABC` + a `Registry` singleton with `register/get/list_available` (e.g. `SegmenterRegistry`). Add two more registries with the same shape so they're plug-and-play and testable in isolation.

3.1 `StorageBackend` registry
Replace the bare `video_path: str` that flows through the pipeline with a **`StorageRef`** (a backend id + a key/URI), resolved by a backend.

```python
class StorageBackend(ABC):
    name: str
    def open_read(self, ref: StorageRef) -> BinaryIO: ...      # stream large video
    def download_to_local(self, ref: StorageRef, dst: str): ...  # for ffmpeg/cv2/WSL
    def upload(self, local_path: str, ref: StorageRef): ...     # persist highlight/artifacts
    def signed_url(self, ref: StorageRef, ttl: int) -> str: ... # hand to LLM / browser
    def stat(self, ref: StorageRef) -> StorageStat: ...
    def list(self, prefix: StorageRef) -> list[StorageRef]: ...
```

- **Recommended implementation:** build the backend over **`fsspec`** (one matrix ? local / S3 (`s3fs`) / GCS (`gcsfs`) / Azure (`adlfs`) + Google Drive via `gdrivefs`, Dropbox via `dropboxdrivefs`), reaching for **`smart_open`** as the robust large-MP4 byte-streaming primitive. Storage becomes one more registry. (Treat Drive/Dropbox as add-ons ? less battle-tested than `s3fs`/`gcsfs`; cover with integration tests.)
- **Backends:** `local` (upload dir), `gdrive`, `s3`, `gcs`, `azure`. **BYO-storage**: the tenant registers their own bucket/Drive; raw video **never leaves their account**.
- **Auth ? never persist users' long-lived keys.** For buckets, have the user create an IAM role you **`AssumeRole`** into, scoped to one bucket/prefix, and mint **presigned URLs** so raw video moves **browser ? the user's bucket directly, never through our servers**. For Drive, OAuth with the narrow **`drive.file`** scope; for academies, a **service account with domain-wide delegation**. Store any per-tenant creds encrypted.
- **Why it matters for the pipeline:** ffmpeg/OpenCV/WSL need a **local file**, while the LLM provider can take a **signed URL**. The backend exposes both `download_to_local` (for compute) and `signed_url` (for the provider/browser), so the pipeline stops caring *where* the bytes live.
- **Migration path:** introduce `StorageRef` alongside the current local path; a `local` backend wraps today's behavior so nothing breaks; adapters land incrementally.

3.2 `ComputeBackend` / `ExecutionBackend` registry
Where the pipeline actually runs for a given job.

- **`local`** ? in-process / local subprocess (today's behavior; dev + self-host).
- **`hosted`** ? our GPU VM pool (the SaaS default).
- **`byo_worker`** ? the tenant runs a **pull-based worker** (a small agent) on their own VM/GPU. **Critical security property:** the worker *pulls* jobs scoped to its tenant from the control plane, pulls input from the tenant's **own** storage, runs the pipeline, and writes artifacts + results back ? so **the orchestrator never holds the user's cloud credentials** and needs **no inbound access** to their network. This

  mirrors the **self-hosted CI runner** pattern (GitHub Actions runners use a 50s
  outbound-only HTTPS long-poll). This pull-based runner is the **cheapest, most
  credential-safe v1** for BYO-compute.
- For users who want the platform to **provision GPUs in their *own* cloud account on
  demand** (an academy with an AWS account but no standing GPU box), add **SkyPilot**
  (BYOC: everything launches in the user's accounts/VPCs; creds stay with them).
  Modal is the managed fallback for users wanting zero infra (accepting that raw
  video then flows through Modal).
- **?? Operational / support burden (a real adoption risk).** For the primary beachhead ?
  Indian badminton **academies/coaches** ? asking them to *run and maintain a worker on
  their own VM/GPU* is a meaningful ongoing load (updates, monitoring, firewall/NAT, the
  "it stopped after our IT change" ticket). So: **default academy users to `hosted`**, and
  treat `byo_worker`/SkyPilot as a **power-user / enterprise / privacy-sensitive tier**,
  not the academy default. If/when BYO ships, plan for **auto-updating, self-healing
  agents + health heartbeats + clear worker status in the UI**, and budget the support cost.
- **Middle tier to consider ? "zero-infra BYO":** a managed-but-isolated path where the
  storage stays in the user's bucket (signed URLs) while compute runs on **our** GPUs
  in a **strictly isolated, per-tenant, ephemeral sandbox** (nothing persists our side).
  This gives the BYO *privacy* benefit (raw video never persists on our infra) **without**
  the user running a worker ? a pragmatic default between full-hosted and full-BYO.

The pipeline code is unchanged; only *where it's invoked* and *how it gets bytes* vary.

3.3 Reuse, don't reinvent
The existing `segmenter` / `provider` / `validator` / `sport` / `annotation` registries
are the proven precedent and stay as the **pipeline** layer. The `annotation` registry +
`candidate_segments` table are also the natural seam for the **coach-correction / golden
-set** loop in the assistant-coach phase (§6, P4).

4. Multi-tenant data model & serving layer

4.1 Entities ? generic core, with the coach model as an optional overlay
The foundational schema is deliberately **domain-neutral** so we don't overfit to the
coach mindset ? the coach/athlete framing is a **market-fit hypothesis that will iterate**
(see C13 / PMF), not a load-bearing assumption. **Generic core:**
```
User ??< VideoCollection ??< Video ??< Highlight ??< HighlightMetadata
                                 ???< PlaySegment ??< Event
Video / Highlight ??< Job (processing record: status, compute backend, timings)
```
- **Generic & re-targetable:** a `VideoCollection` groups a user's videos (a season, an
  event, a "folder") with no assumption of a coach or academy ? a power user, a coach, or a
  parent are all just `User`s with collections.
- **Highlights are first-class** (today compile only writes a file): a `highlights` table
  (segment ids + output `StorageRef` + params + created_at) powers "see past videos and
  highlights by history," and **`HighlightMetadata`** carries the **rich semantic tags**
  (shot type, rally outcome, etc. ? see §6 metadata stage) plus titles / ratings / notes.
- **`video_id` stays the content MD5** (dedup within a tenant), scoped by `tenant_id`.
- **Coach overlay (optional, additive):** for the academy/coach GTM, layer
  `Org/Academy ? Coach ? Athlete` *on top* (e.g. a `Membership`/role table + an `athlete_id`
  tag on collections/videos) **without** baking it into the core tables ? so the schema
  stays usable for solo power users and the coach model can evolve, or be dropped, as PMF
  dictates.
- **History/serving API:** list collections/videos/highlights per user (and, with the

overlay, by athlete/coach/date); **query highlights by `HighlightMetadata`** (e.g. shot type / outcome); re-stitch from stored segments without reprocessing.

4.2 Tenancy isolation ? options & recommendation
Model	Isolation	Ops cost	Best for
Shared DB + `tenant_id` (row-level, ideally Postgres **RLS**)	logical	low	**Hosted SaaS, many small tenants** ? *recommended default*
Schema-per-tenant	medium	medium	tens?hundreds of mid tenants
DB-per-tenant (incl. **SQLite-per-tenant**)	strong/physical	higher at scale	**self-hosted single-academy**, or high-isolation enterprise

Recommendation: for the hosted product, **migrate SQLite ? Postgres with shared-DB + `tenant_id` + Row-Level Security**; keep **SQLite-per-tenant** as the **self-hosted / single-academy** deployment (it reuses today's code almost verbatim). Either way, **abstract DB access behind a repository layer now** so the engine is swappable, and graduate the `PRAGMA user_version` migration discipline to **Alembic** under Postgres.

5. Orchestration / async jobs

This is the serving layer that makes multi-user real. It is the **multi-user superset of [`DEFERRED_HARDENING_PLAN.md`](DEFERRED_HARDENING_PLAN.md) #16** (async job model) ? build that first, single-tenant, then generalize.

- **Job record** (`jobs` table): `id, tenant_id, video_id, type (index|compile|analyze), status (queued|running|done|failed), progress, compute_backend, result_ref, error, timestamps`. `/api/process` returns **202 + job_id**; `/api/jobs/{id}` polls.
- **Idempotency** keyed on `(tenant_id, video_id, pipeline_version)` ? reuse the content-hash discipline already in `compute_video_id`.
- **Dispatch:** the orchestrator enqueues; a worker (hosted or `byo_worker`) claims jobs for its tenant, resolves input via the tenant's `StorageBackend`, runs the pipeline, writes artifacts + DB rows. **Observability reports** (already per-video) become the job artifact surfaced in history.
- **Queue/worker backend ? a decision to prototype in #16, *not* a default here.** The *requirements* are firm ? **idempotency**, **resumability** (a 40-min GPU job mustn't restart on a worker blip), **progress states** ? which rule out fire-and-forget Celery/RQ for the long jobs. The *choice* should be made by prototyping in the [`DEFERRED_HARDENING_PLAN.md`](DEFERRED_HARDENING_PLAN.md) #16 slice, weighing:
  - **Start simplest:** **Arq** (async-native, natural FastAPI fit) or even a **plain Postgres-backed queue + worker** ? likely sufficient for the *first* single-tenant async slice, with the lowest operational complexity and the easiest debugging.
  - **Durable-execution upgrade if/when needed:** **Hatchet** (Postgres-backed, per-step retries/timeouts/concurrency; pairs neatly with the pull-based worker) ? attractive because it adds **no new datastore** once we're on Postgres, but it *is* a new dependency + operational/debugging surface; adopt only when long-job resumability/ observability demands it.
  - **Heaviest:** **Temporal** ? most mature durable workflows, operationally heavy; reserve for genuinely complex multi-stage pipelines at scale.

  Record the decision criteria in #16: operational complexity, maturity, **debugging experience for long GPU jobs**, and whether durability is worth a new dependency. **Bias: start with the simplest thing that gives retries + status; graduate only on evidence.**

5A. Owned generative model (the video-QA assistant) & the data flywheel

The north-star capability: an **owned, self-hostable generative video model** ? fine-tuned on sports video (including **consented, user-contributed** footage) ? that **answers questions about rallies, shots, and technique**. This is the conversational brain of the "assistant coach." Crucially, it does **not** replace the known stack; it is introduced **behind the same abstraction, with the paid API as an always-ready fallback**.

> **The moat is *not* the model.** Anyone can download Qwen-VL/Gemma. The defensible
> assets are **(a) the consented, badminton-specific video?QA dataset** and **(b) the
> eval harness**. Build and message *those* ? not "we own an LLM."

> **? Committed guardrail (legal, non-negotiable).** We respect the licensing findings in
> full. **Paid frontier models (Gemini / OpenAI / Anthropic) are used for *inference /
> serving / fallback* only ? never as a *training* data source.** Harvesting their outputs
> to train our owned model would breach their "no competing model" terms (a terms/copyright
> violation), so it is prohibited in both engineering and legal process. **Training teachers
> are open, Apache-2.0 models only** (Qwen-VL / InternVL / Gemma 4). Detail in the
> distillation constraint below.

Core principle: AI is a pluggable provider with a paid fallback
The codebase already abstracts the cloud AI behind `provider_registry` /
`AIVideoProvider` (`backend/providers/`). Make that a **first-class platform principle**:
- Every generative capability routes through the **provider registry**.
- The **owned model** plugs in as a new provider (`owned_vlm`); **Gemini (paid, via
  Vertex AI)** stays registered as the **fallback** (failover on error/low-confidence,
  or as the quality ceiling/teacher reference).
- This lets us **ship today on the known paid stack** and **swap in the owned model when
  its quality clears a measured bar** ? with zero architectural churn and the paid path
  always one config flip away. (Mirrors the COMMERCIALIZATION C4 A/B/C strategy: the
  owned model is the long-run "(A) eliminate the per-call dependency"; paid is "(C)".)

The model / training layer (new sub-layer, fed by the scaffolding)
```
Data curation        Training              Eval                 Serving
(consented user vid  (fine-tune an open,   (extend the existing (self-host inference
 + golden set +      commercially-usable   eval harness to QA   as a provider /
 rally/segment/event base VLM via          quality + the        ComputeBackend;
 annotations ?       LoRA/QLoRA)           regression gate)     paid fallback)
 video-QA pairs)
```
This layer **reuses every other layer**: **Storage** holds the training corpus;
Data supplies labels (the `annotation` registry + `candidate_segments` +
golden set are already the labeling seam); **Compute/Orchestration** runs training and
inference jobs (even **BYO-compute** can serve inference on the tenant's GPU); the
eval harness already in `backend/eval/` extends to grade QA quality. **In other
words, the rock-solid scaffolding is exactly what makes the model flywheel possible** ?
which is why scaffolding comes first (see §6).

Base model & training (Apache-2.0 only ? licensing is decisive)
Recommended near-term base: `Qwen2.5-VL-7B-Instruct` ? Apache-2.0, native long-video
(dynamic-FPS sampling, temporal mRoPE), mature LoRA tooling. Upgrade path
Qwen3-VL-4B/8B (Apache-2.0); **Gemma 4 multimodal (Apache-2.0, 2026)** is a clean
Google-stack alternative to evaluate.
- ?? **Lock to Apache-2.0-clean bases** so you can fine-tune, **keep weights private,
  gate behind subscriptions, and never owe open-sourcing or branding**. (Note Qwen2.5-VL
  is Apache **only at 7B/32B** ? 3B/72B use the custom Qwen license.)
- ? **Reject for the flagship:** **Gemma 3** ? custom terms with a **viral derivative
  clause** (anything fine-tuned on it, *or even trained on its synthetic outputs*, stays
  permanently bound to Google's *updatable* Prohibited-Use Policy); **Gemma 4's switch to
  Apache-2.0 removes this**. **Llama 3.2 Vision** ? custom license forces a "Llama-?"
  model name + "Built with Llama" badge and a 700M-MAU ceiling. **`gpt-oss`** ? Apache-2.0
  but **text-only**, so usable only as an optional text-reasoning layer, never the video
  encoder.
- **Training method:** **QLoRA** (? r=64, ?=128) on the 7B ? fits one prosumer GPU and

  avoids catastrophic forgetting of general video skills. Build the dataset by converting the existing rally/shot/segment annotations + golden set into templated **clip?QA pairs**, expand synthetically with **open-model teachers only** (see constraint below), and hold out a **golden QA eval** that gates every fine-tune. A few hundred gold pairs ? **5k?20k** curated is a realistic band for this narrow domain.
- **Serving economics:** a 7B VLM ? 14 GB VRAM (FP16) / ~3.5 GB (INT4); a ?16 GB GPU is a comfortable serve target (vLLM for throughput). Self-hosting wins **only** once you have sustained video-QA volume **+** a defensible consented dataset **+** real per-call API cost pain ? below that, the paid API is cheaper than a standing GPU and the second stack is overhead. Hence: **paid-primary now ? owned-primary / paid-fallback later.**

Base-model licensing + training-data policy are tracked as **COMMERCIALIZATION C14**.

Training backend ? managed service vs self-managed (the fine-tune "forge")
Where you run the LoRA/RL fine-tune. Weigh on **video support, weight export/ownership, smallest servable model, RL support, no-train DPA, and infra burden**.

Decisive finding (researched 2026-06): managed fine-tuning services are **image-based** (JSONL + base64 images) ? for video you pre-sample frames and feed them as image sequences. **Only the self-managed `ms-swift` explicitly advertises *native* video (+audio) training.** So for true video-QA, a self-managed framework is the strongest technical fit ? and it also lets you fine-tune the **small 7B** (cheap to serve) and keep full ownership. Managed services trade some of that for zero infra.

Option	Type	Video / VLM support	Smallest VL	RL	Export weights	Best for
ms-swift (ModelScope)	Self-managed (rented/own GPU)	**Native video+audio**; Qwen3-VL/Omni, 300+ MLLMs	any incl **7B**	? DPO/GRPO	? full ownership	**? video-QA; slots into BYO-compute (§3.2)**
LLaMA-Factory	Self-managed	Qwen3-VL + 100+ VLMs; video datasets	any incl 7B	?	?	popular, well-documented
Unsloth	Self-managed (1-GPU optimized)	Qwen3-VL LoRA (vision+lang; frames)	any incl 7B	? GRPO	?	cheapest first runs
Tinker (Thinking Machines)	Managed API (low-level)	Qwen3-VL **30B-A3B+ only**; image input (video **unverified**)	30B-A3B MoE	? PPO/GRPO, custom loss	? adapters + ckpt; no-train ToS	zero-infra RL/research at MoE scale
Together AI	Managed (tune + serve)	Qwen3-VL / Gemma-3 / Llama-4; image data	verify	partial	? adapters downloadable	managed Qwen3-VL + serving
Fireworks AI	Managed (tune + serve)	Full Qwen2.5-VL (3B?72B SFT; LoRA@32B + LoRA-upload)	**7B**	partial	?? verify export (lock-in risk)	managed *small*-VL + serving
Predibase (Rubrik)	Managed **+ VPC self-host**	VLM instruct-tune; **LoRAX** multi-adapter	verify	? (VPC)	? **RFT/GRPO** flagship	RL "creativity" + multi-adapter + privacy
~~OpenAI GPT-4o / Gemini tuning~~	Closed	?	?	?	? no export	**non-fit ? violates "own the model"**

(Enterprise clouds ? Databricks Mosaic / SageMaker / Vertex on open models ? give full control but are only worth it if you're already on that stack.)

Recommendation:
- **Early iteration / true video-QA ?** self-managed **`ms-swift`** (or LLaMA-Factory / Unsloth) on a **rented GPU** (RunPod / Lambda / Modal / SkyPilot-to-your-cloud): native video, any size incl. **7B** (cheap to serve), full ownership, and it drops into the **ComputeBackend / BYO-compute** seam (§3.2). **Likely a better fit than Tinker** for the video requirement *and* serving economics.
- **Managed, no GPU ops ?** **Together AI** (Qwen3-VL + downloadable adapters + serving)

  or **Fireworks** (full Qwen2.5-VL incl. 7B + serving) ? confirm frame/video support and
  adapter export.
- **If RL "coaching creativity" + per-sport multi-adapter is central ?** **Predibase**
  (RFT + LoRAX, VPC self-host for the biometric/privacy posture) or **Tinker** (lowest-
  level RL primitives).
- **Tinker** remains a strong zero-infra RL-research option with clean weight export ? just
  not the cheapest path to a *small video* model (its VL floor is 30B-A3B).
- **Portability is the hedge:** you own the **data + recipe + exported weights**, so the
  backend is swappable ? same ethos as the provider/registry pattern. Don't get locked in.

Verify before committing (any backend): (1) **native video / frame-sequence**
ingestion + max frames per example; (2) **weight export** + a **commercial DPA** covering
consented, minors-excluded video (biometric, §7); (3) the **inference plan** for the size
you train (a self-managed **7B** is far cheaper to serve than a 30B-A3B MoE).

Training harness ? make it agent-runnable (nanochat-style operability)
The goal: **an agent (or CI) can run a fine-tune on a fresh batch of golden-labeled video
and get a trustworthy ship / no-ship verdict, unattended.** Karpathy's **nanochat** is the
reference for that *operability* ? one MIT repo, one `speedrun.sh` that runs
tokenizer?train?eval?serve, a single complexity dial, and an automatic **report-card**
score. It is **not** a usable engine here (text-only, from-scratch GPT-2-class), but its
five properties are exactly what make training agent-runnable, and we copy them:
(1) one command, end-to-end; **(2)** one complexity dial (sane auto-derived defaults);
(3) an automatic **report card** ? a deterministic pass/fail vs a target (the crux);
(4) reproducible + deterministic + cheap; **(5)** minimal/readable.

Engine (don't reinvent): **`ms-swift`** already delivers this for VLMs ? CLI-driven,
native video, Qwen3-VL support, an eval backend (EvalScope), and reproducibility via
`args.json` + `full_determinism`. (LLaMA-Factory is the close second; HF **`nanoVLM`** ?
~750 lines ? is the minimal reference to *learn* VLM training, not the production path.)

The harness = thin glue over components we already have ? one declarative,
agent-invokable job (call it `train_speedrun`):
1. **Build dataset** from golden labels ? clip?QA pairs (reuse the `annotation` registry +
   `candidate_segments` + golden set ? the labeling seam in `backend/`).
2. **Fine-tune** via ms-swift on a **ComputeBackend** GPU (§3.2), run as an orchestration
   job (§5 / DEFERRED #16): one config, idempotent, logged.
3. **Auto-eval + report card** ? extend the existing `backend/eval/` regression harness to
   QA quality; emit a `report.md` pass/fail vs the **held-out golden QA eval**.
4. **Register** the result as the `owned_vlm` provider (Gemini paid fallback stays) **only
   if it beats the bar**.

So: **golden labels ? ms-swift train (on a compute backend) ? eval gate ? register
provider**, behind one command an agent or CI runs. nanochat is the inspiration for the
wrapper; ms-swift is the engine; the **golden QA eval is the report card**.

> **Prerequisite (the crux):** the harness is only as trustworthy as the **golden QA eval
> set**. nanochat's speedrun is meaningful *because* its eval (CORE score) is automatic and
> trustworthy; ours is the golden QA eval ? **build that first**, or an agent-run loop
> optimizes blind. (Same "moat = dataset + eval", scaffolding-first principle.)

?? Teacher / distillation constraint (the single biggest legal landmine)
You **cannot** train the owned model on outputs from the paid frontier APIs ? a general video-QA assistant is squarely a *"competing model"*: **Gemini** ("may not use the Services to develop models that compete"), **OpenAI** ("may not use Output to develop models that compete"), **Anthropic** (bars using Outputs to train competing models). Two safe lanes: **(1)** generate synthetic QA with **open-model teachers only** (Qwen-VL / InternVL / Gemma 4 ? Apache-clean); **(2)** keep the paid API strictly as a **runtime fallback** ? serving answers is fine, *harvesting its outputs to train the model that replaces it is not*. Write a hard **"no training on competitor-API outputs"** rule into eng + legal process. (This makes the ROADMAP's "Gemini as offline teacher only" precise: the teacher is *open models*, not the paid APIs.)

Data flywheel, personalization & consent tiers
Two complementary loops, each with its **own** opt-in (training consent is separate from processing consent ? §7):

- **Global model (shared ? raises the floor).** Train the base owned model on an **explicitly-contributed, licensed** corpus ? *not* on whatever sits in a tenant's bucket (preserves "your video, your storage"). Flywheel: *more consented, labeled matches ? a better shared model ? better highlights/answers ? more reason to contribute.* Higher consent bar (pooled data, biometric/minor care ? §7); **minors excluded by default**.

- **Per-user adapters (personalized ? raises the ceiling) ? owner hunch, under exploration.**
  A small **per-user (or per-tenant) LoRA adapter, fine-tuned *only on that user's own consented videos*** on top of the shared base, served via **multi-LoRA** (LoRAX / Together / Predibase). Why it's compelling:
  - **Precision:** adapts to *that user's* court, camera angle, lighting, and players' styles ? materially better detection/highlights for them.
  - **Cleanest consent story:** *"improve **your** detection using **your** data"* is far lower-friction and lower-risk than contributing to a global model ? the data improves **only the user's own adapter**, with **no cross-user mixing**, sidestepping much of the pooled-corpus biometric/minor entanglement. Reinforces the differentiator: *your video, your storage, your compute ? and now **your model**.* With BYO-compute the adapter can be trained/served on the user's own GPU and **never leave their boundary**; **deletion = drop the adapter** (clean erasure).
  - **Open questions (still a hunch):** cold-start (need enough of the user's labeled data before a personal adapter beats the base ? fall back to the shared model meanwhile); a **per-user held-out eval** to prove the adapter helps (don't ship a regression); base + optional global-adapter + per-user-adapter stacking; per-user training cost at scale. Worth a spike once the harness + golden eval exist.

Two consent tiers, then: **(a)** "improve **my own** detection with **my** data" (per-user, low friction) and **(b)** "contribute to the **shared** model" (global, higher bar). **Default for both is off.**

6. Phased roadmap

Phase	Deliverable	Build / reuse
P0	**Async job model, single-tenant** (DEFERRED #16)	`jobs` table via `user_version` ladder; worker; 202+status endpoints
P1	**Identity + multi-tenant data model + upload + highlight persistence +	

history** | auth (build vs Auth0/Clerk/Supabase ? decide); Postgres + `tenant_id`/RLS; `highlights` table; repository layer; history API + UI |
P2	**`StorageBackend` registry + BYO-storage**	new ABC+registry; `local`?`gdrive`/`s3`/`gcs`; replace `video_path` with `StorageRef`; encrypted per-tenant creds
P3	**`ComputeBackend` registry + BYO-compute**	`local`/`hosted`/`byo_worker`; pull-based worker agent; control/data-plane split
P4	**Rich rally/shot semantic metadata & tagging** *(the next value stage after solid rally detection + highlights)*	shot type (smash / backhand drive / net), rally **outcome** (winner / forced / **unforced error**), context (e.g. **open court but error**) ? query-API filters + `HighlightMetadata` + a **human correction/review loop** feeding the golden set via the `annotation` registry + `candidate_segments`
P5	**Owned generative video-QA model** (§5A)	global flywheel **+ per-user adapters**; P4 metadata is training signal ? fine-tune an open VLM (Qwen-VL) via LoRA ? serve as `owned_vlm` **provider with Gemini paid fallback**; extend the eval harness to QA quality
P6 *(visionary)*	**Gait / biomechanics under a supervising coach**	deliberately **back-burner**; technique feedback; highest Art. 9 bar ? gate behind explicit biometric + parental consent (§7)

P0?P1 deliver the headline ask (per-user upload + stored highlights + history). P2?P3 deliver the "plug-and-play storage/compute" ask. **P4 (rich semantic metadata) is the immediate next value stage once rally detection + highlights are solid** ? e.g. *"rallies that ended in a smash or backhand drive,"* *"rallies where the court was open but the player made an unforced error."* **P5** owns the model; **P6 (gait) is the visionary horizon, intentionally deferred.**
Sequencing principle (owner directive): make the scaffolding (P0?P4) **rock-solid first**, keep the **known paid stack as an always-ready fallback**, and only then shift focus to **owned-model quality** (P5). The provider registry keeps the paid path one config flip away.

7. Privacy & legal (gait raises the bar ? design for it now)

- **The load-bearing nuance:** biometric data is special-category **only when used to *uniquely identify* a person** (GDPR Art. 9). **Highlight/shot detection on players a coach already knows is general analytics ? generally *not* Art. 9.** But **gait/biomechanics that builds a per-athlete identifying template tips into Art. 9** and needs **explicit, granular consent** referencing biometric data. Engineer gait with that boundary (consent gate, purpose limitation, no cross-account identification); decide whether it runs **locally/on-tenant only** (strong posture) vs our cloud. (EDPB Guidelines 3/2019 on video devices apply.)
- **Minors (e.g. academies).** Parental/guardian consent and safeguarding are first-order, not an afterthought. Model consent explicitly per relationship (user?subject, coach?athlete where the overlay applies, parent?minor).
- **BYO-storage is the central privacy/liability lever:** if raw video stays in the tenant's own bucket/Drive and we process via signed URL or their compute, we minimize what we hold ? shrinking breach surface, residency, and erasure obligations.
- **Paid-LLM no-train (C4):** use **Gemini via Vertex AI** (contractual no-training), **not the consumer/free tier**; set short/zero retention, document it in the DPA, and surface "your footage is never used to train AI" as a trust feature for academies. For the gait phase, prefer keeping biometric inference off third-party APIs entirely.
- **Training consent is a distinct, higher bar (for the owned model, §5A).** Training on user video needs a **separate, explicit, opt-in training licence** ? not bundled with processing consent ? and it **compounds** the biometric (Art. 9) and minor-consent issues above. Train **only on explicitly contributed/licensed video**; never train on what merely passes through a tenant's bucket. **Exclude minors' video from the training pool by default**, keep the training corpus a **separate, firewalled, opt-in sub-pool**, and log **per-clip consent/provenance** for revocation + audit. Incentives must not make consent non-free (the product stays fully usable without contributing). And the

teacher-distillation rule (§5A): never train on paid-LLM outputs. **Two consent tiers (§5A):** *(a)* "improve **my own** detection with **my** data" ? a per-user adapter trained only on the user's own video (lowest friction; no cross-user mixing; deletion = drop the adapter); *(b)* "contribute to the **shared** model" ? pooled corpus, higher bar. Default for both is **off**.
- **Right to erasure / retention:** per-tenant, per-athlete delete must cascade across DB + storage artifacts (the `ON DELETE CASCADE` discipline already in `database.py` extends here).

8. Key decisions for the owner (the forks)

1. **Hosted-first vs BYO-first.** Recommended: **hosted control-plane first** (P0?P1) with **BYO-storage soon after** (P2); **BYO-compute later** (P3) as a power-user/enterprise option ? it's the highest-complexity, highest-security item.
2. **Database now or later.** Move to **Postgres at P1** for the hosted path, or stay SQLite-per-tenant if self-hosted-single-academy is the first GTM. (Abstract DB access regardless.)
3. **Auth: build vs buy** (Auth0 / Clerk / Supabase Auth / Cognito).
4. **Orchestration tech** (see research recommendation) ? bias to the simplest that gives retries + status.
5. **Gait processing location** ? local/on-tenant (privacy-max) vs cloud (capability-max).
6. **How far to push BYO-compute** given its security/ops complexity.
7. **Owned-model base + licensing (§5A).** Which commercial-OK video VLM to build on (Qwen-VL Apache-2.0 vs Gemma 3 custom license vs ?) and confirm a gated commercial derivative is permitted ? see COMMERCIALIZATION C14.
8. **Training-data sourcing & consent.** Opt-in contribution terms, incentives, and the teacher-distillation constraint (whether paid-LLM outputs may seed the corpus).
9. **When to start the owned model** ? strictly after the scaffolding + flywheel are solid (the explicit directive), with the paid provider as the standing fallback.

Risks: scope explosion (sequence strictly P0?P5); multi-tenant data-leak bugs (RLS + tests); BYO-compute security; **biometric/minor liability** (get this right before any gait feature ships); **training on user video without proper opt-in/licence**, and **base-model / teacher-distillation licence violations**; cost of hosted GPU (training *and* inference); **premature model-building** before data/eval are ready (mitigated by keeping the paid fallback and gating P5 behind the flywheel).

9. Sources

External best-practices research (2026-06-07) backing the recommendations above. Each underlying claim in the research was tagged fact-vs-synthesis; figures are as-of and should be re-validated before committing budget.

Storage abstraction: [fsspec](https://fsspec.com/) · [cloudpathlib](https://github.com/drivendataorg/cloudpathlib) · [smart_open](https://github.com/piskvorky/smart_open) · [Apache Libcloud](https://libcloud.apache.org/) · [gdrive-fsspec](https://github.com/fsspec/gdrive-fsspec) · [dropboxdrivefs](https://github.com/fsspec/dropboxdrivefs) · [AWS S3 presigned URLs](https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-presigned-url.html) · [token-based S3 with STS](https://oneuptime.com/blog/post/2026-02-12-token-based-access-s3-sts/view) · [signed URLs across clouds](https://medium.com/@shaikhsarwan49/signed-urls-in-aws-gcp-and-azure-a-beginner-friendly-guide-e2e549425865)

Multi-tenancy: [Bytebase: multi-tenant DB patterns](https://www.bytebase.com/blog/multi-tenant-database-architecture-patterns-explained/) · [Hunchbite: RLS vs schema-per-tenant](https://hunchbite.com/guides/multi-tenant-saas-architecture) · [PlanetScale: tenancy in Postgres](https://planetscale.com/blog/approaches-to-tenancy-in-postgres) · [DEV: DB-per-tenant vs shared schema](https://dev.to/young_gao/multi-tenant-architecture-database-per-tenant-vs-shared-schema-1n2e)

BYO-compute / orchestration: [Pinecone BYOC](https://www.pinecone.io/blog/byoc/) · [Nuon: control/data-plane](https://nuon.co/blog/byoc-control-plane-data-plane-architectures) · [GitHub self-hosted runners (ARC)](https://docs.github.com/en/actions/hosting-your-own-runners/managing-self-hosted-runners-with-actions-runner-controller/about-actions-runner-controller) · [SkyPilot](https://github.com/skypilot-org/skypilot) · [SkyPilot AI control plane](https://blog.skypilot.co/ai-job-orchestration-pt2-ai-control-plane/) · [Modal (AWS Marketplace)](https://aws.amazon.com/marketplace/pp/prodview-j727623xqhh2k) · [Northflank: Anyscale alternatives](https://northflank.com/blog/anyscale-alternatives-for-ai-ml-model-deployment) · [DevPro: Python task queues 2025](https://devproportal.com/languages/python/python-background-tasks-celery-rq-dramatiq-comparison-2025/) · [Judoscale: choosing a task queue](https://judoscale.com/blog/choose-python-task-queue) · [Hatchet vs Temporal](https://hatchet.run/versus/hatchet-vs-temporal) · [Hatchet](https://github.com/hatchet-dev/hatchet)

Privacy / legal: [GDPR Art. 9](https://gdpr-info.eu/art-9-gdpr/) · [GDPR Local: biometric compliance](https://gdprlocal.com/biometric-data-gdpr-compliance-made-simple/) · [Legiscope: Art. 9](https://www.legiscope.com/blog/gdpr-article-9-special-categories.html) · [EDPB Guidelines 3/2019 (video)](https://www.edpb.europa.eu/sites/default/files/files/file1/edpb_guidelines_201903_video_devices_en_0.pdf) · [Fox Rothschild: EDPB video guidance](https://www.foxrothschild.com/publications/video-surveillance-under-gdpr-edpb-issues-final-guidance) · [ICO: children & UK GDPR](https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/childrens-information/children-and-the-uk-gdpr-old/what-are-the-rules-about-an-iss-and-consent/) · [EDPB Guidelines 05/2020 (consent)](https://www.edpb.europa.eu/sites/default/files/files/file1/edpb_guidelines_202005_consent_en.pdf) · [AI data-retention compared](https://ax-sentinel.com/blog/ai-data-retention-policies-compared) · [Xenoss: enterprise LLM platforms](https://xenoss.io/blog/openai-vs-anthropic-vs-google-gemini-enterprise-llm-platform-guide)

Comparable products: [Veo](https://www.veo.com/) · [Veo Help Center](https://support.veo.com/hc/en-us/articles/4459159526929-Overview-of-cam-veo-co-Record-manage-and-upload-footage-with-your-Veo-Cam-1) · [Latterly: Hudl competitors](https://www.latterly.org/hudl-competitors/) · [Apple: Behind the Design ? SwingVision](https://developer.apple.com/news/?id=0pg4dthn) · [SwingVision FAQ](https://swing.vision/faq) · [Sportsvisio: coach video-analysis comparison](https://www.sportsvisio.com/stories/video-analysis-software-for-coaches-complete-comparison)

Training backends & harness (§5A): [karpathy/nanochat (harness reference)](https://github.com/karpathy/nanochat) · [HF nanoVLM (minimal VLM)](https://github.com/huggingface/nanoVLM) · [nanoVLM blog](https://huggingface.co/blog/nanovlm) · [Tinker (GA + vision)](https://thinkingmachines.ai/blog/tinker-general-availability/) · [Tinker model lineup](https://tinker-docs.thinkingmachines.ai/model-lineup) · [Tinker ToS (weights/no-train)](https://thinkingmachines.ai/legal/tos/) · [Fireworks VLM fine-tuning](https://fireworks.ai/blog/vlm-tuning) · [Together AI fine-tuning](https://www.together.ai/fine-tuning) · [Predibase: fine-tune & serve VLMs](https://predibase.com/blog/how-to-fine-tune-and-serve-vlms-in-predibase) · [Predibase RFT/GRPO](https://docs.predibase.com/user-guide/fine-tuning/grpo) · [ms-swift (native video/audio)](https://github.com/modelscope/ms-swift) · [LLaMA-Factory (VLMs)](https://github.com/hiyouga/LlamaFactory) · [Unsloth (Qwen3-VL)](https://qwen.readthedocs.io/en/latest/training/unsloth.html) · [Fine-tuning tools 2026 (LoRA/QLoRA)](https://www.index.dev/blog/top-ai-fine-tuning-tools-lora-vs-qlora-vs-full)

Owned model & licensing (§5A): [Qwen2.5-VL](https://qwenlm.github.io/blog/qwen2.5-vl/) · [Qwen2.5 license split](https://qwenlm.github.io/blog/qwen2.5/) · [Qwen3-VL](https://github.com/QwenLM/Qwen3-VL) · [Gemma 3 (multimodal)](https://huggingface.co/blog/gemma3) · [Gemma Terms of Use](https://ai.google.dev/gemma/terms) · [Gemma license-risk analysis](https://wcr.legal/google-gemma-license-risks/) · [Gemma 4 / Apache-2.0](https://www.mindstudio.ai/blog/what-is-gemma-4-apache-2-license-commercial-ai-deployment) · [Llama 3.2 license (700M MAU + naming)](https://www.llama.com/llama3_2/license/) ·

[gpt-oss (text-only, Apache-2.0)](https://openai.com/index/introducing-gpt-oss/) · [Gemini API terms (compete clause)](https://ai.google.dev/gemini-api/terms) · [OpenAI terms](https://openai.com/policies/row-terms-of-use/) · [Anthropic: outputs & training](https://support.claude.com/en/articles/12326764-can-i-use-my-outputs-to-train-an-ai-model) · [QLoRA practical](https://apxml.com/courses/fine-tuning-adapting-large-language-models/chapter-4-parameter-efficient-fine-tuning/practice-qlora-fine-tuning) · [VLM LoRA cookbook](https://huggingface.co/learn/cookbook/en/fine_tuning_vlm_trl) · [GDPR Art. 9](https://gdpr-info.eu/art-9-gdpr/) · [athletes & biometric data](https://www.architectureandgovernance.com/applications-technology/biometric-data-and-athletes-privacy-law-and-compliance-implications/) · [federated learning + DP](https://research.google/blog/federated-learning-with-formal-differential-privacy-guarantees/) · [self-hosted LLM VRAM/cost](https://www.gpu-mart.com/guides/self-hosted-llm) · [Gemini API pricing](https://ai.google.dev/gemini-api/docs/pricing)

See also
- [`COMMERCIALIZATION.md`](COMMERCIALIZATION.md) ? C12 (SaaS foundations), C4 (paid-LLM), economics.
- [`DEFERRED_HARDENING_PLAN.md`](DEFERRED_HARDENING_PLAN.md) ? #16 async job model (the P0 infra slice), #7 packaging.
- [`PMF_MARKET_RESEARCH.md`](PMF_MARKET_RESEARCH.md) ? academy/coach beachhead (who this serves).
- [`CODE_MAP.md`](CODE_MAP.md) ? current architecture + the registry extension points.
- [`ROADMAP.md`](ROADMAP.md) ? offline self-improving rally detection (the pipeline's own trajectory).

11. user (2026-06-10T05:33:50.771Z)

PMF & Market Study ? AI Highlight-Reel Tool for Self-Recorded Racket Sports

> **Research snapshot ? 2026-06-07.** Figures are cited as-of with source + year;
> validate live before relying on them for investment/pricing decisions. This is the
> market-research-of-record behind the PMF section of the living
> [`COMMERCIALIZATION.md`](COMMERCIALIZATION.md) tracker (§6). Product framing:
> a closed-source hosted API/SaaS (plus local mode) that ingests long-form,
> self-recorded, **static-camera, full-court amateur** racket-sports video and
> auto-detects rallies and compiles highlight reels. Primary sport: badminton;
> sport-generic architecture roadmapped to tennis, pickleball, table tennis, cricket.

> **Confidence convention.** Figures from named industry reports, federations, or
> company pricing pages are tagged with source + year. Inferred items are tagged
> **(estimate)** / **(low confidence)**. Emerging-category market sizes vary widely
> between vendors; ranges are shown rather than false precision.

---

1. Executive summary

The product sits at the intersection of three favorable trends: (a) a fast-growing
sports analytics market (~USD 5?6B in 2025, ~18?22% CAGR to USD 24?36B by ~2034
depending on vendor); (b) a structural shift to **amateur self-recording** on action
cameras and phones (action-camera market ~USD 6.5B in 2024, sports the largest
segment); and (c) **strong racket-sport participation** ? 220M+ badminton players,
106M tennis players (2024), and a near-vertical pickleball curve (~19.8M US players in
2024, +45.8% YoY).

The central tension is the **static-camera constraint**: the tool works best on fixed,
full-court amateur footage and *fails* on broadcast/panning footage. This is both a
moat (most competitors target team sports / broadcast) and a filter on the addressable
user ? the customer must already set a camera on a tripod behind the court. That
behavior is concentrated in **coaches/academies, clubs, and serious enthusiasts**, not
the median casual player.

Recommended beachhead (strongest single): **badminton coaching academies and
serious club/competitive players in India** ? a large, rapidly formalizing academy

ecosystem (Khelo India tailwind), genuine fixed-angle recording behavior, existing willingness to pay for coaching, an underserved tooling gap (incumbents are team-sports/broadcast), and a direct-but-immature competitor field. The product is *native badminton-first* ? a genuine edge here. Secondary: **(2) US/global pickleball players & clubs** (capital-rich, fastest-growing, but SwingVision already strong); **(3) badminton academies/clubs in East/SE Asia + Denmark** (highest density).

Pricing clusters into hardware-bundled auto-filming (Veo, Trace, Pixellot, Spiideo) and software-only phone AI (SwingVision USD 179.99/yr; BadPro+ pay-per-use). A software-only BYO-camera model undercuts the hardware players; realistic anchors **(estimate): B2C USD 60?180/yr; B2B academy/club USD 300?1,500/yr per location.**

Top risks: (1) amateurs won't reliably set up a tripod ? TAM collapses to coaches/clubs; (2) free/near-free alternatives cap B2C WTP; (3) capture-quality variance is the core failure mode; (4) badminton's strongest markets (India, Indonesia, China) are the most price-sensitive and hardest to bill at Western SaaS prices.

---

2. Market sizing & trends

2.1 Sports analytics / sports-video market
- **2025 size:** ~USD 5.47B (Precedence Research) to ~USD 5.79B (Fortune Business Insights); other vendors USD 5.6?6.0B. *(2025)*
- **Growth:** CAGR 15.7%?22.5%. Precedence ? ~USD 29.75B by 2034 (20.63%); Fortune BI ? USD 24.03B by 2032 (22.5%); MarketsandMarkets (narrower) USD 4.75B by 2030 (15.7%); Future Market Insights ? ~USD 36.2B by 2035 (22.1%). *(reports 2024?2025)*
- **Video analytics is the leading segment** within sports analytics (Grand View / Fortune BI) ? the directly relevant sub-market.
- **Interpretation (estimate):** the *amateur/club auto-highlight* slice is a small but fast-growing fraction; most analytics revenue today is pro/broadcast/team-sports. The product targets the long tail the big numbers under-serve.

2.2 Action-camera / amateur recording adoption
- **2024 size:** ~USD 6.5B (GMInsights) ? ~USD 16.9B by 2034 (~10% CAGR). *(2024?2025)*
- **Sports is the largest end-use segment** of action cameras (GMInsights, 2024).
- **GoPro ~47% share, DJI ~20%** (electroIQ/industry, 2024); phones + cheap tripods are the de-facto amateur rig.
- **Implication:** the capture-side behavior the product depends on is already widespread and growing ? *for the segments that record* ? validating "BYO-camera, software-only."

2.3 Participation & popularity by sport and geography

Sport	Participation signal	Source (year)
Badminton	220M+ play regularly; 160+ countries	BWF est. via BadmintonBites (2023)
Badminton ? China	~80M registered, up to ~100M (largest base)	industry compilations (2023)
Badminton ? India	20M+ recreational by 2023; academy boom + Khelo India	industry compilations (2023)
Badminton ? Indonesia	National sport; ~1M club players	industry (est.)
Badminton ? Denmark	100,000+ registered club members (Europe #1)	industry (2023)
Badminton ? Japan/Korea	Major school-club + workplace sport	industry (2023)
Tennis	**106M players (2024)**, +25.6% over 5 yrs (87M in 2021)	ITF Global Tennis Report (2021, 2024)
Tennis ? Asia	~1/3 of players (33M of 87M)	ITF (2021)
Pickleball ? US	**19.8M players (2024)**, +45.8% YoY, +311% over 3 yrs; ~6.2M core	SFIA / Pickleheads / The Dink (2024)
Pickleball ? US courts	~70,000 courts, +55% places to play YoY	SFIA / Pickleheads (2024)
Pickleball ? global	IFP 78 member countries; UK +73% in 2024; ~282M played monthly per PPA claim **(low confidence)**	various (2024?2025)
Table tennis	30M+ players; 227 member associations	ITTF (current)

| **Cricket** | 2.5B+ fans; 500M+ in India (mostly *fans*, not self-recorders) | Cricket Rise / Nielsen (2024?2025) |

- **Pickleball market value:** ~USD 2.2B (2024) ? USD 9.1B by 2034 at 15.3% CAGR
  (Market.us/Custom Market Insights, 2024?2025).
- **Badminton equipment market:** ~USD 7.23B global (2023), Asia-Pacific dominant.

Geographic read: badminton density is overwhelmingly Asian + Denmark; tennis is
broad/high-income; pickleball is US-dominant, capital-rich, globalizing fast; cricket
is huge as *fandom* but its amateur self-recording + static-full-court fit is weak
(large field, not a fixed-frame court). For *this product's static-camera fit*,
badminton/pickleball/tennis/table-tennis are the core; cricket is a long-tail roadmap
item, not a beachhead.

---

3. Customer segment evaluation

Segment	Market-size signal	Willingness to pay	Accessibility	Static-camera fit	B2C/B2B	Verdict
Individual amateur/enthusiast	Largest raw TAM (100s of millions)	**Low** ? competes with free; ~USD 5?15/mo ceiling	High via app stores; high CAC/churn	**Mixed** ? most won't set a tripod	B2C	Volume play, weak economics; use as funnel
Coaches & coaching academies	India academy boom; thousands across Asia	**High** ? already sell coaching; USD 300?1,500/yr plausible	Reachable via federations/distributors/social	**Strong** ? already film fixed-angle	B2B/B2B2C	**Best fit** ? buyer, recorder, payer aligned
Clubs & leagues	Denmark 100k+; thousands of clubs Asia/EU	Medium-high; USD 500?3,000/yr	Medium ? committee sales cycle	**Strong**	B2B/B2B2C	Strong secondary; longer cycle
Tournament/event organizers	Many small/regional events	Medium ? per-event/seasonal	Medium ? episodic	**Strong**	B2B	Good per-event upsell; lumpy
Schools & universities	Large (esp. Asia/Japan clubs)	Low-medium ? budget-constrained	Low ? slow procurement	Strong	B2B	Slow; grant/Khelo-India dependent
National/regional federations	Few, high-value	Medium-high; procurement-heavy/political	Low ? long cycles	Strong	B2B	Lighthouse/credibility, not beachhead

12. user (2026-06-10T05:33:51.695Z)

Architecture Decision Records

ADR-001: Precise shuttle tracking via self-hosted TrackNetV4/WASB on WSL2

Date: 2026-05-25
Status: Accepted

Context

The pipeline uses an AI multimodal provider (Gemini) for *semantic* rally segmentation.
We need **precise frame-level detection of the moving shuttlecock** (and, generically,
balls for other sports) to produce higher-quality candidate rallies and a fine-tuning
signal.

We evaluated whether a commercially available model/API could do this. Findings:

- **Multimodal LLM APIs (Gemini, Claude, GPT-class) are the wrong tool for precise
  localization.** They reason well about *what* is happening (rally vs. dead time) but
  cannot reliably track a tiny, fast (~up to 400 km/h), motion-blurred shuttle frame by
  frame.
- The precision leaders are **purpose-built trajectory models**, which are open source and
  self-hosted, not big-vendor APIs:
  - **TrackNetV4** (TensorFlow) ? badminton-specific, heatmap + motion-attention,
    highest reported shuttle accuracy; ships `src/predict.py` producing per-frame (X, Y)

  coordinate CSV. https://github.com/AR4152/TrackNetV4
  - **WASB** (PyTorch) ? multi-sport strong baseline (badminton/tennis/volleyball/?),
    fits our `SportProfile` abstraction; needs a custom-video inference wrapper.
    https://github.com/nttcom/WASB-SBDT
- True turnkey commercial APIs (Roboflow hosted models) exist but cap below a tuned
  TrackNet; enterprise systems (Hawk-Eye, SkillCorner, TRACAB) are not API-accessible.

Decision

1. Adopt **self-hosted TrackNetV4** as the primary precise shuttle detector, with **WASB**
   as the multi-sport fallback.
2. Run these models under **WSL2 (Ubuntu)**, **not** native Ubuntu. The entire indexer
   already runs on Windows; WSL2 keeps a single dev environment and quarantines the
   TensorFlow/PyTorch dependencies. Native Ubuntu would require dual-boot and migrating
   the whole project for marginal gain.
3. Integrate via a **subprocess adapter**: a detector module that shells out to the WSL
   `predict.py`, reads the coordinate CSV, and converts trajectory ? candidate rallies
   feeding the existing stitching flow. This keeps the TF dependency fully isolated from
   the main Python environment.

Consequences

- **WSL I/O caveat:** `/mnt/c` access is slow. Stage match videos inside the WSL
  filesystem (e.g. `~/clips/`) before inference rather than reading large mp4s across the
  Windows?WSL boundary.
- TrackNetV4 (TensorFlow) and WASB (PyTorch) live in **separate conda envs**.
- Next step after integration: manual video annotation ? fine-tune the detector.

ADR-002: Use the sports-data-collector's rally annotations as a golden eval + temporal-segmenter training set

Date: 2026-05-26
Status: Accepted

Context

A sibling project, **sports-data-collector**, curates sports videos and stores per-rally
`[start, end)` timestamps (plus stroke counts, scores) in an SQLite DB, with a
commercial-license flag per source. We asked whether this data could improve the
indexer's rally detection, and for which task.

Key findings:

- The annotations are **temporal-only** (when rallies start/end). They do **not** contain
  per-frame shuttle `(x, y)` coordinates, so they **cannot train the shuttle detector**
  (TrackNetV4/WASB ? see ADR-001), which needs coordinate labels.
- They are an excellent **golden evaluation set** and a **training signal for the
  temporal rally segmenter** (active vs. dead play over time) ? the stage that is today
  a hand-tuned heuristic, not a learned model.
- The collector's five sports tables (`badminton_rallies`, `tennis_points`,
  `cricket_deliveries`, `pickleball_rallies`, `tabletennis_rallies`) all share a
  `(video_id, ordinal, start_time, end_time)` shape, and the indexer already has
  `sport_registry` / `segmenter_registry`. So a *generic* `(start, end)` bridge serves
  every sport.
- The two repos key videos differently: the collector by a human `video_id`, the indexer
  by **file MD5**. A re-encode/re-download changes the bytes, hence the MD5.

Decision

1. Treat the collector's rally timestamps as (a) the **golden eval set** for rally
   segmentation and (b) the **labels** for training a temporal segmenter ? explicitly
   not as shuttle-detector training data.
2. Bridge the repos with a sport-agnostic **export** (`curate export` in the collector):
   per-video `start,end` CSVs (the indexer's eval format) + a `manifest.json` that pairs
   each CSV with its video file. Gate on `is_commercial` so only commercially usable data
   flows through.
3. Honour the **MD5 keying** contract: evaluation/processing must run on the exact video
   file recorded in the manifest; acquisition (ADR-003) therefore precedes processing.

Consequences

- The collector becomes the *producer* of labels; the indexer the *consumer* (processing +
  scoring + training). Neither takes a hard dependency on the other beyond the manifest.
- Scoring is free and deterministic (reads the DB; no API). See `docs/EVALUATION.md`.
- Only invest in frame-level coordinate labels (for the detector) **if** evaluation shows
  detection, not *segmentation*, is the bottleneck.

ADR-003: Video acquisition ? always best resolution, upgrade in place, authenticated cookies

Date: 2026-05-26
Status: Accepted

Context

The collector's initial crawl downloaded videos with `yt-dlp -f worstvideo` (256×144),
mislabelled as 1080p. The indexer needs ?720p MP4 for meaningful YOLO/shuttle detection.
Re-acquiring full-resolution sources surfaced several hard-won realities:

- The ShuttleSet corpus **tops out at 1080p** (no 4K); "best available" resolves to 1080p.
- Authenticating with a **YouTube Premium** account is required for reliable, unthrottled
  downloads ? anonymous downloads are throttled (~1.3 MiB/s) and **non-deterministic**
  (identical commands variously yielded 1080p DASH, 1080p HLS, or a 144p fallback).
- **Chrome** cookies can't be read on Windows (app-bound encryption / DPAPI, yt-dlp
  #10927); **Firefox** cookies read cleanly even while open.
- The `web`/`web_safari` player clients are forced into **SABR streaming**, which strips
  download URLs and silently drops to 144p (yt-dlp #12482). yt-dlp's `default` client
  serves real downloadable 1080p DASH with auth cookies.
- Bulk bursts trip YouTube **rate-limiting** (a 22-video burst failed all at once, though
  each worked in isolation).

Decision

1. **Always fetch the best resolution available** (`bv*+ba/b -S res,...`, merged to MP4);
   ffprobe the result and store the *actual* width/height/fps ? never assume.
2. **Upgrade in place, never clean-slate:** the `fetch` command re-downloads a video and
   updates only `local_path`/`resolution`/`fps`, preserving every rally annotation and
   curation status (deleting would cascade-delete the valuable labels).
3. Use **authenticated cookies** (`--cookies-from-browser firefox` / `--cookies file`) and
   the **`default` player client**; enable the EJS challenge solver (deno).
4. **Throttle bulk runs** (`--sleep` + per-video `--retries`/backoff) and treat a
   below-min-height download as a **failure** (discard, don't commit) so a flaky low-res
   result never overwrites a good entry.

Consequences

- All 22 fetchable ShuttleSet matches are now true 1080p (one match, shuttleset_12, has no source URL in the catalog and remains video-less but keeps its annotations).
- Acquisition is reproducible and resumable; the ~26 GB corpus is kept outside git.
- Future non-YouTube or 4K sources are supported by the same code (optional `max_height`).

ADR-004: Add an offline, API-free *learned* rally segmenter as a first-class mode

Date: 2026-05-26
Status: Proposed (substrate pending data)

Context

The default high-quality path (`yolo_hybrid`) refines YOLO candidate windows with **Gemini**, which costs money and requires network access. With a now-1080p, commercially-licensed, rally-annotated corpus (ADR-002/003), we can learn the refinement step offline instead.

Crucially, the **API-free signal already exists and is already persisted**: `yolo_hybrid` Stage 1 is pure YOLO (player tracking ? per-window motion features: peak/mean velocity, active-frame count, track-id count) stored in `candidate_segments.stats`. Gemini is only the Stage-2 refiner. So the rally annotations can **train a classifier to replace Gemini**, with no external API and **no new labeling**.

Three candidate substrates were weighed:

- **Tune `motion` thresholds** on the annotations ? trivial, fully offline, but a low ceiling (pixel motion can't separate rally from crowd/pan/replay). Use as a *floor*.
- **YOLO features ? trained classifier** ? reuses already-stored, semantically meaningful player-motion features; medium-high ceiling; moderate effort. *Recommended primary.*
- **WASB shuttle-motion ? rally detector** ? highest ceiling (the shuttle defines play), fully offline but heaviest (needs the WSL2 WASB runner, ADR-001); can't train WASB itself (no xy labels), only a rally layer on its output.

Decision

1. Build an **offline, API-free learned segmenter** as a new registry mode (e.g. `yolo_learned`): label detector windows by overlap with GT rallies, train a lightweight classifier on the persisted features, emit final segments with **zero API calls**, and score it with the existing harness against `motion` and a Gemini reference.
2. **Choose the substrate from data, not assumption.** A learned refiner can only sharpen the windows the detector found, so the deciding metric is the detector's **candidate-stage recall** vs. the annotations: high recall ? build on YOLO features; low recall ? bring in WASB's shuttle signal to raise the ceiling.
3. Keep the Gemini-backed `yolo_hybrid` as an **upper-bound reference**, not a production dependency.

Consequences

- The final tool gains a **self-contained, offline, free** rally mode trained on its own curated data ? runnable air-gapped, improvable as more matches are annotated.

- The approach is **sport-generic**: the same label?feature?classifier machinery extends to
  tennis/cricket once those have annotations.
- The full paid 22-video Gemini evaluation is **not** run; spend is limited to a single
  reference video.

Update (2026-05-26): Phase 3 evidence ? substrate is WASB, not YOLO

The deciding candidate-recall measurement is in. On shuttleset_8 (64 GT rallies), pure
YOLO Stage-1 produced 181 candidate windows with:

- **Recall @ IoU 0.5 = 0.05**, but recall @ IoU 0.1 = 0.53 and @ IoU 0.01 = 0.72.
- **72%** of rallies have *some* candidate overlap, yet the **median best coverage of a
  rally by any candidate is only ~20%**.
- Post-hoc **merging adjacent candidates does not help** (recall stays ~0.05 and degrades
  as the merge gap grows ? windows overshoot rather than align).

Interpretation: YOLO isn't *blind* to rallies, but its windows are **badly misaligned** ?
because the **broadcast camera tracks the players**, their on-screen (pixel) velocity dips
during rallies and spikes on pans/cuts/replays. Player-pixel-motion is therefore a poor
rally proxy on broadcast footage (it was designed for *static* GoPro recordings). A
classifier that only *filters* these windows is capped at ~0.25 recall ? below any useful
bar ? and no simple boundary post-processing recovers it.

Decision update: per this ADR's data-driven rule (low candidate recall ? bring in the
shuttle signal), the offline segmenter's substrate is **WASB shuttle-motion** (ADR-001),
not YOLO player-motion. The shuttle's motion/presence is camera-invariant and directly
defines active play. The `motion` segmenter is likewise unusable here (0 TP ? moving-camera
pixel motion is everywhere).

Caveats / next: this is one video; the mechanism (camera tracking) generalizes but
should be confirmed on 2?3 more via a *free, local, candidates-only* YOLO pass (no Gemini).
Processing is also slow (~50 min per 1080p video) ? the detection pass likely needs reduced
resolution and/or cached detector outputs to be practical across 22 matches. The
substrate-agnostic training-data + classifier layers already built still apply: they will
consume **WASB-derived** windows/features instead of YOLO's.

ADR-005: Permissive Dependency Stack for the Hosted Monetization Path

Date: 2026-05-30
Status: Accepted

Context

The project will be monetized as a **closed-source hosted API** (users upload videos to a cloud GPU VM and get highlights). Because we never distribute code/binaries, GPL/LGPL distribution copyleft is not triggered ? but **AGPL §13** (network clause) and **metered-API terms** are, and **video-codec patent royalties** are a separate concern from software licenses. A deep-research audit (2026-05-30, 25 adversarially-verified claims against primary sources ? see `docs/MONETIZATION_AUDIT.md`) identified the blockers and clean alternatives. This ADR also **resolves the open licensing question in ADR-001/ADR-004** about WASB.

Decision

1. **Drop `ultralytics`.** Both the library and the `yolov8n.pt` weights are **AGPL-3.0**; §13 makes them a blocker for a closed-source hosted service. Swapping the codebase while keeping the `.pt` does **not** escape ? the weights are AGPL too. Replace player/motion-gating detection with a permissive detector: **torchvision Faster R-CNN / RetinaNet (BSD, default ? zero new deps, person = COCO class 0)**, escalating to **YOLOX** or **RT-DETRv2 (Baidu repo, Apache-2.0)** if more speed/accuracy is needed. **Ultralytics Enterprise License** is a paid fallback only (quote-only pricing).
2. **Shuttle tracking stays permissive.** **WASB-SBDT and TrackNetV3 are MIT (CLEAR for commercial use)** ? this resolves the licensing unknown flagged in ADR-001/ADR-004. **MonoTrack (Adobe Research, noncommercial) and YOLO-NAS weights (Deci, noncommercial) are banned.** Train our own checkpoints (or confirm per-checkpoint terms), since distributed research weights/datasets aren't covered by the MIT *code* grants.
3. **LLM step ? keep all three strategies open:** (A) eliminate via an offline temporal segmenter (**ActionFormer, MIT** ? the Phase-4 direction), (B) self-host **Qwen2.5-VL-7B/32B (Apache-2.0)** or **InternVL2.5 (MIT)**, or (C) keep **Gemini 2.5 Flash on the paid tier** (required ? the free tier trains on and human-reviews uploaded video). The `google-genai` SDK itself is Apache-2.0 (clear).
4. **Emit royalty-free output.** Standardize highlight output on **AV1 (+ Opus)** to avoid AVC/HEVC encode-side patent exposure; provision an **Ada-class GPU (L4/L40)** for hardware AV1 (NVENC), falling back to **SVT-AV1 (CPU, BSD-3-Clause-Clear)** on T4/A10/A100.

Consequences

- Eliminates the only two hard legal blockers; the rest of the stack (torch, opencv, fastapi, numpy, lapx, ffmpeg-as-subprocess, google-genai SDK) is already permissive.
- Detector swap is contained to `backend/indexer/yolo_hybrid.py` (the `from ultralytics import YOLO` load + the `model.track(...)` call) plus `config.json` knobs and `tests/test_yolo_hybrid.py` mocks. Stage-1 feature outputs (velocity/active-frame/track-id stats) and Stages 2?4 are detector-agnostic and unchanged.
- We now own the frame loop + tracking (ultralytics did both internally). Use a permissive ByteTrack (e.g. `supervision`, MIT, or the MIT ByteTrack repo) on top of `lapx` (already a BSD dep).
- Residual items needing legal sign-off: exact codec-pool royalty thresholds (AVC/HEVC decode of uploads), Ultralytics Enterprise pricing, and per-checkpoint weight licenses ? tracked in `docs/MONETIZATION_AUDIT.md` §7.
- The `yolo_hybrid` registry key is now a slight misnomer (no YOLO); keep it for back-compat or rename in a follow-up.

ADR-006: Golden-Set Generation via Vendor Bake-off + a Pluggable Annotation Provider

Date: 2026-05-30
Status: Accepted (process v1; vendor shortlist pending research)

Context

The product needs **golden-set ground truth** (hand-verified rally `[start,end)` boundaries) to (a) measure pipeline quality and (b) run **on-demand / surprise regression tests** against the production tool. Two forces shape the design: the hosted-API monetization goal raises a "who views the user's video" concern (ADR-005 / MONETIZATION_AUDIT.md), and the founder is **Bangalore-based** and prefers **local/India vendors**. We already own a deterministic interval scorer (`backend/eval/metrics.py`) and a growing collection of **licensed/owned** match videos.

Decisions locked with the owner: **licensed/owned video only** (no user uploads to vendors) · **rally `[start,end)` annotation depth** · **tiered turnaround** (fast automated + slow human) · **vendor bake-off** to choose · **Bangalore/India-local preference**.

Decision

1. **One scorer, three jobs.** A vendor's `[start,end)` output is structurally identical to a prediction, so `metrics.py` grades product-vs-truth, vendor-vs-key, and annotator-vs-annotator (IAA) with no new scoring code.

2. **Select vendors by measured quality-per-dollar**, via a bake-off: write a precise rally-definition guideline, build a small internal gold key (5?10 licensed clips), send the same clips to 2?3 vendors (favoring Bangalore/India), and grade every submission with our own harness. The numbers are the decision.
3. **Pluggable `AnnotationProvider` registry** mirroring the existing registry pattern: `FileProvider` (today's manual/ShuttleSet flow), `HumanVendorProvider` (slow tier ? authoritative truth), `AutomatedProvider` (fast tier ? independent model oracle).
4. **Regression = existing harness + one new component.** `regress(video)` calls a provider for ground truth, runs the production pipeline, and scores with the unchanged `metrics.py` against a per-video `baseline.json`.
5. **Privacy by construction.** Golden sets use only licensed/owned footage; all third-party video egress flows through the single provider boundary where DPA / no-reuse / delete-on-delivery / access-logging controls live.

Consequences

- Eliminates the "human views user video" concern for golden-set/regression work, because user uploads never enter the pipeline.
- **The oracle problem is a first-class caveat:** the fast-tier `AutomatedProvider` must be a *different model lineage* than production (else shared blind spots make the test falsely green); fast-tier alarms escalate to the slow human tier before blocking a release.
- New docs: `docs/GOLDEN_SET_VENDORING.md` (the full process v1) and `docs/VIDEO_RECORDING_GUIDELINES.md` (a tracked draft: capture-quality thresholds to be derived empirically from precision/recall, since input quality bounds output quality ? cf. ADR-004's broadcast-camera finding).
- The vendor shortlist itself is **pending a primary-source research pass** (favoring India/Bangalore) and is intentionally left blank until verified ? no un-sourced vendor claims.
- **Honest limitation:** a licensed/owned corpus may not fully represent real user capture, so regression greens are strong evidence, not guarantees ? the recording-guidelines work aims to narrow that gap.

Update (2026-06-02): shortlist researched; Roora selected for the initial pilot

- The **vendor shortlist** is filled in (GOLDEN_SET_VENDORING.md §6, primary-source pass 2026-05-30).
- The founder selected **Roora ML Pvt Ltd** (HSR Layout, Bengaluru) for the **initial paid pilot**, diverging from the doc's iMerit/Taskmonk default toward a Bangalore-local, startup-friendly shop. Roora's existence, image/video/temporal capability, and location are verified (rooragrp.com); **pilot quality is still to be measured** ? run as a **single-vendor pilot first** (the pilot *is* the calibration), not a multi-vendor bake-off.
- **Step 0 ? the annotation guideline ? is authored** (the highest-leverage artifact): collector repo `docs/annotation/` ? `ROORA_BRIEF.md` (rally CSV schema), `ANNOTATION_GUIDE.md` (illustrated guide), plus intake-manifest and NDA templates.
- **Pilot scope is 3 sports** (badminton, pickleball, table tennis); the collector's `import-local` ingestion for all three is implemented (collector ADR-001).
- **Still pending:** the internal **gold key** (5?10 expert-labeled clips) and the vendor-dependent **GS-3 `HumanVendorProvider`** ? GS-1/GS-2/GS-4 (FileProvider, regression runner, automated-oracle scaffold) have already landed (see GOLDEN_SET_IMPLEMENTATION.md).

ADR-007: Audio "thwack" hit-detection (fused with WASB) is the next rally-detection cue; pose/stance parked

Date: 2026-06-07
Status: Accepted (direction); experiments gated on golden eval labels

Context

After building the WASB shuttle-trajectory substrate (ADR-001), a Gemini-teacher ? local-student distillation (ADR-004 lineage), and measuring both, two honest findings emerged: **(a)** threshold-tuning the local windowing ceilings around F1 ? 0.44 and the learned student *underperforms* the threshold baseline at n=2 labeled videos; **(b)** the dominant failure is **candidate RECALL** ? WASB-derived windows simply don't align with (or miss) true rallies (IoU-0.5 recall ceiling ~0.43 on one clip). To decide whether teacher?student was even the right backbone, we ran a deep-research survey (`docs/RALLY_DETECTION_RESEARCH.md` ? 23 sources, 25 adversarially-verified claims). Verdict: **teacher?student is sound but NOT uniquely best**; the strongest direction is a **complementary, fully-local multi-cue stack**, and the **#1 cheapest lever is audio hit detection fused with the trajectory** (audio+visual fusion measurably lifts rally *recall* ? tennis HMM 83% fused vs 54% visual / 63% audio). Pose/court serve-gating is the validated #2 but is engineering-heavier.

Decision

1. **Adopt a fully-local AUDIO hit-detection cue** (shuttle "thwack" impacts) and **fuse it with the WASB trajectory** ? primarily to recover rallies WASB misses (recall) and secondarily as features for the distilled student. Detailed design: `docs/AUDIO_RALLY_DETECTION_PLAN.md`.
2. **Gate on an empirical GO/NO-GO probe (E1).** Shuttle impacts are quiet vs tennis/TT, so before building anything heavy, measure on *real GoPro audio* whether hit-clusters align with golden rallies and recover WASB misses. Build E2/E3 only if E1 passes.
3. **Keep Gemini as an OFFLINE teacher only** (never runtime) ? research confirms this is the correct role for a cloud VLM; it is not the deployed localizer.
4. **Park pose/court-geometry stance detection as the #2 cue** (serve-ready START + flight-gradient END). It is expected to be substantially harder to get right; revisit after audio (and a TBD owner hunch).
5. **Make "audio always enabled" a capture requirement** (`docs/VIDEO_RECORDING_GUIDELINES.md`), since the cue is only as good as the recording.
6. Stay permissive (ADR-005): classical DSP (numpy + stdlib `wave`, BSD), no restrictive weights; any later model trained in-house.

Consequences

- New `backend/pipeline/audio/` module (extraction + onset/peak + confidence), feeding `distill_local` and scored by the existing `metrics.py` / golden loaders. *(As built: the detector is **pure numpy + stdlib `wave`, no scipy** ? the anticipated scipy dependency proved unnecessary.)*
- **All audio measurement is blocked on golden eval labels** ? which the VLC rally-annotator (now a standalone MIT, multi-sport open-source repo: https://github.com/avidullu/rally-annotator) lets the owner self-produce, and the in-repo `--labels` loaders / collector `import-local` ingest.
- Sport-generic: tennis/table-tennis hits are *louder*, so the cue should transfer at least as well there.
- **Honest risk:** if E1 shows shuttle SNR is too poor on real amateur audio, audio is deferred ? this ADR commits to the *experiment and direction*, not to a guaranteed win.

Integration architecture: modular fusion now (Option A), an owned end-to-end model later (Option B)

A recurring design question: do we **feed audio into WASB** so it refines its own output, or **keep audio as a separate signal fused in our code**? The answer today is the latter ? and understanding *why* sets up the long-term route.

Why fusion lives in our rally layer, not inside WASB (Option A ? now). WASB is a **frozen, vision-only shuttle *detector*** (image frames ? per-frame shuttle `(x, y, visibility)`; MIT weights we *run*, not train). It has no audio pathway and it does not decide rallies at all ? **rally detection is our own layer** on top of its trajectory (`trajectory_to_action_windows` ? gate ? the distilled classifier). Making WASB consume audio would mean bolting on an audio encoder and **retraining the network**, which needs per-frame shuttle-coordinate-*plus*-audio labels we don't have (we only have rally `[start,end]` labels) and would fork the clean MIT model. It also conflates two different questions: WASB answers ***where* the shuttle is**; audio answers ***when* a hit happened**. Audio doesn't make WASB localize better ? it's an independent rally-activity cue. So fusion belongs where cues are *combined into rallies*: our layer.

```
frames ?? WASB (frozen, MIT) ?? shuttle trajectory ??
audio  ?? hit detector (ours) ?? hit events ??????????? RALLY LAYER (ours) ?? rallies
[pose  ?? stance heuristic (ours; the parked #2) ????   (feature + decision fusion)]
```

Two complementary fusion modes (both in `docs/AUDIO_RALLY_DETECTION_PLAN.md`):
1. **Feature-level (primary):** audio features (hit_rate, inter-hit cadence/regularity, time-since-last-hit) join WASB-derived features (velocity, net-crossings) in the **single distilled classifier** that emits rallies ? "use audio to sharpen our rally metrics" (experiment E3).
2. **Decision-level recall-recovery:** where WASB proposed nothing but a dense hit-cluster exists, propose a rally there ? directly correcting WASB's recall misses.

Each cue (WASB, later the person/pose detector) stays a **swappable black box**; every signal is independently testable ? which matters while we're label-starved.

The long-term route (Option B ? the intended destination, once a healthy golden corpus exists). As we accumulate enough golden eval/training videos (owner self-rating via the annotator + vendor labels), the right end-state is to **collapse the modular cues into a single OWNED multimodal rally model**, trained/fine-tuned end-to-end on (frames + audio + pose/stance + ?) ? rally boundaries ? the HitNet / temporal-action-localization direction surfaced in `docs/RALLY_DETECTION_RESEARCH.md`. Crucially, that lets us **fine-tune from our own code and data instead of depending on WASB's pretrained weights** ? removing both the *recall ceiling* imposed by WASB's candidate windows and the *licensing question* on its academic-trained weights.

Why we don't jump to Option B now ? and why Option A is the path to it. Option B needs data we don't yet have, and it's far harder to debug as one black box. So we deliberately build **modular cues first** ? audio (this ADR), then a stance heuristic (the parked #2), then other signals ? each cheap, debuggable, label-efficient, and *each one generating the very golden labels an end-to-end model will need*. When the corpus is healthy, we train the unified model. **Option A is not a detour from Option B; it is how we earn it.**

387

13. user (2026-06-10T05:33:56.141Z)

Checkpoint: 2026-06 Strategy Foundation (post-scaffolding)

Date: June 2026
STATUS: COMPLETED
Trigger: Owner merged the post-scaffolding strategy docs ? #52 (config/identity cleanup), #53 (deferred-hardening design), #55 (commercialization tracker), #57 (PMF market study), #58 (platform architecture).

Purpose
Captures the state once the **strategic / direction-setting foundation** for the post-scaffolding phase was laid: the commercial dimension, market/PMF, the multi-user platform architecture (pluggable storage/compute + the owned-model layer), and the deferred-hardening designs. **Strategy only ? no implementation started.** Provides a regression-detection baseline for priorities/decisions as quality + implementation work begins.

Key living documents at checkpoint time
- **COMMERCIALIZATION.md** ? commercial tracker (dashboard C1?C14, grouped Pending ? In-flight ? Completed with a P0/P1/P2 column).
- **PMF_MARKET_RESEARCH.md** ? cited market study; beachhead = **India badminton academies**.
- **PLATFORM_ARCHITECTURE.md** ? multi-user "assistant coach" platform: pluggable `StorageBackend`/`ComputeBackend` (BYO), generic schema (coach overlay optional), owned video-QA model layer (§5A), training-backend survey + nanochat-style harness, committed legal guardrail, per-user-adapter direction, v1-vs-vision scoping.
- **DEFERRED_HARDENING_PLAN.md** ? packaging, train/eval guardrails, async job model (#16) ? *pending approval*.
- **CODE_MAP.md / BACKLOG.md / ROADMAP.md** ? updated for the config/identity cleanup.

See git history around #52/#53/#55/#57/#58 for exact snapshots.

Committed decisions / non-negotiables recorded here
- **Paid frontier models (Gemini/OpenAI/Anthropic): inference / serving / fallback only ? never a training data source** (competing-model terms). Teachers/bases = open Apache-2.0 only.
- **Owned-model base = Apache-2.0-clean, video-capable** (Qwen2.5-VL-7B/32B, Qwen3-VL, Gemma 4); **reject Gemma 3** (viral license clause); **minors excluded** from the training pool; per-user training needs a separate opt-in.
- **v1 = rally detection + highlight compilation + per-user history + coach/user correction loop** ("assistant-coach lite"); **gait/biomechanics strictly future** (Art. 9).
- **Generic data schema** (User ? VideoCollection ? Video ? Highlight ? HighlightMetadata); coach ? athlete is an **optional overlay** (a market-fit hypothesis).

What comes next (per owner)
- First implementation slice = **async job model (DEFERRED #16)**, single-tenant ? **pending explicit go-ahead** (no implementation started without approval).
- Quality iterations with new labeled videos; golden-set expansion + the **golden QA eval set** (the prerequisite for the owned-model training harness).

When the first implementation slice or a quality milestone completes, add a new dated checkpoint here (or under `quality-iterations/`).

14. user (2026-06-10T05:33:56.695Z)

Owned-model training study (rally segmentation ? end-to-end multimodal)

Date: 2026-06-08 · **STATUS: DESIGN BLUEPRINT (no implementation until PLATFORM P0?P4 + healthy corpus; owner sign-off required).**

> How to train our **OWNED** rally/highlight model **reusing the videos we label**, aligned to the

> Apache-2.0 / no-paid-LLM-training / privacy-local guardrails. Produced by an 11-agent research
> workflow (ground ? 5 web-research dimensions ? adversarial verification of costs/licenses/privacy ?
> synthesis + completeness critic; 35 sources). Empirical results from THIS session (LOVO + a learned
> prototype) are folded in and **resolve the study's biggest open unknown** ? see §"What we proved today".
> Numbers from web sources are 2026 snapshots in a ±15?25% band; **re-quote live before budgeting.**

Update (2026-06-09, post-PR-#61-review) ? authoritative roadmap is now `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md`
A verified strategy workflow + owner review refined this study. Key changes (the plan supersedes where they differ):
- **VISION (foregrounded):** the goal is **rally/shot/player analytics + highlights for ALL net-separated racquet
  sports** (badminton/tennis/TT/pickleball/padel) ? eventually all sports, as **ONE sport-agnostic, conversational
  model**. This does **not** change the model/framework picks; it scales **data + per-sport trajectory detectors** (the
  accepted "good problem") and adds a licensing front (clean per-sport detectors). Single-model temporal generalization
  is an **open hypothesis** (corpus is badminton-only; the once-cited RacketVision evidence was **refuted** ? it's
  ball-tracking, not temporal).
- **Framework = ms-swift (Apache-2.0)** (only forge with native video+audio+timestamp-grounding+video-GRPO).
  Base = Qwen3-VL (Apache-2.0, uniform sizes, timestamp emission) ? primary, **replacing the Qwen2.5-VL-7B framing**.
- ? **Gemma 4 stays AMBER and bench-only** (NOT "confirmed clean" as the table below says): Apache grant likely but
  Prohibited-Use posture unconfirmed (counsel) + video/timestamp capability disputed ? not the primary extractor.
- **Start with the TAL head, not the VLM** (VLMs miss strict boundaries ? boundary accuracy is the product). Head now ? VLM later.
- **Immediate goal = a high-P/R extraction *harness*, not on-device** (on-device deferred). **Reframe supersedes the on-device-now framing.**

Thesis

Train the owned model in **three staged generations off ONE model-agnostic labeled corpus** ? a label
collected once feeds every generation, so labels are never re-collected. **The moat is the consented
dataset + the eval harness, never the weights.**

Gen	When	Approach	Base / model	Compute
Gen-0	now ? unblocks at n?5 matches	tiny **temporal action segmentation (TAS)** head over a hand-built per-frame feature stack from the WASB trajectory we already produce; per-frame rally(1)/non-rally(0) ? merge into [start,end]	**ASFormer (MIT, ~1.1M params)** primary; **ActionFormer (MIT)** A/B challenger (overtakes once corpus = hundreds of matches)	**local RTX 2070 Super**, trains in minutes, $0
Gen-1 / Option A	6?12 mo	concatenate more local cue channels onto the SAME head: trajectory + audio-onset + pose/court serve-gating; fusion in our rally layer, cues swappable	same ASFormer/ActionFormer, wider input	local + occasional cloud burst (pseudo-labeling)
Gen-2 / Option B	12?24+ mo, after scaffolding + healthy corpus	end-to-end multimodal VLM: (frames+audio+pose+trajectory) ? rally [start,end] (+semantics); SFT cold-start then **GRPO/RFT with a temporal-IoU-vs-golden reward**	**Qwen3-VL-4B-Instruct** (Apache-2.0, on-device) primary; **Qwen2.5-VL-7B** fallback; **Gemma 4 E4B/12B-Unified** (Apache-2.0 + native video+audio) end-state	rented 24?48 GB A100/L40S (does NOT fit 8 GB box)

What we proved today (de-risks the blueprint)

The study's **"dominant empirical unknown"** was: *can a low-dim WASB trajectory (?3?4 raw dims) feed a TAS head, given ASFormer's small-data result used 2048-d I3D features?* This session answered it.

- **Cross-video LOVO of the current heuristic** (`calibrate_local`, Adarsh-96 + Kushagra-32, human labels):
  mean held-out F1 **0.443 ± 0.279**; Adarsh 0.722, **Kushagra 0.163** (its own ceiling ? *no* velocity-windowing config does better). **The ±0.279 spread is method variance, not config.** ? This also **corrects the study's "~0.73 bar"**: that was a fit-on-self single-match number; the honest cross-video heuristic bar is **~0.44**, which *lowers* the Gen-0 ship gate.
- **A learned per-frame prototype** (`backend/eval/rally_seq_proto.py`: 6 hand-built features ? visibility rate, mean/max shuttle speed, **net-crossing rate**, x/y spread ? into the repo's dependency-free LogisticRegression, honest LOVO): **per-frame AUC 0.878 (Adarsh) / 0.841 (Kushagra), held-out.** On Kushagra ? the match the heuristic *cannot* segment (0.163) ? a calibrated threshold reaches **segment F1 0.578 (3.5×)**. The naive cross-video threshold gives only 0.049, so **the remaining gap is threshold/operating-point calibration at n=2, not representation.**

- **n=3 update (doubles match added ? `LargeTestVideo`, 49 rallies):** the learned prototype **overtakes the heuristic.** Mean held-out LOVO F1 ? learned **0.377 ? 0.615**, heuristic **0.443 ? 0.544**. Kushagra's learned held-out F1 jumped **0.049 ? 0.417** (one added match closed most of the calibration gap); the **doubles** match, held out and trained only on the two singles, scored AUC **0.914** / F1 0.729 (generalizes across play type + frame rate). The heuristic still cannot rescue Kushagra at any n (0.151, capped). **The honest "bar to beat" is thus ~0.54 (n=3), not 0.73 ? and the learned path already clears it on the mean and keeps rising with n.**

Conclusion: the rally signal IS learnable from the WASB trajectory features, cross-footage (AUC ~0.85?0.91) ? the heuristic just can't exploit it. This **validates the Gen-0 ASFormer-over-WASB-features path**, shows the hand-built feature stack the study calls for is the right lever, and confirms the bottleneck is **data + per-video calibration (n?5 matches), not method.** The prototype is the seed of Gen-0, and the n=2?n=3 trend (learned mean +0.24, overtaking the heuristic) is the empirical green light.

Platform ranking (privacy + cost lens; the fine-tune is a sub-24h, $10?50 job ? privacy/ergonomics dominate)

1. **Modal ? PRIMARY.** Strongest *written* no-access contract (verbatim: "Modal will never access? the inputs/outputs to your Functions, or any data you store"), ?7-day TTL, SOC2-II, HIPAA-BAA; per-second scale-to-zero matches retrain-on-each-batch. A100 80 GB ~$2.50/hr, H100 ~$3.95/hr (corrected down from cited). Privacy is the load-bearing reason, not price.
2. **RunPod (Secure Cloud tier ONLY) ? FALLBACK / Option-B scale-up.** Cheapest vetted-DC self-serve; A100 80 GB ~$1.39?1.49/hr. Community/P2P tier is UNVETTED ? **never** touches consented video. Confirm AES-256/DPA at booking.
3. **Vertex AI / GCP ? ON BENCH.** Only strong-privacy option with an India region, but it's **capacity-gated** (asia-south1 high-end GPU is invitation-only, H100-class A3 not A100) and **tuning Gemini yields NO portable on-device weights** ? only tuning an Apache-2.0 base in Model Garden does. Use only if India residency-at-rest is mandated.

4. **Self-hosted RTX 2070 Super 8 GB** ? Gen-0 training + ALL inference + CI only. **Cannot** QLoRA a 7B video VLM (~20?24 GB). Best privacy (nothing leaves the box).
5. **Fireworks** (managed, contractual no-train + ZDR) usable; **Together BLOCKED** until a signed DPA + explicit no-train clause. AWS SageMaker / Azure ML = heavy for a solo founder; India-residency only.

Label reuse ? one corpus, three transcoders (the actual moat)

Promote the rally CSVs from eval-only files to **one model-agnostic corpus**: JSONL, one row per `(video_id, rally_ordinal)` = `{start, end, ending_reason, sport, source (human|pseudo|vendor), annotator/provenance, consent/opt-in, label_type, split (BY MATCH not clip), trajectory_ref}`. The provenance/consent fields are the **enforcement point** for the legal guardrails. Three deterministic transcoders read it, zero re-labeling:
1. **Heuristic/LogReg view** ? already exists (`backend/eval/training.py::label_candidates` via `metrics.match_intervals`).
2. **TAL view** ? ActivityNet JSON (`segment:[start,end]`, `label:'rally'`); our seconds map 1:1.
3. **VLM clip?QA view** ? "list every rally [start,end] in this clip"; `ending_reason` ? a second QA task.

One scorer spine: reuse `metrics.match_intervals` for heuristic-labeling, TAL eval, AND the Gen-2 RFT reward ? no metric drift. **Leakage guard:** split **by match, never by clip**. **Amplification (all guardrail-clean):** open-teacher pseudo-labels (WASB/own-model, confidence-gated ? never paid-LLM), boundary-preserving temporal augmentation, active learning to spend the Roora budget on highest-uncertainty matches. ? **The new partial-label windowing** (`eval_partial_labels.restrict_to_window`) must flow into the TAL/VLM views so prefix-labeled videos aren't mis-scored (a corpus that's partially tagged is the norm).

Licensing & privacy guardrails (all respected; verify per-checkpoint)

- **Bases: Apache-2.0/MIT only.** APPROVED: Qwen2.5-VL-7B/32B (Apache-2.0), Qwen3-VL-4B/8B (Apache-2.0), Gemma 4 E2B/E4B/12B-Unified (Apache-2.0 + native video/audio), InternVL3 ?38B (MIT/Apache), ASFormer/ActionFormer/TriDet (MIT). **REJECTED:** Gemma 3 (viral Gemma Terms + PUP), Qwen2.5-VL-3B (NC research), Qwen-72B/InternVL3-78B (100M-MAU cap), Llama Vision (700M-MAU), DAMO VideoLLaMA3 weights (NC + paid-LLM-trained), TimeChat/VTimeLLM/Momentor (NC/no-license = method-reference only), VideoMAE weights (CC-BY-NC), `yabufarha/ms-tcn` (Commons Clause ? use `sj-li/MS-TCN2` MIT). ? **Gemma 4 size names = E2B/E4B/26B-A4B/31B/12B-Unified ? there is no plain "4B".**
- **No paid-LLM training data** ? Gemini/OpenAI/Anthropic = inference/eval/teacher-reference only, **never** a training signal. Gemini silver labels stay out of every transcoder.
- **Exclude minors by default + separate training opt-in;** per-user adapters trained only on that user's consented video (deletion = drop the adapter). Enforced at the corpus-membership layer before transcoding.
- **Privacy lever = local inference** (Gen-0 ~1.1M params; Gen-2 4B quantized). Any cloud *training* needs a signed DPA + no-train assurance; route only owned/consented footage, never user uploads.

Cost (spend on labels, not GPUs)

```
  - **Gen-0:** ~$0 (local 2070 Super; trains in minutes; ~4.5 GB VRAM).
  - **Gen-2 LoRA:** ~$5?40/run on a rented 24?48 GB GPU (RunPod cheapest; **VRAM/token cost on real 1080p/60fps
    clips is UNVERIFIED ? 24 GB is a floor, re-benchmark**). Managed (Fireworks/Together) ~$2?30 *if* text-priced
    (video tokens can balloon). RFT = compute-heavy (multiple rollouts/sample).
  - **Inference crossover (the unanchored number):** self-hosted 7B beats Gemini (~$0.05/10-min clip) **only**
    above sustained QA volume; below it, **paid Gemini is cheaper** ? paid-primary now, owned-primary later.
    **Benchmark $/call before committing** (COMMERCIALIZATION C7).

## Open risks & critic caveats (read before acting)
```

15. user (2026-06-10T05:34:01.817Z)

```
## Open risks & critic caveats (read before acting)

  - **?? WASB checkpoint provenance (C3) propagates.** The study calls WASB "MIT," but that's the *code*; the
    distributed `wasb_badminton_best.pth.tar` is **academic-data-trained, provenance unresolved** (COMMERCIALIZATION
    C3, still open). Every Gen-0/1 plan uses WASB features as input AND as a pseudo-label teacher ? a "clean" owned
    model trained on provenance-unclear features **does not launder the provenance.** ROADMAP Phase-4 "train own WASB
    weights" is a **prerequisite, not an aside.**
  - **Gemma 4 "clean + native-video" rests on post-cutoff secondary sources** ? keep AMBER until a raw HF LICENSE
    revision is verified, given the Gemma-3 rejection. Its timestamp-emission recipe is less documented than Qwen's.
  - **Existing distilled model was trained on Gemini silver labels** ? the (correct) eval-only policy now **forbids
    this; that pipeline must be purged of Gemini-derived training labels** to comply.
  - **Policy ratifications needed:** guardrail literally says "Apache-2.0 only" but the approved list includes MIT ?
    ratify "Apache-2.0/MIT". The MAU-cap rejections (72B/78B) may be over-conservative at our scale.
  - **On-device privacy claim is unbenchmarked** ? no tokens/sec for a quantized 4B VLM on 8 GB Turing at
    1080p/60fps. **Measure before relying on it.**
  - **Amplification is load-bearing for Gen-2** ? 128 raw rallies can't reach the ~5k?20k clip?QA band alone; if
    augmentation + QA-multiplexing + pseudo-labels underdeliver, Option B is under-fed. Amateur 60fps may need more
    labels than broadcast-trained literature implies.
  - **Dropped option worth keeping:** InternVideo2 encoder (MIT) + light grounding head ? the *only* data-realistic
    owned-video experiment available at n?128 today. And: a **one-time ~$1?2k 24 GB local card** (e.g. used 3090)
    would preserve the privacy lever for *training* and dissolve most of the cloud-DPA analysis ? uncosted here.

## Open questions (need the GPU/WSL machine-side suite ? not runnable in this dev container)

1. Does ASFormer/ActionFormer over the WASB feature stack beat the **honest ~0.44 LOVO** bar at n?5 matches?
    (Today's prototype says the per-frame AUC is there ? 0.84 ? so this is now *likely*; confirm at segment level.)
2. Real VRAM + token cost of fine-tuning Qwen3-VL-4B / Qwen2.5-VL-7B on our actual clip?QA frame counts?
3. Working Gemma-4 timestamp-emission recipe + exact checkpoint (E4B vs 12B-Unified)?
4. Measured self-hosted-7B $/call + latency vs Gemini on the target workload (the owned-model ROI crossover)?
```

5. Will Together/Lambda/Azure sign a DPA with explicit no-train for consented video, India-configurable?
6. Realistic clip?QA yield from 128 rallies via augmentation + QA-multiplexing + pseudo-labels?

Sequencing (owner directive, non-negotiable)

This is a **design blueprint only.** It unblocks PLATFORM **P5** but starts **no implementation** until P0?P4
scaffolding (async jobs, storage/compute backends) is rock-solid AND the golden corpus is healthy. Paid Gemini
stays the always-ready primary/fallback. The only items safe to act on now ? **and even these need owner
sign-off before code** ? are (a) the model-agnostic corpus format and (b) growing n?2 ? n?5 matches (a small
Roora batch), which unblocks honest tuning immediately and is the **highest-ROI spend (labels, not GPUs).**

Full agent transcript + 35 sources: workflow `wf_49aab888-e10` (this session).

16. user (2026-06-10T05:34:13.032Z)

```
Found 2 files
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\training.py
```

17. user (2026-06-10T05:34:14.505Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py:105:    default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter", "motion")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py:371:        seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None), "default_segmenter", "motion")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py:47: default_segmenter: str = "yolo_hybrid"
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py:69: @field_validator("default_segmenter")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\harvest.py:210: config_default = (config.get("indexing") or {}).get("default_segmenter")
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\spec.yaml:112:      The segmenter that ran differs from config.json's default_segmenter, and no
C:\Users\avidu\Projects\badminton-highlight-indexer\README.md:74:This outputs a validation summary and indexes all rallies directly into the local SQLite database, using whichever segmenter is set as `default_segmenter` in `config.json` (`yolo_hybrid` by default).
C:\Users\avidu\Projects\badminton-highlight-indexer\README.md:124:        "default_segmenter": "yolo_hybrid",
C:\Users\avidu\Projects\badminton-highlight-indexer\README.md:155:* `indexing.default_segmenter` ? `yolo_hybrid` | `gemini` | `motion`.
C:\Users\avidu\Projects\badminton-highlight-indexer\README.md:215:   *Or set `"default_segmenter": "yolo_hybrid"` (or `"gemini"`/`"motion"`) under `"indexing"` in `config.json`.*
C:\Users\avidu\Projects\badminton-highlight-indexer\README.md:219:[Omitted long matching line]
C:\Users\avidu\Projects\badminton-highlight-indexer\README.md:237:       "default_segmenter": "gemini",
C:\Users\avidu\Projects\badminton-highlight-indexer\config.json:26:        "default_segmenter": "gemini",
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md:29:  -> seg_name = req.segmenter or global_config.indexing.default_segmenter (fallback 'motion')
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md:86:  - ? `--segmenter` argparse `choices` frozen at build time from `segmenter_registry.list_available()` ? a lazily-registered segmenter wouldn't appear. CLI default_segmenter fallback `'motion'`.
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md:96:[Omitted long matching line]
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md:102:- `@config.json` ? on-
```

disk config; mirrors `AppConfig`. ? **`indexing.default_segmenter='gemini'`** diverges from model default `'yolo_hybrid'` ? file wins at runtime.
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md:177:[Omitted long matching line]
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md:238:- **Segmenter default:** single source of truth = model `IndexingConfig.default_segmenter='yolo_hybrid'`; `config.json` may override at runtime (ships `gemini`). run_*.py read the model value (no literals); `main.py` keeps the defensive `'motion'` getattr fallback noted above.
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md:264:| Change a config default / add a config field | `@backend/config/models.py` (single source of truth) -> regenerate via `setup.py`/`save_default_config`; ? config.json may override (default_segmenter) |
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\TRACKNET_WSL_SETUP.md:71:Then select it as the segmenter (e.g. set `indexing.default_segmenter` to
C:\Users\avidu\Projects\badminton-highlight-indexer\run_process_and_stitch.py:72:
seg_name = cfg.indexing.default_segmenter
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_config.py:24:   assert cfg.indexing.default_segmenter == "yolo_hybrid"
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_config.py:34:
"default_segmenter": "gemini",
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_config.py:45:   assert cfg.indexing.default_segmenter == "gemini"
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_config.py:58:
"indexing": {"default_segmenter": "yolo_hybrid"},
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_config.py:66:   assert cfg.indexing.default_segmenter == "yolo_hybrid"
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_config.py:78:   assert cfg.indexing.default_segmenter == "yolo_hybrid"
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_config.py:122:   assert cfg.get("indexing", {}).get("default_segmenter") == "yolo_hybrid"
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py:46:
config={"ai_provider": "gemini", "ai_model": "gemini-flash", "indexing": {"default_segmenter": "wasb_hybrid"}},
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py:98:
config={"indexing": {"default_segmenter": "motion"}})
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py:107:   cfg = {"ai_provider": "gemini", "indexing": {"default_segmenter": "wasb_hybrid"}}
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py:150:
config={"ai_provider": "gemini", "indexing": {"default_segmenter": "wasb_hybrid"}},

18. user (2026-06-10T05:34:21.768Z)

```
"""Build a training set for the offline, API-free learned rally segmenter.

Phase 4 of the collector->indexer bridge (see docs/ROADMAP.md, ADR-004). The idea:
a detector (YOLO player-motion today, optionally WASB shuttle-motion later)
produces candidate rally windows with features; the collector's ground-truth
rally intervals tell us which windows are real. A lightweight classifier learns
that mapping, replacing the paid Gemini refinement with a local model.

This module is the *data layer* ? pure and unit-tested:

  - `label_candidates`  ? generic: label windows positive/negative by whether they
    match a ground-truth rally (same IoU matching the evaluator uses, so training
    labels are consistent with how we score).
  - `extract_yolo_features` ? YOLO-specific feature vector from a candidate row's
    persisted `stats`. Pluggable: a WASB substrate supplies its own feature_fn.
  - `build_training_set` ? glue: rows + GT -> (X, y, intervals).

No model training or I/O here; that lives in the trainer/segmenter once the
substrate is chosen from the Phase 3 candidate-recall result.
"""

from typing import Callable, Dict, List, Tuple

```

```
from backend.eval.metrics import Interval, match_intervals

# YOLO Stage-1 persists these per candidate window in `candidate_segments.stats`
# (see HybridYoloSegmenter). Duration is derived from the window bounds.
YOLO_FEATURE_NAMES = [
    "duration", "peak_velocity", "mean_velocity", "active_frame_count", "track_id_count",
]


def extract_yolo_features(row: Dict) -> List[float]:
    """Feature vector for one YOLO candidate row (dict from query_candidate_segments).

    `stats` is already JSON-deserialized by the DB layer. Missing fields default
    to 0.0 so a sparse/old row still yields a well-formed vector.
    """
    stats = row.get("stats") or {}
    duration = float(row["end_time"]) - float(row["start_time"])
    return [
        duration,
        float(stats.get("peak_velocity", 0.0)),
        float(stats.get("mean_velocity", 0.0)),
        float(stats.get("active_frame_count", 0)),
        float(stats.get("track_id_count", 0)),
    ]


def label_candidates(candidate_intervals: List[Interval], gt_intervals: List[Interval],
                     iou_threshold: float = 0.5) -> List[int]:
    """Label each candidate window 1 (real rally) or 0 (spurious).

    A window is positive iff it is matched one-to-one to a ground-truth rally at
    `iou_threshold` ? the *same* matching the evaluator uses for true positives,
    so the classifier is trained on exactly what we score. Substrate-agnostic:
    works for YOLO or WASB-derived windows alike.
    """
    mr = match_intervals(candidate_intervals, gt_intervals, iou_threshold)
    positive = {m.pred_index for m in mr.matches}
    return [1 if i in positive else 0 for i in range(len(candidate_intervals))]


def build_training_set(
    candidate_rows: List[Dict],
    gt_intervals: List[Interval],
    iou_threshold: float = 0.5,
    feature_fn: Callable[[Dict], List[float]] = extract_yolo_features,
) -> Tuple[List[List[float]], List[int], List[Interval]]:
    """Turn one video's candidate rows + ground truth into (X, y, intervals).

    `feature_fn` is pluggable so a WASB substrate can supply shuttle-motion
    features without changing the labeling logic. Returns the feature matrix `X`,
    binary labels `y`, and the candidate intervals (handy for downstream merging).
    """
    intervals: List[Interval] = [
        (float(r["start_time"]), float(r["end_time"])) for r in candidate_rows
    ]
    y = label_candidates(intervals, gt_intervals, iou_threshold)
    X = [feature_fn(r) for r in candidate_rows]
    return X, y, intervals

```

19. user (2026-06-10T05:34:30.473Z)

```
"""Distill Gemini (teacher) into a LEARNED local rally classifier (student).

```

```
Threshold-tuning the local knobs hit a low ceiling (see calibrate_local). This goes
further: learn "is this WASB candidate a real rally?" from Gemini silver labels,
using a few resolution/fps-INVARIANT local features, so the model transfers across
videos and runs 100% local at inference (no Gemini, privacy lever intact).

Flow (all local at runtime):
  WASB trajectory -> recall-oriented candidate windows (propose many; classifier
  filters) -> per-candidate features [duration, peak_velocity, mean_velocity,
  active_ratio, net_crossings] -> LogisticRegression(prob >= threshold) -> merge
  -> rally windows.

Training labels come from Gemini full-context labels via the SAME IoU matching the
evaluator uses (training.label_candidates). Honest leave-one-video-out: train on the
other videos' candidates, predict the held-out one, score vs its Gemini labels, and
compare against the threshold baseline. Reuses classifier.py, training.label_candidates,
rally_gate, metrics, calibrate_wasb.load_golden_gts, gemini_refine.merge_overlaps.

Run:
    python -m backend.eval.distill_local --manifest videos.json --model-out output/local_rally_model.json
"""
from __future__ import annotations

import json
import argparse
from typing import Dict, List, Tuple

import numpy as np

from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
from backend.pipeline.detectors import rally_gate
from backend.eval.calibrate_wasb import load_golden_gts
from backend.eval.training import label_candidates
from backend.eval.classifier import LogisticRegression
from backend.eval.metrics import score
from backend.eval.gemini_refine import merge_overlaps

Interval = Tuple[float, float]

FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings"]
# Recall-oriented proposal: deliberately permissive so real rallies are among the
# candidates (the classifier removes the junk). (velocity_thresh, merge_gap, min_dur)
PROPOSAL = (0.005, 1.0, 0.5)
BASELINE_LOCAL = (0.02, 2.0, 2.0)   # today's production-ish windowing, no learning


def build_candidates(trajectory_csv: str, fps: float, frame_width: float,
                     proposal: Tuple[float, float, float] = PROPOSAL) -> Tuple[List[Interval], List[List[float]]]:
    """WASB trajectory -> (candidate intervals, fps/resolution-invariant feature rows)."""
    points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
    vt, mg, mwd = proposal
    wins = TrackNetRunner.trajectory_to_action_windows(
        points, fps=fps, frame_width=frame_width,
        velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
    intervals = [(w["start"], w["end"]) for w in wins]
    gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0)  # for net-crossings
    X: List[List[float]] = []
    for w, g in zip(wins, gated):
        dur = max(1e-6, w["end"] - w["start"])
        win_frames = max(1.0, dur * fps)
        X.append([
```

```
            dur,                                              # rally length (sec) ? fps-invariant
            float(w["peak_velocity"]),                        # already normalized by frame_width
            float(w["mean_velocity"]),
            float(w["active_frame_count"]) / win_frames,      # active-shuttle ratio ? fps-invariant
            float(g.crossings),                               # net crossings (count) ? the rally signal
        ])
    return intervals, X


def baseline_windows(trajectory_csv: str, fps: float, frame_width: float) -> List[Interval]:
    points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
    vt, mg, mwd = BASELINE_LOCAL
    wins = TrackNetRunner.trajectory_to_action_windows(
        points, fps=fps, frame_width=frame_width,
        velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
    return [(w["start"], w["end"]) for w in wins]


def load_video(spec: Dict, iou: float = 0.5) -> Dict:
    fps, fw = float(spec["fps"]), float(spec["frame_width"])
    intervals, X = build_candidates(spec["trajectory"], fps, fw)
    gts = load_golden_gts(spec["labels"])
    y = label_candidates(intervals, gts, iou)
    return {"name": spec.get("name", spec["trajectory"]), "intervals": intervals, "X": X,
            "y": y, "gts": gts, "baseline": baseline_windows(spec["trajectory"], fps, fw)}


def predict_rallies(model: LogisticRegression, X: List[List[float]], intervals: List[Interval],
                    threshold: float = 0.5) -> List[Interval]:
    if not intervals:
        return []
    proba = model.predict_proba(X)
    pos = [iv for iv, p in zip(intervals, proba) if p >= threshold]
    return merge_overlaps(pos, gap_tol=1.0)


def run(specs: List[Dict], iou: float = 0.5, threshold: float = 0.5, model_out: str = None) -> dict:
    videos = [load_video(s, iou) for s in specs]
    print(f"=== Distilled local rally classifier vs Gemini silver-GT "
          f"({len(videos)} videos, IoU>={iou}, prob>={threshold}) ===")
    for v in videos:
        ceil = score(v["intervals"], v["gts"], iou).recall
        print(f"[{v['name']}] {len(v['intervals'])} candidates ({sum(v['y'])} pos) | "
              f"proposal recall ceiling {ceil:.3f} | {len(v['gts'])} Gemini rallies")

    result: dict = {"videos": [v["name"] for v in videos]}
    if len(videos) >= 2:
        print("\n--- HONEST leave-one-video-out: learned vs threshold baseline (held-out) ---")
        clf, base = [], []
        for i, held in enumerate(videos):
            others = [v for j, v in enumerate(videos) if j != i]
            X = [row for v in others for row in v["X"]]
            y = [t for v in others for t in v["y"]]
            model = LogisticRegression().fit(X, y, FEATURE_NAMES)
            preds = predict_rallies(model, held["X"], held["intervals"], threshold)
```

```
            sc = score(preds, held["gts"], iou)
            bsc = score(held["baseline"], held["gts"], iou)
            clf.append(sc)
            base.append(bsc)
            print(f"  held-out {held['name']}: LEARNED P={sc.precision:.3f} R={sc.recall:.3f} F1={sc.f1:.3f}"
                  f"  |  baseline P={bsc.precision:.3f} R={bsc.recall:.3f} F1={bsc.f1:.3f}")
        mean = lambda xs, a: sum(getattr(s, a) for s in xs) / len(xs)
        print(f"  MEAN held-out: LEARNED P={mean(clf,'precision'):.3f} R={mean(clf,'recall'):.3f} "
              f"F1={mean(clf,'f1'):.3f}  |  baseline P={mean(base,'precision'):.3f} "
              f"R={mean(base,'recall'):.3f} F1={mean(base,'f1'):.3f}")
        result["mean_learned_f1"] = mean(clf, "f1")
        result["mean_baseline_f1"] = mean(base, "f1")

    # Final model on ALL videos (for production / running on a fresh held-out video).
    Xall = [row for v in videos for row in v["X"]]
    yall = [t for v in videos for t in v["y"]]
    final = LogisticRegression().fit(Xall, yall, FEATURE_NAMES)
    weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for w in final.w]))
    print(f"\nfinal model standardized weights (sign/size = influence): {weights}")
    if model_out:
        final.save(model_out)
        print(f"final model (all videos) -> {model_out}")
        result["model_out"] = model_out
    return result


def main() -> None:
    ap = argparse.ArgumentParser(description="Distill Gemini labels into a local rally classifier.")
    ap.add_argument("--manifest", required=True,
                    help="JSON: [{name, trajectory, fps, frame_width, labels}, ...] (labels = Gemini CSV)")
    ap.add_argument("--iou", type=float, default=0.5)
    ap.add_argument("--threshold", type=float, default=0.5, help="classifier probability cutoff")
    ap.add_argument("--model-out", default="output/local_rally_model.json")
    args = ap.parse_args()
    with open(args.manifest, encoding="utf-8") as f:
        specs = json.load(f)
    run(specs, iou=args.iou, threshold=args.threshold, model_out=args.model_out)


if __name__ == "__main__":
    main()
```

20. user (2026-06-10T05:34:45.341Z)

```
No files found
```

21. user (2026-06-10T05:34:46.446Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\quality-iterations\2026-06-multivideo-lovo\README.md:25:GPU never double-booked, race-free via a `kushagra_traj.csv`-exists gate). Manifest: `output/human_lovo_manifest.json`.
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\quality-iterations\2026-06-multivideo-lovo\README.md:76:`output/largetestvideo_vs_human.csv`. Manifest now n=3 (`output/human_lovo_manifest.json`).
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\quality-iterations\2026-06-multivideo-lovo\README.md:241:python -m backend.eval.calibrate_local --manifest output/human_lovo_manifest.json --metric f1   # heuristic LOVO
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\quality-
```

```
iterations\2026-06-multivideo-lovo\README.md:242:python -m backend.eval.rally_seq_proto
--manifest output/human_lovo_manifest.json               # learned prototype
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md:138:[Omitted long matching
line]
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\rally_seq_proto.py:26:
python -m backend.eval.rally_seq_proto --manifest output/human_lovo_manifest.json
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\NEXT_STEPS.md:44:- Eval corpus:
`output/human_lovo_manifest.json` (badminton n=6). Roora CSVs:
`%TEMP%\roora\{pickleball,tabletennis}_rallies.csv`;
```

22. user (2026-06-10T05:34:52.548Z)

```
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval\__init__.py
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval\annotations.py
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval\classifier.py
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval\metrics.py
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval\training.py
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval\calibration.py
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval\gemini_refine.py
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval\audio\__init__.py
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval\audio\e1_probe.py
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval\calibrate_local.py
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval\calibrate_wasb.py
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval\distill_local.py
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval\export_wasb_segments.py
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval\harness.py
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval\regression.py
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval\batch.py
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval\eval_partial_labels.py
C:/Users/avidu/Projects/badminton-highlight-indexer/backend/eval\rally_seq_proto.py
```

23. user (2026-06-10T05:34:55.316Z)

```
EXISTS on disk
.gitignore:39:output/   output/human_lovo_manifest.json
```

24. user (2026-06-10T05:36:17.534Z)

```
 docs/NEXT_STEPS.md | 46 ++++++++++++++++++++++++++++++++++++++++++++++
 1 file changed, 46 insertions(+)
26ee2de docs: next-steps + multisport findings (WASB transfers to table tennis, not pickleball)
```

25. user (2026-06-10T05:36:18.138Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\BACKLOG.md:25:- ? **`yolo_hybrid`
naming** ? the registry key, `yolo_*` config keys, and `yolo-hybrid-*` DB detector tags keep the
"yolo" name for back-compat despite no YOLO remaining. Optional rename. **Revisit:** if/when a
clean break is worth the migration.
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md:243:- **spec.yaml <->
harvest.py coupling:** rule `path` strings dotted-navigate harvest's stage/metric/detail names;
rename a detail in harvest.py and the matching rule silently stops firing. Add rules in
spec.yaml, not code.
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\DEFERRED_HARDENING_PLAN.md:38:-
**Rename the diagnostic `setup.py`** (e.g. `scripts/doctor.py`) so it no longer
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\DEFERRED_HARDENING_PLAN.md:44:-
`setup.py` rename touches docs and any muscle-memory invocations ? grep first.
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\NEXT_STEPS.md:39:3. **Housekeeping:**
**rename repo off "badminton"** (multisport); confirm collector = corpus/event-store system-of-
record.
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md:73:
feature / per-sport calibration). The collector + indexer are already sport-generic. ? **ACTION:
rename the repo off
C:\Users\avidu\Projects\badminton-highlight-
indexer\docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md:154:| 5 | Rename repo off "badminton" | **owner
```

action** ? |
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\decisions\DECISIONS.md:254:-
The `yolo_hybrid` registry key is now a slight misnomer (no YOLO); keep it for back-compat or rename in a follow-up.
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md:75:?   ??? segmenters/   # (renamed/moved from current indexer/)

26. user (2026-06-10T05:36:28.630Z)

```
28:                 CREATE TABLE IF NOT EXISTS videos (
29-                     id TEXT PRIMARY KEY,
30-                     filepath TEXT NOT NULL,
31-                     processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
32-                     status TEXT NOT NULL,
33-                     validation_results TEXT
34-                 )
35-             """)
36-
37-             # Play Segments Table (Extensible metadata JSON)
38-             cursor.execute("""
39:                 CREATE TABLE IF NOT EXISTS play_segments (
40-                     id INTEGER PRIMARY KEY AUTOINCREMENT,
41-                     video_id TEXT NOT NULL,
42-                     start_time REAL NOT NULL,
43-                     end_time REAL NOT NULL,
44-                     duration REAL NOT NULL,
45-                     server TEXT,
46-                     receiver TEXT,
47-                     metadata TEXT,
48-                     FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
49-                 )
50-             """)
51-
52-             # Events Table
53-             cursor.execute("""
54:                 CREATE TABLE IF NOT EXISTS events (
55-                     id INTEGER PRIMARY KEY AUTOINCREMENT,
56-                     segment_id INTEGER NOT NULL,
57-                     timestamp REAL NOT NULL,
58-                     type TEXT NOT NULL,
59-                     player TEXT,
60-                     details TEXT,
61-                     FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE CASCADE
62-                 )
63-             """)
64-
65-             # Versioned migrations live here. PRAGMA user_version is the schema
66-             # contract ? bump it for every change and add an ordered `if version < N`
67-             # block below. Tests assert the expected version, so accidental drift fails CI.
68-             version = cursor.execute("PRAGMA user_version").fetchone()[0]
--
75:                     CREATE TABLE IF NOT EXISTS candidate_segments (
76-                         id INTEGER PRIMARY KEY AUTOINCREMENT,
77-                         video_id TEXT NOT NULL,
78-                         detector TEXT NOT NULL,
79-                         chunk_index INTEGER,
80-                         start_time REAL NOT NULL,
81-                         end_time REAL NOT NULL,
82-                         duration REAL NOT NULL,
83-                         confidence REAL,
84-                         stats TEXT,
85-                         detection_params TEXT,
86-                         confirmed_segment_id INTEGER,
87-                         human_label TEXT,
```

```
88-                         notes TEXT,
89-                         created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

27. user (2026-06-10T05:36:42.411Z)

```
C:/Users/avidu/Projects/badminton-highlight-indexer\run_all_tests.py
C:/Users/avidu/Projects/badminton-highlight-indexer\run_eval.py
C:/Users/avidu/Projects/badminton-highlight-indexer\run_phase3_eval.py
C:/Users/avidu/Projects/badminton-highlight-indexer\run_process_and_stitch.py
C:/Users/avidu/Projects/badminton-highlight-indexer\run_regression.py
```

28. user (2026-06-10T05:36:46.420Z)

```
local_rally_model.json EXISTS on disk (gitignored)
docs/NEXT_STEPS.md
docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md

docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md
```

29. user (2026-06-10T05:36:56.483Z)

```
90      - `@run_regression.py::main` ? `regress_with_provider` vs baseline. Exit codes semantic:
**0=ok, 1=REGRESSION (CI gate), 2=missing DB**. ? `--update-baseline` runs AFTER compare
(snapshots even a regressing run); `::_default_db_path` resolves via
`backend.config.load_config`.
91
92      - `@backend/config/__init__.py` ? ? canonical config import surface. Re-exports
`load_config`, `save_default_config` (from `.loader`) + `AppConfig`, `Config`, `IndexingConfig`,
`ValidationConfig` (from `.models`); `__all__` = exactly those 6. Use `from backend.config
import load_config, AppConfig`.
93      - `@backend/config/models.py` ? Pydantic v2 models; **single source of truth for
defaults**.
94        - `::AppConfig` ? root. Scalars `sport='badminton'`, `ai_provider='gemini'`,
`ai_model='gemini-2.5-flash'`; sections `validation`,`indexing`,`output`,`stitching`
(`default_factory`). `model_config = {extra:'allow', validate_assignment:True}`. ?
`extra='allow'` -> unknown/typo'd config.json keys silently kept (legacy tolerance).
95        - ? `::AppConfig.get` ? legacy dict-compat shim: returns nested **sections as plain
dicts** (`model_dump(mode='json')`, NOT the model) so chained
`config.get("indexing",{}).get(...)` keeps working; falls back to searching a leaf key in any
section (sections discovered dynamically from `model_fields`). ? "bit every validator on real
runs" per docstring ? returning the model breaks `.get`. `::AppConfig.__getitem__` raises
`KeyError` for unknown top-level keys else delegates to `.get`.
96        - `::IndexingConfig` ? segmenter/detector knobs (`motion_threshold=0.08`,
`min_segment_duration=3.0`, `dead_time_merge_gap=2.0`, `chunk_duration=90.0`,
`chunk_overlap=10.0`, `yolo_velocity_threshold=0.15`, `yolo_frame_skip=3`,
`ai_handoff_padding=2.0`, `ai_max_workers=5`, `log/store_yolo_candidates=True`, nested
`tracknet`). ? `default_segmenter='yolo_hybrid'` HERE but config.json overrides to `'gemini'`. ?
`::IndexingConfig.segmenter_must_be_registered` `@field_validator` is a **no-op pass-through** ?
a typo'd segmenter validates fine, only fails later at `segmenter_registry.get(...)`.
97        - `::ValidationConfig` (`min_resolution_width=1280`,`min_resolution_height=720`,`min_f
ps=29.9`,`min_blur_threshold=20.0`,`max_scene_cuts_per_minute=0.5`, nested `relevance`);
`::ValidationRelevanceConfig` (court HSV gates, `court_pixels_min_ratio=0.05`).
`::TrackNetConfig` (WSL invocation contract: `wsl_distro='Ubuntu'`, `repo_dir`,
`conda_env='TrackNetV4'`, `weights_path=''`, `queue_length=5`, `velocity_threshold=0.02`).
`::OutputConfig` (`default_highlights_name`, `db_name='sports_indexer.db'`,
`output_dir='output'` ? CLI `--output-dir` overrides it on the typed model in `main()`).
`::StitchingConfig`
(`begin_padding=0.5`,`end_padding=0.5`,`use_cache=False`,`min_shot_count=1`). `::Sport` Literal
of 5 sports. `::Config = AppConfig` alias.
98      - `@backend/config/loader.py` ? load/save with defaults-merge.
99        - `::DEFAULT_CONFIG_PATH = Path("config.json")` (? CWD-relative, NOT repo-root).
`::_get_builtin_defaults` = `AppConfig().model_dump(...)`.
100       - `::load_config` ? precedence built-in-defaults -> file overrides. ? **shallow top-
level merge** (`{**defaults, **file_data}`): a section in config.json REPLACES the whole
```

defaults dict for that section (Pydantic then re-applies field defaults for missing leaves). Missing/unreadable/parse-error file -> fully-defaulted AppConfig + `[CONFIG WARNING]` (non-fatal).

101   - `::save_default_config` ? writes fresh `config.json`, **overwrites** if present (used by setup.py/`--setup`). `::get_default_config` ? ? transitional plain-dict alias.

102   - `@config.json` ? on-disk config; mirrors `AppConfig`. ? **`indexing.default_segmenter='gemini'`** diverges from model default `'yolo_hybrid'` ? file wins at runtime.

103   - `@setup.py` ? `::init_default_config` ? delegates to `backend.config.save_default_config` (single-source defaults; skips if config.json exists). `::run_pip_install` GPU-aware (`nvidia-smi` -> cu124 else cpu). `::run_diagnostics` (Python?3.8, ffmpeg/ffprobe on PATH, optional torch+CUDA).

104

105   ### 2.1 Segmenters (under pipeline/segmenters) ?

106   - `@backend/pipeline/segmenters/base.py`

107   - `::BasePlaySegmenter` ABC ? `__init__(db, config)`; abstract `name`,`description`,`process_video`. § `process_video(video_id, video_path, player_pool=None, max_frames=None) -> List[dict{id,start_time,end_time,duration,metadata}]`.

108   - `::SegmenterRegistry`, `::segmenter_registry` (singleton). ? **register a new segmenter**: subclass `BasePlaySegmenter`, call `segmenter_registry.register(MyCls)` at module bottom, AND import the module in `@backend/pipeline/segmenters/__init__.py` (registration is an IMPORT side-effect). ? `register()` builds `cls(None, {})` just to read `.name` -> name/description/`__init__` MUST NOT touch `self.db`/`self.config`. ? `list_available()` re-instantiates + swallows exceptions into "No description available." (a throwing `description` is masked).

109   - `@backend/pipeline/segmenters/__init__.py` ? imports all 5 (`motion,gemini,yolo_hybrid,tracknet_hybrid,wasb_hybrid`) = the de-facto registration manifest; re-exports them + `segmenter_registry` via `__all__`.

110   - `@backend/pipeline/segmenters/wasb_hybrid.py::WasbHybridSegmenter` (name `'wasb_hybrid'`) ? WSL WASB substrate (MIT; ships pretrained badminton weights, the LIVE default). Imports `WasbRunner`/`WasbConfig` from `pipeline/detectors/wasb_runner`. Config `indexing.wasb.velocity_threshold(0.02)` + shared `dead_time_merge_gap/min_action_window_duration/ai_handoff_padding/ai_max_workers`. ? whole-video single pass; reuses YOLO-named keys `log_yolo_candidates/store_yolo_candidates`; needs `{PROVIDER}_API_KEY` (missing -> []); `detection_params` has NO `queue_length` (unlike tracknet). ? divergent shot_count: `int(shot_count)` inside try with no `is None` short-circuit (copy-paste drift).

111   - `@backend/pipeline/segmenters/tracknet_hybrid.py::TrackNetHybridSegmenter` (name `'tracknet_hybrid'`) ? copy-paste twin of wasb_hybrid; imports `TrackNetRunner`/`TrackNetConfig` from `pipeline/detectors/tracknet_runner`; `indexing.tracknet.velocity_threshold(0.02)`; `detection_params` adds `queue_length`. ? any P3/P4 fix must be applied to BOTH (and arguably yolo_hybrid).

112   - `@backend/pipeline/segmenters/yolo_hybrid.py::HybridYoloSegmenter` (name `'yolo_hybrid'`, no YOLO remains ? torchvision detector + supervision ByteTrack; ADR-005). `::_PERSON_CLASS_ID=1` ? (ultralytics used 0; wrong value -> zero detections). `::_lazy_load_model` (lazy `torchvision` import so module/tests load without it), `::_extract_action_windows_from_chunk`, `::_make_tracker` (per-chunk ByteTrack ? track IDs chunk-local), `::_DETECTOR_FACTORIES`/`::_DEFAULT_DETECTOR='fasterrcnn_resnet50_fpn'`. ? velocity at `effective_fps = fps/frame_skip` -> `yolo_velocity_threshold` coupled to `frame_skip`; pipelined-vs-sequential P2 (prefetch ffmpeg overlaps sequential GPU); `yolo_tracker`/`bytetrack.yaml` is dead/back-compat. ? `gemini_*` config keys are deprecated aliases still honored.

113   - `@backend/pipeline/segmenters/gemini.py::GeminiPlaySegmenter` (name `'gemini'`) ? pure-AI, no P2. ? chunks only if `duration>chunk_duration`; `max_workers=5` hardcoded; `max_segment_duration=90.0` hardcoded; `debug_duration` silently truncates; `parse_time` accepts colon timestamps (warns); does NOT persist candidate segments (only hybrids feed the fine-tuning table).

114   - `@backend/pipeline/segmenters/motion.py::MotionPlaySegmenter` (name `'motion'`) ? pixel absdiff baseline. ? **metadata + serve/smash/foul/winner events are RANDOM** (random.sample/randint/uniform/choice); only segmenter using `db.add_event` and passing server/receiver; uses `print()`; honors `max_frames`.

115

116   ### 2.2 Detectors & windowing (WSL-quarantined; ABC abstraction complete)

117   - `@backend/pipeline/detectors/base.py` ? DetectorRunner ABC (healthcheck() -> Tuple[bool,str] never-raises; run_predict(...) -> Optional[str] CSV path). Contract deliberately

matches the pre-existing tuple returns and call sites in the hybrids. Rich module docstring includes "how to add a new runner", Linux-native path notes (the de-risk goal), and guidance on threading health into reports. Re-exported from `detectors/__init__.py`. This finishes the WSL de-risk seam (low-regret 6 / review item 1).
118   - `@backend/pipeline/detectors/tracknet_runner.py`
119   - `::TrackNetConfig`(+`.from_indexing_cfg`; ? key drift `wsl_distro` JSON -> `distro` field), `::TrackNetRunner` ? runs TrackNet `predict.py` in WSL.
120   - `::to_wsl_mnt_path` (`C:\x`->`/mnt/c/x`; ? POSIX/`~` passthrough, UNC unhandled), `::TrajectoryPoint`(frame,visible,x,y).
121   - **`::parse_trajectory_csv` @staticmethod** + **`::trajectory_to_action_windows` @staticmethod** = MODEL-AGNOSTIC brain, re-exported verbatim by WasbRunner. ? `weights_path` defaults `''` -> `run_predict` returns None; predict.py only `.mp4/.avi` + hard-names `<stem>_predict.csv`; `visible==False` resets prev (occlusion gap breaks track); `fps<=0->30`, `frame_width<=0->1280` silent; `track_id_count` hardcoded 1; `timeout_sec=0` -> NO timeout (hangs forever). `::_stage_video` "OK" sentinel.
122   - `@backend/pipeline/detectors/wasb_runner.py::WasbRunner` (inherits DetectorRunner) (+`::WasbConfig`, from `indexing.wasb`; ? `weights_path` HAS a real default `wasb_badminton_best.pth.tar` ? no required guard) ? stages `wasb_infer.py` (`::_INFER_WIN`) + video into WSL, runs in `wasb` env. `::parse_trajectory_csv`/`::trajectory_to_action_windows` = `staticmethod(TrackNetRunner.*)` (explicit rebind = the plugin-equivalence point). Output hard-named `<stem>_wasb.csv`. ? `wasb_infer.py` re-staged each run (editing Windows copy inert until staged); `_wsl_bash` duplicated (drift risk). Both runners now satisfy the DetectorRunner ABC for future swappability.
123   - `@backend/pipeline/detectors/wasb_infer.py` (WSL) ? `::run` core. ? imports WASB-SBDT repo modules + torch ? NOT importable on Windows. Reboot-resumable detector cache: `::_DET_FILE='det_raw.jsonl'`,`::_MANIFEST_FILE='manifest.json'`, `::CACHE_VERSION=1`, `::load_and_clean_cache` (reads longest CONTIGUOUS prefix where `w==line index`, rewrites to heal crash-truncated tail), `::manifest_compatible` (? `frames_dir` abspath'd but `weights` compared raw), `::_atomic_write_text/_atomic_write_trajectory`, `::_dets_to_tracker` (xy MUST be np.array), `::_build_cfg` (`runner.gpus=[0]` hardcoded), `::extract_frames` (cv2-PNG SLOW), `::default_cache_dir` (`<stem>_wasbcache` next to out). ? GPU pass resumable; CPU tracker STATEFUL -> always fully re-run; `--fresh` ignores cache.
124   - `@backend/pipeline/detectors/rally_gate.py` ? Gate A net-crossing post-filter (pure, no I/O/model). `::estimate_play_axis` (larger-spread axis; ? ties -> 'x'), `::count_net_crossings` (sign changes vs median net; deadband jitter zone), `::gate_windows`/`::apply_gate` (`min_crossings=2` default; kills single-toss service feeds). `::GatedWindow`(start,end,crossings,visible_points,kept). ? window contract here is `(start,end)` tuples NOT the rich window dict ? caller must adapt. Consumed by calibrate_local / distill_local / export_wasb_segments.
125
126   ### 2.3 Eval & calibration (all pure core; I/O in CLI drivers)
127   - `@backend/eval/metrics.py` ? scoring primitive (THE spine). `::interval_iou`, `::match_intervals` (greedy, deterministic), `::score`->`::Score`(`.as_dict()` = wire shape), `::pr_curve`, `::Match`, `::MatchResult`. ? greedy not Hungarian; empty matches -> 0.0 (not NaN) for mean_iou/boundary errors.
128   - `@backend/eval/annotations.py` ? generic GT loader. `::load_annotations` (`start,end` CSV; RAISES on bad rows/`start>=end`), `::parse_timestamp` (decimal or MM:SS/HH:MM:SS), `::write_scaffold`.
129   - `@backend/eval/calibration.py` ? overfit guard. `::cross_validate` (within-video k-fold; ? defaults `metric='recall'` deliberately), `::leave_one_video_out` (honest cross-video; needs ?2 labelled videos), `::improvement_report` (OPTIMISTIC ? selected on full GT), `::select_best`, `::kfold_indices`, `::pct_improvement` (? from-zero -> `inf`), `::evaluate`. § `PredictFn = Callable[[Config], List[Interval]]` = the plug-in seam. ? `generalization_gap >~0.1` = overfit alarm.
130   - `@backend/eval/calibrate_wasb.py` ? WASB tuning CLI. `::BASELINE=(0.02,2.0,2.0)`, `::default_grid` (45 cfgs), `::make_predict_fn` (parses trajectory once, closes over `trajectory_to_action_windows`), `::load_golden_gts` (§`start_time,end_time` cols; ? SILENTLY skips bad rows ? opposite of annotations.load_annotations). ? `{load_golden_gts, make_predict_fn, BASELINE}` imported by 4 downstream drivers (calibrate_local, export_wasb_segments, gemini_refine, audio/e1_probe).
131   - `@backend/eval/calibrate_local.py` ? distill Gemini->local thresholds (windowing + net-crossing gate, no cloud at runtime). `::local_grid` (180 cfgs), `::make_local_predict_fn` (adds `rally_gate.apply_gate` when `min_crossings>0`), `::build_videos` (? `SystemExit` if a labels file yields no rallies), `::run`/`::main`. § `LocalConfig=(velocity,merge,min_dur,min_crossings)`, `BASELINE_LOCAL=(0.02,2.0,2.0,0)`;

manifest JSON shared with distill_local.
132 - `@backend/eval/export_wasb_segments.py` ? tuned cfg -> golden/slicer CSV + comparison + optional clip cut. `::TUNED=(0.05,1.0,1.0)` (? from ONE video), `::SEG_FIELDS`,`::COMP_FIELDS`, `::build_comparison`, `::seconds_to_mmss`, `::_maybe_slice` (lazy-imports rally_slicer). `--slice` requires `--video`. ? `write_segments_csv` output loads straight into `rally_slicer.load_rally_labels` AND re-imported by gemini_refine ? schema is load-bearing.
133 - `@backend/eval/batch.py` ? corpus aggregation (pure; orchestration is in run_phase3_eval). `::aggregate` (micro pool TP/FP/FN, TP-weighted mean_iou, macro F1), `::default_stages_for` (`auto`; `::_FINAL_ONLY_SEGMENTERS={motion,gemini}`), `::resolve_video_path` (normalizes collector `./data\..` Windows paths), `::format_aggregate`.
134 - `@backend/eval/harness.py` ? DB-pred scorer (no re-run). `::STAGES=('candidates','final')`, `::get_predictions` (candidates->`query_candidate_segments(detector=)`; final->`query_play_segments({})`; ? unknown stage raises ValueError; reused by regression.py), `::evaluate`->`::EvalReport`(`::StageReport`)->`::report_to_dict` (input contract for batch.aggregate), `::format_report_text`.
135 - `@backend/eval/training.py` ? learned-segmenter data layer. `::YOLO_FEATURE_NAMES` (5-d), `::extract_yolo_features` (? reads `row['stats']` dict from DB; missing fields default 0.0), `::label_candidates` (same IoU `match_intervals` as evaluator), `::build_training_set`->(X,y,intervals); `feature_fn=` is the substrate plug-in seam.
136 - `@backend/eval/classifier.py::LogisticRegression` ? numpy-only logreg (no sklearn). `fit/predict_proba/predict/to_dict/from_dict/save/load`. ? `class_weight='balanced'` default; stores feature standardization (mean/std) ? inference MUST use same feature ORDER; `_sigmoid` clips z to [-30,30]; `lr=0.1,epochs=1000,l2=1e-3`; JSON `"model":"logreg"` tag.
137 - `@backend/eval/eval_partial_labels.py` ? score pipeline vs **PARTIAL human labels** (PR #60). `::restrict_to_window` (drop preds outside `[0, last GT end]` so un-labeled-tail detections aren't FPs ? THE partial-label methodology), `::run` (baseline/+gate/tuned/+gate ops table + per-rally diff via `export_wasb_segments.build_comparison`), `::sweep`/`--sweep` (grid vs labels; ? fit-on-self = optimistic), `::DEFAULT_GRID`, `::TUNED=(0.05,1.0,1.0)`. ? `window_end` inferred from `max(GT end)` is WRONG if a video is labelled PAST its last rally ? P1 fix = collector persists `labeled_window` in the manifest. Reuses calibrate_wasb + rally_gate + metrics.
138 - `@backend/eval/rally_seq_proto.py` ? **PROTOTYPE** learned per-FRAME rally segmenter = owned-model **Gen-0 seed** (PR #61). 6 features over the WASB trajectory (`vis_rate, spd_mean, spd_max, cross_rate, x_std, y_std`) -> `classifier.LogisticRegression`, honest LOVO. `::build_frame_arrays`, `::features` (scipy `uniform/maximum_filter1d` windowed; ? speed NOT fps-normalized yet ? 30 vs 60fps), `::labels_for`, `::probs_to_segments` (smooth->threshold->merge/min-dur), `::roc_auc` (rank-based, no sklearn), `::run` (per-video AUC + LOVO-threshold F1 + oracle-threshold F1). ? DIRECTIONAL only (n?5, linear, threshold-transfer unsolved); held-out per-frame AUC 0.83?0.92, **beats the heuristic on multi-court footage** (Kushagra/gBaaddy). Manifest `output/human_lovo_manifest.json` (the n=5 human corpus) shared w/ calibrate_local.
139 - ? **MULTI-COURT / background-game confound (2026-06-08 finding):** WASB tracks EVERY shuttle in frame incl. other courts; videos with background games (Kushagra, gBaaddy) have high dead-time shuttle visibility (41?57% vs 18?22% clean) ? the velocity-windowing heuristic can't find clean rally boundaries (F1 0.16/0.28 vs ~0.75 on single-court). Diagnostic = in-rally÷dead-time visibility "contrast" (?3.6× heuristic works, ?1.9× fails). Lever = **foreground-court isolation** (spatial gate to the prominent court, needs real court-region detection). Academy = multi-court = the target env. See `docs/archives/quality-iterations/2026-06-multivideo-lovo/`.
140 - `@backend/eval/distill_local.py` ? distill Gemini -> a LEARNED classifier (beyond calibrate_local's threshold ceiling). `::FEATURE_NAMES` (5 fps/res-INVARIANT: duration,peak,mean,active_ratio,net_crossings), `::build_candidates` (recall-first proposal), `::predict_rallies` (proba filter then `gemini_refine.merge_overlaps(gap_tol=1.0)`), `::run`/`::main`. consts `PROPOSAL=(0.005,1.0,0.5)`, `BASELINE_LOCAL=(0.02,2.0,2.0)`. ? leave-one-video-out needs ?2 videos; final model trained on all.
141 - `@backend/eval/gemini_refine.py` ? refine WASB candidates with Gemini over SHORT padded clips (precision lever; eval/tuning, NOT the live pipeline). `::merge_overlaps(gap_tol=0.0)` (reused by distill_local with gap_tol=1.0), `::refine_window` (? FRESH genai client per worker ? shared throws socket errors), `::full_video_rallies`, `::run`/`::main` (`--full-video` skips `--trajectory`). ? requires `GEMINI_API_KEY` (raises SystemExit); imports `BASELINE` from calibrate_wasb + `TUNED`/`write_segments_csv`/`seconds_to_mmss` from export_wasb_segments.
142 - `@backend/eval/regression.py` ? CI gate. `::DEFAULT_TOLERANCE=0.05`, `::DEFAULT_BASELINE_PATH=output/regression_baseline.json`, `::regress`,

`::regress_with_provider` (GT via ?`annotation_registry.get(tier)`;
"file"->FileProvider/"vendor"->HumanVendor/"auto"->oracle),
`::save_baseline`/`::load_baselines`, `::RegressionResult`. ? imports `annotation_registry` from
`backend.annotations` NOT `backend.eval.annotations`; gate fires on precision OR recall drop
(F1 reported, not gated); no baseline -> `regressed=False` (silent pass).
143
144   ### 2.3a Audio (E1 probe; NOT adopted for fusion ? ADR-007)
145   - `@backend/pipeline/audio/hit_detector.py` ? classical-onset shuttle-hit detector. Pure
numpy + stdlib `wave` (NO scipy); ffmpeg only for extraction. `::HitEvent`(t,strength),
`::extract_audio` (ffmpeg -> mono 16k PCM WAV), `::load_wav`, `::onset_envelope` (pure-numpy
STFT spectral flux; `band=(lo,hi)` band-limit), `::detect_hits` (adaptive `mean+k·std`;
`k`=sensitivity knob), `::detect_hits_in_video`. ? separate signal ? WASB never modified by
this; per audio-e1-findings recall 100% but discrimination ~2.2×, band-pass null -> NOT adopted
(PR #35).
146   - `@backend/eval/audio/e1_probe.py` ? E1 GO/NO-GO diagnostic (no training/fusion).
`::cluster_hits` (groups thwacks; ? `min_hits>=2` so a lone service-feed hit isn't a rally),
`::run` (discrimination ratio + recall recovery + audio-only P/R/F1), `::main`. reuses
`hit_detector` + `calibrate_wasb.{load_golden_gts,make_predict_fn,BASELINE}` + `metrics`.
147
148   ### 2.4 Stitcher / tools / utils
149   - `@backend/pipeline/stitcher/compiler.py::VideoCompiler` ? `compile_highlights(raw,
segments:[(start,end)], out_name)`. Per-clip re-encode -> `-f concat -c copy`. `::_parse_time`
(mirrors gemini.parse_time; ? unparseable -> 0.0), `::_get_video_duration`. ? if duration probe
fails (0.0) NO boundary validation; `begin_padding`(0.5)/`end_padding`(1.0) defaults; temp clips
in a TemporaryDirectory under output_dir; raises ValueError/FileNotFoundError/RuntimeError; uses
`print()`. ? two-stage codec contract: clips re-encoded with IDENTICAL settings precisely so
final concat can stream-copy ? change a preset and concat may break.

30. user (2026-06-10T05:36:57.674Z)

C:\Users\avidu\Projects\badminton-highlight-indexer\docs\COMMERCIALIZATION.md:43:[Omitted long
matching line]
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\NEXT_STEPS.md:11:rally-vs-non-rally
problem). **Framework = ms-swift, base = Qwen3-VL** (Gemma-4 AMBER/bench) come **later** for the
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\PLATFORM_ARCHITECTURE.md:260:> **The
moat is *not* the model.** Anyone can download Qwen-VL/Gemma. The defensible
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\PLATFORM_ARCHITECTURE.md:269:> are
open, Apache-2.0 models only** (Qwen-VL / InternVL / Gemma 4). Detail in the
C:\Users\avidu\Projects\badminton-highlight-
indexer\docs\PLATFORM_ARCHITECTURE.md:304:**Qwen3-VL-4B/8B (Apache-2.0)**; **Gemma 4 multimodal
(Apache-2.0, 2026)** is a clean
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\PLATFORM_ARCHITECTURE.md:309:- ?
Reject for the flagship: **Gemma 3** ? custom terms with a **viral derivative
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\PLATFORM_ARCHITECTURE.md:311:
permanently bound to Google's *updatable* Prohibited-Use Policy); **Gemma 4's switch to
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\PLATFORM_ARCHITECTURE.md:347:|
Together AI | Managed (tune + serve) | Qwen3-VL / Gemma-3 / Llama-4; image data | verify |
partial | ? adapters downloadable | managed Qwen3-VL + serving |
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\PLATFORM_ARCHITECTURE.md:419:InternVL /
Gemma 4 ? Apache-clean); **(2)** keep the paid API strictly as a **runtime
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\PLATFORM_ARCHITECTURE.md:531:   (Qwen-
VL Apache-2.0 vs Gemma 3 custom license vs ?) and confirm a gated commercial
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\PLATFORM_ARCHITECTURE.md:563:[Omitted
long matching line]
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md:25:
(Ratifies the prior "Apache-2.0 only" to include MIT/BSD.) Reject viral/NC/custom-restrictive:
Gemma 3 (viral),
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md:26:
Gemma 3n (Gemma Terms follow derivatives ? even cleaner reject), Llama 4 (700M-MAU cap),
Commons-Clause repos.
C:\Users\avidu\Projects\badminton-highlight-
indexer\docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md:35:**Framework ? base** (the Gemma-4-vs-ms-swift
clarification): a *framework* is the forge (runs SFT/GRPO, emits
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md:42:
pre-release. **Gemma-4 stays AMBER and bench-only** ? its Apache grant is likely but its

```
Prohibited-Use posture is
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md:93:
target stays reachable without re-architecting. (On-device base stays on the Qwen3-VL-4B Apache
lineage ? *not* Gemma-3-270M.)
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\checkpoints\2026-06-strategy-
foundation\README.md:25:- **Owned-model base = Apache-2.0-clean, video-capable**
(Qwen2.5-VL-7B/32B, Qwen3-VL, Gemma 4); **reject Gemma 3** (viral license clause); **minors
excluded** from the training pool; per-user training needs a separate opt-in.
C:\Users\avidu\Projects\badminton-highlight-
indexer\docs\archives\research\OWNED_MODEL_TRAINING_STUDY.md:22:- ? **Gemma 4 stays AMBER and
bench-only** (NOT "confirmed clean" as the table below says): Apache grant likely but
C:\Users\avidu\Projects\badminton-highlight-
indexer\docs\archives\research\OWNED_MODEL_TRAINING_STUDY.md:37:| **Gen-2 / Option B** | 12?24+
mo, after scaffolding + healthy corpus | end-to-end multimodal VLM:
(frames+audio+pose+trajectory) ? rally [start,end] (+semantics); SFT cold-start then **GRPO/RFT
with a temporal-IoU-vs-golden reward** | **Qwen3-VL-4B-Instruct** (Apache-2.0, on-device)
primary; **Qwen2.5-VL-7B** fallback; **Gemma 4 E4B/12B-Unified** (Apache-2.0 + native
video+audio) end-state | rented 24?48 GB A100/L40S (does NOT fit 8 GB box) |
C:\Users\avidu\Projects\badminton-highlight-
indexer\docs\archives\research\OWNED_MODEL_TRAINING_STUDY.md:98:  Gemma 4 E2B/E4B/12B-Unified
(Apache-2.0 + native video/audio), InternVL3 ?38B (MIT/Apache), ASFormer/
C:\Users\avidu\Projects\badminton-highlight-
indexer\docs\archives\research\OWNED_MODEL_TRAINING_STUDY.md:99:  ActionFormer/TriDet (MIT).
**REJECTED:** Gemma 3 (viral Gemma Terms + PUP), Qwen2.5-VL-3B (NC research),
C:\Users\avidu\Projects\badminton-highlight-
indexer\docs\archives\research\OWNED_MODEL_TRAINING_STUDY.md:102:  (Commons Clause ? use `sj-
li/MS-TCN2` MIT). ? **Gemma 4 size names = E2B/E4B/26B-A4B/31B/12B-Unified ? there is no plain
"4B".**
C:\Users\avidu\Projects\badminton-highlight-
indexer\docs\archives\research\OWNED_MODEL_TRAINING_STUDY.md:127:- **Gemma 4 "clean + native-
video" rests on post-cutoff secondary sources** ? keep AMBER until a raw HF LICENSE
C:\Users\avidu\Projects\badminton-highlight-
indexer\docs\archives\research\OWNED_MODEL_TRAINING_STUDY.md:128:  revision is verified, given
the Gemma-3 rejection. Its timestamp-emission recipe is less documented than Qwen's.
C:\Users\avidu\Projects\badminton-highlight-
indexer\docs\archives\research\OWNED_MODEL_TRAINING_STUDY.md:147:3. Working Gemma-4 timestamp-
emission recipe + exact checkpoint (E4B vs 12B-Unified)?
```

31. user (2026-06-10T05:38:02.068Z)


```
{
    "sport": "badminton",
    "ai_provider": "gemini",
    "ai_model": "gemini-2.5-flash",
    "validation": {
        "min_resolution_width": 1280,
        "min_resolution_height": 720,
        "min_fps": 29.9,
        "min_blur_threshold": 20.0,
        "max_scene_cuts_per_minute": 0.5,
        "relevance": {
            "court_green_lower": [
                35,
                40,
                40
            ],
            "court_green_upper": [
                85,
                255,
                255
            ],
            "court_pixels_min_ratio": 0.05
        }
    },
    "indexing": {
```

```
            "default_segmenter": "gemini",
            "use_gpu": true,
            "motion_threshold": 0.08,
            "min_segment_duration": 3.0,
            "dead_time_merge_gap": 2.0,
            "chunk_duration": 90,
            "chunk_overlap": 10,
            "detector_model": "fasterrcnn_resnet50_fpn",
            "detector_score_threshold": 0.5,
            "yolo_velocity_threshold": 0.15,
            "yolo_frame_skip": 3,
            "yolo_tracker": "bytetrack.yaml",
            "yolo_parallel_chunks": false,
            "yolo_chunk_workers": 2,
            "min_action_window_duration": 2.0,
```

32. user (2026-06-10T05:38:03.354Z)

```
Found 15 files
tests\test_normalize_annotations.py
src\cli\normalize_annotations.py
docs\MANUAL_INGESTION_POSITIONING.md
src\reporting\adapter.py
src\reporting\__init__.py
src\cli\curate.py
docs\INGESTION_PIPELINE.md
src\parsers\cvat\rally_cvat.py
tests\test_cvat_parser.py
docs\annotation\ROORA_BRIEF.md
README.md
tests\test_cli.py
src\cli\ingest.py
docs\DECISIONS.md
tests\test_local_import.py
```

33. user (2026-06-10T05:38:14.027Z)

```
No matches found
```

34. user (2026-06-10T05:38:15.430Z)

```
C:\Users\avidu\Projects\badminton-highlight-
indexer\docs\NEXT_STEPS.md:4:`docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md`. **Latest blow-by-blow:**
memory `current-state.md` / `session-handoff.md`.
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\NEXT_STEPS.md:42:- Roadmap:
`docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md` · Study:
`docs/archives/research/OWNED_MODEL_TRAINING_STUDY.md`
C:\Users\avidu\Projects\badminton-highlight-
indexer\docs\archives\research\OWNED_MODEL_TRAINING_STUDY.md:12:## Update (2026-06-09, post-
PR-#61-review) ? authoritative roadmap is now `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md`
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\quality-
iterations\2026-06-multivideo-lovo\README.md:116:`docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md`
(Gen-0 first), gated on n?5 + owner sign-off.
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\quality-
iterations\2026-06-multivideo-lovo\README.md:253:  + `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md`,
after scaffolding).
```

35. assistant (2026-06-10T05:39:30.566Z)

All evidence gathered. Compiling the structured findings now.

36. user (2026-06-10T05:41:30.768Z)

Structured output provided successfully

37. assistant (2026-06-10T05:41:39.769Z)

Audit complete. Structured findings returned.